Degree Project

Implementing Bayesian Networks for online threat detection

Author: Mauro José Pappaterra
Supervisor: Francesco Flammini
Semester: VT2018
Area: Computer Science

Abstract

Cybersecurity threats have surged in the past decades. Experts agree that conventional security measures will soon not be enough to stop the propagation of more sophisticated and harmful cyberattacks. Recently, there has been a growing interest in mastering the complexity of cybersecurity by adopting methods borrowed from Artificial Intelligence (AI) in order to support automation. Moreover, entire security frameworks, such as DETECT (*Decision Triggering Event Composer and Tracker*), are designed aimed to the automatic and early detection of threats against systems, by using model analysis and recognising sequences of events and other tropes, inherent to attack patterns.

In this project, I concentrate on cybersecurity threat assessment by the translation of Attack Trees (AT) into probabilistic detection models based on Bayesian Networks (BN). I also show how these models can be integrated and dynamically updated as a detection engine in the existing DETECT framework for automated threat detection, hence enabling both offline and online threat assessment. Integration in DETECT is important to allow real-time model execution and evaluation for quantitative threat assessment. Finally, I apply my methodology to some real-world case studies, evaluate models with sample data, perform data sensitivity analyses, then present and discuss the results.

Keywords: Cybersecurity, Bayesian Networks, Threat Detection, Attack Trees, DETECT, Risk Evaluation, Threat Assessment.

Preface

“We are called to be architects of the future, not its victims.”

R. Buckminster Fuller

“Security is not a product, but a process.”

Bruce Schneier

Table of Contents

1 Introduction 1
1.1 Background 4
1.2 Previous research 4
1.3 Problem formulation 4
1.4 Motivation 4
1.5 Research Question 5
1.6 Scope/Limitation 5
1.7 Target group 5
1.8 Outline 6

2 Method **7**
2.1 Scientific approach 7
2.2 Method description 7
2.3 Reliability and Validity 7
2.4 Ethical considerations 8

3 Results **9**
3.1 Literature review on Artificial Intelligence based frameworks and methodologies. 9
3.2 Integrating Bayesian Networks in the DETECT framework 17
3.2.1 Introduction to DETECT 17
3.2.2 The DETECT framework Architecture 18
3.2.3 Bayesian Networks for online threat detection in DETECT 20
3.2.4 Attack Trees. 22
3.2.5 Bayesian Networks. 24
3.2.6 Model-to-model (M2M) transformation proposal: from Attack Trees to Bayesian Networks 25
3.2.7 Data population of the probability tables 28
3.2.8 Transformation of Bayesian Networks to machine-readable XML code. 29

4 Case Studies: Attack Scenarios **33**
4.1 Scenario 1: Attacker gets access to internal system. 33
4.2 Scenario 2: Obtaining username and password. 41
4.3 Scenario 3: Intercepting login information and session hijacking. 48

5 Analysis **55**
5.1 Analysis Scenario 1 55
5.2 Analysis Scenario 2 56
5.3 Analysis Scenario 3 56
5.4 Overall Analysis 57

6 Discussion **59**

7 Conclusion **61**
7.1 Future research 62

Appendices
Appendix 1: Basic Terminology. 63
Appendix 2: Most common cyberattacks in 2017 69
Appendix 3: Scenario 1 XML code 73
Appendix 4: Scenario 2 XML code 75
Appendix 5: Scenario 3 XML code 77
Appendix 6: List of acronyms used in alphabetical order. 81

References 83

1 Introduction

Over the last decades, the Internet has radically changed almost every aspect of our everyday life. Information is power, and on the 21st century cybersecurity has become an important field of study and research. Moreover, data breaches, orchestrated cyberattacks, information leakage, and the violation of privacy have become platitude in current times [1]. Every year, companies around the globe expend millions of dollars in order to guard and secure their digital assets from these constant savaging attacks [2].

In December 2009, Google's headquarter offices in China suffered from a security breach that compromised the private information of thousands of users. Due to the sophistication of these attacks, the company threated to cease operations in the Asian giant. Until today, the Chinese government is regarded as the main suspect for the attacks, as it was later revealed that computer hackers specially targeted the personal email accounts of human right activists in the Asian country [3].

In 2010, a malicious worm called *Stuxnet* infected an entire network of industrial computer systems that were in control of nuclear power facilities in Iran. The intrusion of this worm was very damaging. It specifically targeted the programming logic controllers in the Industrial Control System (ICS) in charge of the automation of different mechanical procedures implemented for the separation of nuclear material. Papers have been published on this infamous intrusion, allegedly orchestrated by the government of the United States, sabotaging the production of energy in Iran, and causing millions of dollars in damages. This is only an example of the havoc a computer virus can cause on a widely implemented ICS. These attacks can impact not only the computer systems within the business industry, but the economy of an entire country [4][5][6].

In the same year, the biggest data leak in the world was taking place. More than 1.7 GB of highly confidential government data were released to the general public by the *Wikileaks* foundation throughout several months. *Wikileaks* was, at the time, the biggest case of information leaking, not only in the 21st century, but in the entire recorded history. Hundreds of thousands of files containing sensitive data were leaked and made available to the public overnight. These files contained highly classified information, such as documentation on the ongoing military intervention of the United States in Afghanistan, Iraq and other countries in the Middle East, intel gathered by the United States government regarding foreign politicians, and even international espionage schemes. The man behind this outrage was the political activist and hacker Julian Assange. Until today, Assange has detention warrants from both the United States and the Swedish government, and he is currently under political asylum at the Ecuadorian embassy in London [7][8].

In July 2013, former American *Central Intelligence Agency* (CIA) agent Edward Joseph Snowden leaked classified information from the US *National Security Agency* (NSA), dismantling an entire mass surveillance scheme with a network involving not only the NSA and the government of the United States, but several European governments and top telecommunication and tech companies as well. Hundreds of thousands of documents were disclosed to the press, and top notch classified information was simultaneously revealed worldwide in a matter of minutes [9].

In July 2015, the adults date website *Ashley Madison* was attacked. Detailed information on its user database was released to the public, including names, addresses, telephone numbers

and even credit card details. *Ashley Madison* is a dating platform website whose primary mission is to facilitate extramarital affairs. The resulting chaos from this security breach if left to the imagination of the reader. The French media alone claimed that more than 12.000 Saudi Arabian *.sa* extension email addresses were released. It is not a trivial to point out that this is a country where adultery can be punishable with death penalty [10].

Most recently, in August 2016, 11.5 million documents from the Panamanian law firm *Mossack-Fonseca* were leaked in what is known as the biggest political scandal of the century. The data leak was simultaneously released by the world press under the name of *The Panama Papers*. The staggering 2.6 TB of compromised information is the largest information leak in recorded history at the time of writing. The data leaked from the guileful firm, revealed an international money laundry network that included thousands of off-shore shell companies, hundreds of well-known names, and billions of dollars in tax evasion. A political scandal with prominent figures, such as the prime minister of Iceland, the president of Argentina, several members of the Saudi Arabia royal family, and the Russian political elite, among other high-profile criminals around the globe [11][12].

The social impact of cybersecurity is one of my main inspiration for writing this thesis project. There is no precedence of anything similar in human history. In the last decade, we have witness how information security has taken a protagonist role on today's political landscape. A new scheme on information war has born. As recent attacks have demonstrated, the problem of safeguarding digital assets is valuable not only for individuals, but for big companies, small businesses and world governments alike. Without a doubt, the future of cybersecurity is gaining ground as a fundamental part of everyday life. Moreover, the amount of data that populates the Internet is rapidly growing in parallel. The *International Data Corporation* estimates a staggering 100% yearly increase in computerized data and calculates the total amount of data online to reach 44 zerabytes (equivalent to 44×10^{21} bytes of data) by the year 2020 [13].

Recent advances in the field of Artificial Intelligence (AI) can be implemented in the development of intelligent cybersecurity frameworks. In order to address the capabilities of AI in cybersecurity an intensive three-day confluence on *Artificial Intelligence and Machine Learning Applied to Cybersecurity* took place in October 2017. The confluence included the IEEE along with several well-known technology companies, academic institutions, and governmental organisms including MIT, IBM, Microsoft, Stockholm University and the United States Department of Defence among others. The attendees identified three major changes in the landscape of cybersecurity in the past years. These major changes include: *motivation*, over the past decades amateur, white hat and enthusiast hacking has shrunk while cyber warfare and criminal organizations have taken over the scene with more sophisticated and elaborated cyberattacks; velocity of *propagation*, as the number of devices connected to the Internet have increased, and different automation tools have been developed to identify vulnerabilities and even execute entire exploits (such as the *Metasploit* framework); last but not least, and as a consequence of the changes mentioned afore, the potential impact of a successful intrusion has increase as more devices are connected online and more IoT devices and other computer systems control everyday tasks, making cyberattacks a potential physical menace. All parties present in the confluence agreed that we are living in times where conventional security measures will soon not be enough to stop the propagation of more

sophisticated and potentially more harmful cyberattacks. The increasing number of devices connected to the Internet add up to the fact that even a small percentage of failures will translate into a significant security hole that will be out of reach for human operators to contain. The implementation of AI and Machine Learning (ML) technology as augmentation of cybersecurity is the most promising solution to this increasing problem according to the atendees. Nonetheless, the report written after the confluence emphasizes the importance in reliability in technology, as well as human operators for the future of AI applied on cybersecurity [14].

The *Escal Institute of Advanced Technologies*, also known as *SANS Institute*, is a private information security company based in the United States, which has been surveying companies on the use of analytics and intelligent tools for threat detection and security automation since 2013. In a recently published result from the *SANS Security Analytics survey* made in December 2016, a pool of 438 participants were surveyed. The candidates were composed by security analysts (37%), IT or Security Managers (24%), IT managers, directors or CTO's (12%) and security managers, directors or CSO's (12%). The survey had a global reach, including organizations worldwide, with top participation from the United States (78%), Europe (40%) and Asia (38%). Some of the surveyed companies with operation in multiple geographical locations, and representing the top industries in different areas, including, in order of relevance: banking and finance, technology, governmental, cybersecurity related, educational, manufacturing, and healthcare among others. The researchers found out that 88% implement analytics to some degree in their security prevention programs, and 33% integrate their analytics functions with SIEM systems. From a 62% percent of candidates that recognized some form of automation for pattern recognition from their analytics, only 3.6% referred as having a fully automated scheme for the development of meaningful patterns from the information collected. Moreover, the researchers concluded that ML is still not widely implemented by security teams worldwide, and only a small percentage of 22% are incorporating any form of ML for threat detection processes. Although the results show that on average system intrusions are detected more rapidly, and threats and incidents are remediated it faster (44% of organizations could successfully quantify improvements), the results suggest an increasing number of attacks and an alarming grow in complexity and sophistication of threats. To summarize, the study shows that even though there is a big improvement in the quantity and quality of data and analytics collected by companies, there are still much to do regarding the utilization of the data for the automatic detection and response against cybersecurity threats [15].

Several studies address the implementation of AI formalisms such as Bayesian Networks (BN), Hidden Markov Chains (HMC), ML, Big Data (BD) analysis and others in the field of cybersecurity [16]. In the past, entire security frameworks, such as DETECT (*Decision Triggering Event Composer and Tracker*), were developed, aimed to the automatic and early detection of threats against systems, by using model analysis and recognising sequences of events and other tropes, inherent to attack patterns [17]. Nevertheless, there are still several gaps in related literature when it comes to the use of statistical and probabilistic models, such as Bayesian Networks in the development of intelligent security frameworks for online threat detection.

1.1 Background

As part of the background description of this report, I would like to introduce some notions, terminologies, frameworks, tools and organizations that would help the reader to better understand the context for this project. The terminology is presented in alphabetical order and can be found on ***Appendix 1***. For all the case studies presented in Chapter 4 I conducted a short background investigation on the most common cyberattacks in 2017, a brief explanation on each of the attacks can be found on ***Appendix 2***. Moreover, ***Appendix 6*** presents a list of all acronyms used throughout the report.

1.2 Previous research

There are a good number of published papers regarding the implementation of Artificial Intelligence on cybersecurity. This is no fortuity, since cybersecurity is a wide subject that purviews an extensive number of fields in computer science (if not all of them). There are papers that address different formalisms and concepts related to Artificial Intelligence and their corresponding application on cybersecurity. These formalisms include Bayesian Networks, Hidden Markov Chains, Machine Learning, Data Mining and Data Analysis among others. Nonetheless, there is a literature gap in the implementation of Bayesian Networks in cybersecurity for online threat detection. I will conduct a literature overview as part of my project and to laid foundations of the system I intend to develop. Previous research on the implementation of BNs for online threat detection will be presented in Chapter 3.1.

1.3 Problem formulation

Cybersecurity and Artificial Intelligence are extremely broad fields that engulf theories and methodologies from different disciplines in Computer Science. My problem formulation narrows down to the analysis of the implementation of statistic and probability models, developed using existing datasets in cybersecurity studies in the context of the DETECT framework, in order to implement an online threat detection engine for cyber physical security. Bayesian Networks (BN) are the methodology of choice, since they allow modelers to take into account the inherent uncertainty in the threat scenarios using some aspects of fuzzy logic.

1.4 Motivation

Online threats and vulnerabilities have exponentially grown along with the Internet, the largest human endeavour on history. To make things worse, virtually every single human task is moving into cyberspace and is now prone to cyberattacks. I consider that technology can foster both power and peril, hence the importance of this topic. As the usage of internet and computers continues to increase exponentially through the past decades, with over 4 billion users word wide, vulnerabilities in cyberspace have now become an everyday threat. [18] Cybersecurity is not only important for the industry in general terms, but for science and society as a whole. Various experts agree that AI is one of the most viable solutions to the

rapidly growing problem of cybersecurity attacks. [14] Even though knowledge on the latest technologies, and on the procedure that attackers implement can significantly improve the defence against such attacks, the intrinsic complexity and ever-changing nature of online threats can render this task extremely difficult. AI methods and the development of a framework can be used to mitigate this problem. By retrieving knowledge from previous studies, previous direct experiences within the same organization or similar organizations, existing datasets, simulations and more, we can infer the course of action taken by attackers and systematize them into a framework. Statistical and probability-based models, such as Bayesian Networks, can be very powerful tools. Based on conjectures derived from Bayes models, it is possible to automatize the best course of action for online defence. [16] My choice is also motivated by an existing gap in related literature that I intend to fulfil with literature research, the development of a Bayesian Network based model for online threat detection and documentation on my findings.

1.5 Research Questions

RQ1.	Which are the current industrially relevant frameworks and methodologies that implement online cyberthreat detection, including the ones specifically based on Machine Learning and Bayesian Networks?
RQ2.	How can Bayesian Network based probabilistic models be used to detect common cyberthreats scenarios and how uncertainties can be managed?
RQ3.	How can intelligent online cyberthreat detection models be developed based on Bayesian Networks and the DETECT framework? How can detection models be applied in real cyberthreat scenarios?

1.6 Scope/Limitation

For this report I will base my research on creating a blueprint for the development of BN based models for online threat detection using the DETECT framework as a point of reference. In this report, I have created an approach to translate ATs into BNs models for stochastic inference. Some of the limitations of this project are related to time and extension of the report.

1.7 Target group

The target group for this report includes network security professionals, software developers, computer users, and any other person that is concerned about the state of modern security today, and interested about the implementation of Artificial Intelligence in cybersecurity. This work aims to be as informative and detailed as possible, in order to offer a good insight on the implementation of AI methods in cybersecurity, more specifically, the use of Bayesian Networks for online threat detection.
This project is particularly targeted to Computer Science students and software developers.

1.8 Outline

In Chapter 1, this current chapter, I introduce my thesis project, including a broad overview, presentation of my research questions and background information necessary to understand the basic notions used in the project. Chapter 2 describes the scientific approach I have implemented in order to answer my research questions. These include literature review and development of the blueprints for a systematic detection framework using Bayesian Networks, based on the DETECT framework, formal threat scenario modelling using Attack Trees and pertinent perturbation tests and further analysis. In Chapter 3 I introduce the results of my research. In Chapter 4 I implemented my proposed solution in three different case studies. Chapter 5 analyses the results of my experiments and draws conclusions. In Chapter 6, I discuss my findings and I will attempt to answer my research questions. Finally, in Chapter 7 I end my report with a conclusion on my findings and how I could have done anything differently. I conclude with reflexions for future research on the matter.

2 Method

2.1 Scientific approach

The scientific approach of this report includes qualitative data, including the evaluation of existing research and technologies and the systematic review of related literature and case studies. I will develop a methodology allowing to build and run Bayesian Network detection models starting from (semi)formal representations of the relevant threat scenarios. These scenarios will be based on structural and behavioural views representations (Attack Trees). The architecture of the existing DETECT framework will be used as a point of reference.

In order to gather quantitative information, an experiment will be conducted on the resulting models developed. I will collect the data from the results in order to analyse it and draw my own conclusions. This will help me determine the relevance of the implementation of Bayesian Networks in online threat detection frameworks.

2.2 Method description

I plan to implement different methods for this project. For the first part of the project, I will conduct a literature research from various sources. I plan to consult a wide variety of papers on the subject. Newer papers will be prioritized over older material, so is the case of papers with more references and from more reputable sources over papers of inferior quality and characteristics. I will look for papers in the following websites: *LNU University Library One Search*, *Google Scholar*, *ACM Digital Library*, *IEEE Xplore Digital Library*. These papers are rather short but contain concise information. The literature research will be also complemented with books and manuals published by well-known editorials, as well as web articles and publications related to cybersecurity and Artificial Intelligence. All references can be found in the last chapter of the project.

For the second part of the project, I will reason theoretically on the application of Bayesian Networks for online threat detection, and discuss a simplified model implementation based on the DETECT framework. This simplified model could possibly be extended to more complex scenarios, for experimentation in case-studies of industrial relevance. For these experiments, the resulting Bayesian Networks will be modelled using reliable software applications such as Netica, GeNie or similar. The resulting data collected from the corresponding test will be gathered and analysed using MATLAB for further examination.

2.3 Reliability and Validity

In order to ensure the reliability of any conjecture presented in this paper, I will describe in detail the procedure to model threat scenarios, and the application of Bayesian Networks for threat detection using the suggested implementation. The models should be easy to replicate, and similar numeric data and outcomes should be obtained. In terms of reliability, all the concepts and technologies applied are well known and trusted tools used in the industry.

Moreover, in order to guarantee the quality of the models created, a sensitivity analysis will be performed, and its results presented to the reader and further analysed.

The literature overview is from trusted resources, such as papers published on *LNU University Library One Search*, *Google Scholar*, *ACM Digital Library*, *IEEE Xplore Digital Library*. All publications that I have consulted have been cited on the reference chapter of this project. I aim for an objective interpretation of the qualitative data, and I will base my analysis on empirical and theoretical research. I will also relay on guidance provided by my thesis supervisor and professors.

2.4 Ethical considerations

The ethical considerations that I need to take into account are primary the distribution of information that in the wrong hands could be misused for deleterious purposes. I firmly believe that information is power, and concealing sensitive information is more harmful than making it public. I hope my contribution sheds some light on the implementation of Bayesian Networks on cybersecurity.

3. Results

3.1 Literature review on Artificial Intelligence based frameworks and methodologies

Artificial Intelligence (AI) encompass a range of different technologies, including Machine Learning (ML), Data Mining (DM), Markov Models (MM) and Bayesian Networks (BN). Various of these methods have been combined for cybersecurity efforts. I have reviewed a brief selection of literature, in order to analyse existing frameworks and methodologies that use these AI techniques for cyberthreat detection.

Situational Awareness

The concept of *situational awareness* is defined by *M.R. Endsley* as "The perception of the elements in the environment within a volume of time and space, the comprehension of their meaning and the projection of their status in the near future". This definition includes not only the perception of relevant data, but the process of interpreting and combining information into solid knowledge in order to make pertinent predictions in the future [19]. Implementing AI in cybersecurity frameworks aims to the augmentation of situational awareness, and the automatic detection of threats.
A systematic literature review made by *U. Frank and J. Brynielsson* for the *Swedish Defence Research Agency (FOI)* reveals the analysis of more than a hundred published articles in situational awareness applied to cybersecurity. This is referred to as *cyber situational awareness,* or *situational awareness* applied on a computerized environment, and it should be used as a complementation of *M. R. Endsley* definition. The term of *situational awareness* is defined in this study as the compilation, processing, and fusion of data, in order to abide a technical evaluation of pieces of information that are related to each other. Then the analyst can make a cognitive analysis of the presented data in order to draw conclusions and decide on the best course of action (informed decision making). The data is collected, fed to a fusion algorithm, and then interpreted by a decision maker. To achieve such awareness, different sensors (machine and human), metrics and other techniques, such as the use of Intrusion Detection Systems (IDS), are implemented in order to obtain the data needed to make an inference. In order to succeed, it is important to measure the level of *situational awareness* of an individual, and to measure how the awareness is maintained in time.
The authors introduced their research highlighting official statements by several countries around the world applying situational awareness in cybersecurity in their national security programs, as officially communicated in conferences and press releases. The surge in the attention on the matter is a result of the lack of implementation of *situational awareness* in cybersecurity, the consequent research gap in the subject, and the pronounce incrementation in cyberattacks worldwide. Although not all countries have come to a single agreement on how to proceed, there are some ideas in common. Mostly, the necessity of enhancing *situational awareness* in order to be able to execute counter measures to mitigate cyberattacks against national assets, such as ICS, critical infrastructures and information systems that could potentially undermine national security and disturb the world peace in general. The results from the literature review indicate that, even though there is plenty of papers on *cyber situational awareness*, there is little empirical research on the matter [20].

Artificial Intelligence applied to Intrusion Detection Systems

An intensive three-day IEEE confluence on *Artificial Intelligence and Machine Learning Applied to Cybersecurity*, that took place on October last year, addressed the rapid growth of technologies and the amount of data available in cyberspace. As a consequence of this rapid surge, the attendees concluded that automated, AI based, cybersecurity methods are necessary for the future of computing. On the other hand, completely autonomous counter-security systems were deprecated. The participants concluded that unsupervised AI and ML algorithms could have devastating consequences if derailed. AI systems should be understood with their flaws, and the latter should be taken into account during the decision-making process. AI models should be used as a complementary tool that extends human cognition, and aids to the routine activity of risk assessments and system monitoring. Moreover, future AI based cybersecurity models should be developed within frameworks. This would allow rapid adaptability to constant changes in the technological landscape of AI, and computer technology in general. The attendees also agreed that recognizing and describing attack patterns, in order to generate representative attack scenarios, will aid AI technology to predict attacks before they take place, including zero-day vulnerabilities. During the conference, it was also suggested that these approaches should be problem-specific, taking into account the uncertain nature of a cyberattack scenarios during their design processes, and be able to handle enormous amount of data [14].

In todays, more immediate, cybersecurity landscape, IDS are used in order to prevent cyberattacks. Some IDS include the use of Data Analysis (DA) and AI methods in order to detect threats. A recent survey in AI methodologies applied on IDS by *A. L. Busczak and E. Guven*, state that DA can be further sub-divided in three categories according to their intended use for threat detection. These categories are the following:

1- ***Misuse-detection*** approaches recognize identifiable attacks by their signature analysis. Analytics are maintained manually on a database. This approach is limited by database data, but does not infer or derive new information. As a consequence, a misuse-detection approach is not very effective for the detection of attacks that exploit zero-day vulnerabilities.

2- ***Anomaly-detection*** approaches implement Data Analysis and is fundamentally based on the set standards of what is expected to be normal behaviour in a network or computer system. Whenever an anomality is detected, the IDS signals are activated. Different systems have different standards and definitions on what is considered to be normal or abnormal activity. As a result, anomaly-detection based approaches vary widely. This is a positive consequence, making uncertain to the attacker which events triggered during an attack are passed undetected and which are not. This approach would, in theory, detect a zero-day vulnerability exploit. On the negative side, some false alarms might be triggered regularly.

3- A combination of both methods, known as ***hybrid-detection*** analytics is most implemented in modern uses of AI for IDS according to the results of the survey conducted by *A. L. Busczak and E. Guven*. Nonetheless, none of the other two approaches

mentioned above are entirely based on signature analysis or anomaly-detection alone [16].

These three approaches mentioned above are used to categorize the analysis of different AI methods for IDS. The same study indicates that after major advances in ANN, ML and DM techniques, AI systems that fall in these categories are trained for all three analytics approaches. Training occurs in supervised, semi-supervised and unsupervised environments. More specifically, the study emphasizes the use of the Predictive Model Markup Language (PMML), an Extensible Markup Language (XML) based model developed by the *Data Mining Group* for storing models for future reference. Moreover, PMML can be used to mine data on the state of the systems, and it supports logistic regression, feed-forward Neural Networks (FFNN), and standardized DM metadata [16].

On the importance of datasets

For both DM and ML approaches in cybersecurity, the data feed to the algorithms is, in essence, the core of the IDS system. Hence it is important to also consider the datasets that are being used to feed these algorithms. *A. L. Busczak and E. Guven* recognize three main datasets from their study. The *Packet-level* dataset is generated with information retrieved from different network tools, packet sniffers, network monitoring applications, IDS and traffic generators. These sources are commonly referred to as *pcap* application interfaces. Examples of these programs the reader might be familiar with are *Wireshark*, *Nmap* and *Snort*. The *NetFlow* dataset contains network traffic information collected from the homonymous Cisco router/switch protocol. The third most used dataset corresponds to the well-known, and publicly available, DARPA datasets, compiled by the homonymous agency DARPA (the Defence Advanced Research Projects Agency) and researchers from MIT (Massachusetts Institute of Technology). The DARPA datasets, from 1998 and 1999 respectively, were harvested from various simulations of network activity that lasted for weeks [16].
Nonetheless, various other datasets have been known to be implemented in the past for AI based IDS. A paper published in 1998 by *J. Cannady* analyses the use of ANNs for a misuse-detection based approach, which implements the attack signatures collected by a *RealSecure* network monitor. The dataset was harvested after simulating around ten thousand events, including three thousand simulated attacks, and collecting information on the packets and data instances generated. The data collected was used to train the ANN to determine if each event corresponded to a normal event or to a cyberattack. Although there are not many details revealed on the experiments, the author claims a high level of detection of root-mean-square 0.07 [21].

Another research that applied ANNs, in this case on an anomaly-detection approach, was carried by P. *Lippmann et al*. For the dataset, Telnet sessions were transcript and statistics were derived using special keywords from a list. The input to one of the ANN implemented retrieved the probability of an attack. Once an attack was identified, a second NNA will try to determine the nature the attack and use the data for further classification. The authors of this research attribute an accuracy rate of 80% [22].

A similar anomaly-detection based research was conducted by A. *Bivens et al.* In this case, the ANNs were trained on the normal function of a system using data such as: most commonly used protocols, normal traffic patterns, most implemented TCP/IP port numbers, and more, that were taken from one of the DARPA datasets. Once the normal patterns of a system were learnt at this unsupervised pre-processing stage, they were used to train a multi-layer perceptron ANN. Data was then classified on clusters implementing groups of packets, with an astonishing 100% detection rate for normal behaviour, but triggering high rates of false positives for certain attacks [23].

As mentioned before, datasets implemented are the core of IDS systems that are based on ML and other AI techniques. A recent paper published in 2017 by *Viegas et al.* regarding the reliability of IDS systems [24], dwells partly on the use of datasets in anomaly-based IDS. The authors consider the following properties to be desirable on a dataset implemented on an IDS:

(i) Must use real network traffic.
(ii) Must present valid scenarios.
(iii) Proper labelling to distinguish regular activity from attacks.
(iv) Must be variant.
(v) Must be correct.
(vi) Easy to update.
(vii) Easy to reproduce.
(viii) Sharable, it should not contain confidential or sensitive data.

Moreover, a recent paper published by *H. Hindy et al.* presents an updated taxonomy on IDS systems, and a survey on the most common used datasets. The study compends 67 different datasets on a period range from 2008 to 2018, including DARPA, KDD and DEFCON among others widely implemented collections of data. After surveying and benchmarking current ML based IDS systems implementing these datasets, the authors concluded that current IDS systems are based on outdated datasets, this render most modern detection tools inadequate to countermeasure modern cyberattacks. What is more, current datasets are not relevant to modern day attack scenarios, and most importantly, are not able to adapt to everchanging scenarios [25].

Application of Bayesian Networks and Dynamic Bayesian Networks in cybersecurity

Apart from ML, other AI techniques, such as Bayesian Networks (BN), are a good answer to the problem of learning from experimental data on a systematic way. The logical underpin of BN is perfect for inferring results from experimentation with partial observations. The association probabilities inferred by BN can be a great approach for the detection of attack signatures. Based on probabilistic relations, we can also try to infer a probable outcome using input states. BNs need to be designed with conditional probability tables (CPT), whose values can be obtained from different sources, including expert knowledge systems or specially tailored algorithms [26].

A survey by *A. L. Busczak and E. Guvens* states that IDS based on BN applied with a misused-detection approach will certainly react to unexpected behaviour. An IDS designed with these characteristics would have a given input in continuous check against a list of attack patterns. [16] There are two major recent experiments. On the first study, *R. Wash et al.* used BNs for Denial of Service (DoS) attacks inference, by analysing and classifying network streams. The result from this study was used for the detection of botnet traffic on the Internet Relay Chat (IRC), with a reported satisfactory result of 93%, and a false positive rate of barely 1.93% [27].
The second study, by *F. Jemili et al.* implemented an IDS framework that used BNs for the detection and classification of anomalies in the network. The public data set, implemented as a knowledge base, was derived from the DARPA 1998 dataset, and it implemented Knowledge Discovery in Database (KDD) process to generate new data. The anomaly detection module identified the following results on different attacks: DoS 89%, Probe (Scan) 99%, R2L 21%, U2R 7%, Other 66%. The researches explain the lower detection results for R2L and U2R due to inferior BN training on these particular attacks in comparison to the more successfully detected attacks [28].

Logical relation represented on attacks models are often deterministic, but this is not the case in real world cyberattacks. An experiment on the use of BN for cybersecurity analysis conducted by P. *Xie et al.* [29] intended to shed light on the use of graphical models translated as BN in non-deterministic, real life scenarios. The authors state that previous approaches to building BN from attack graphs fail to consider uncertainty in real attack situations. In an effort on capturing these uncertainties, they built a BN based on security graphs models, and later applied their prototypes to experimental studies. The paper recommends the application of the following parameters for construction of BN for cyberattacks analysis. The first parameter is to modularize as much as possible, in order to avoid coupling between uncertainties, and to keep modules properly identified and labelled separately. In order to heighten the level of security analysis, most of the values for CPTs used in the BN should be obtained from reputable sources. The last parameter recommend by the authors is that the resulting BN model should not be too sensitive to data perturbation.
In order to better pin the nature of the uncertainties, *P. Xie et al.* identify two mandates that are necessary for an attack to happen. The first mandate is the *physical path*, which relates to network connectivity and the attacker's reachability to the target system. This mandate is directly related to the physical network layout and is specific to every possible network layout in a system. The second mandate, is the very *structure of the attack*, and the vulnerability it intends to exploit in the target system. This mandate includes any abstract knowledge related to the execution of the attack, and the preconditions that need to be present in the target system for the attack to take place. Both mandates can be used along with existing datasets that standardize the metrics for an easier quantitative analysis, such as Common Vulnerability Scoring System (CVSS), in order to infer stochastic variables on the probability of an attack to succeed. But the biggest uncertainty, as the authors point out, is the actions performed by the attacker. Even though all physical and structural prerequisites might be present, this does not necessarily mean that the attack is actually taking place. Hence, their BN design approach considers three different types of uncertainty: the success of an attack, the choices made by the attacker, and the information derived from the IDS sensors.
The *P. Xie et al.* BN model makes use of the intrinsic nature of BN, and effectively solves the problem of uncertainties with a parameter perturbation method approach. As the results in

their experimentation probed, their BN model could successfully apply parameter perturbation methods without modifying the outcomes significantly [29].

A systematic review conducted by *S. Chockalingam el al.* on the use of BN models in cybersecurity [30] intends to fill the research gap in the field, while shedding some light on the shortcomings in the implementation of BNs in cybersecurity. In an effort to identify standardized patterns, the authors researched different papers that presented standardized BN models for cybersecurity applied in different fields of computing. For the paper review, the authors take into account citation details, source of data used for the construction of the BN directed acyclic graphs (DAG), population of CPTs, BN model complexity (based on the number of nodes present), type of threat actor, application sector, and type purpose among others. The scope of the review was narrowed to 17 research papers in the application of BN in cybersecurity. The BN models presented in each of these papers corresponded to different application sectors related to cybersecurity, including ICS, educational systems, banking, health care and software among others.
The approaches that the authors used to validate each of these models included mostly, case studies, cross-validation, Monte Carlo simulations, expert analysis and sensitive data analysis, but 8 out of the 17 models researched were not validated in any way.
The review results point out that the applications of the proposed BN models studied introduced improvements in security systems in all different areas where they were announced, including: risk management, forensic investigation, governmental systems, threat hunting and vulnerability management. On the other hand, the lack of extensive historical data was alluded as one of the main shortcomings in the implementation of BNs. Nonetheless, the report highlights the potential for BNs to overcome this problem, by combining different knowledge bases to infer stochastic values in indeterministic scenarios and other instances where information is not always readily available.
Regarding the main characterization of threat actors, most of the proposed models reviewed emphasize on insider attacks over outsider attacks. Malicious insiders are also prevalent over accidental and random attackers. Nonetheless, the study acknowledges that there is not a BN model that simultaneously integrates both insider and outsider attacks into consideration. The authors point this as a far-reaching shortcoming, as some critical attack scenarios include coordinated efforts from both insider an outsider attackers. Moreover, most BN models analyzed, especially those related to ICS, do not take into account the human factor in cybersecurity, as social engineering is often completely disregarded [30].

Other study by G. *Modelo-Howard et al.* on determining the placement of intrusion detectors in security systems, implements BN models. They present their own BN model to make this evaluation. The BNs implemented by the authors are based on Attack Trees (AT), and they are used in order the infer on distribution of threat detections sensors in large scale and distributed systems. Large scale distributed systems are then measured against the BNs, and their patterns are analysed. They change the placement of detection sensors in the modelled systems, and evaluate the results obtained from the BN models to derive their results [31].

A case study M. *Frigault et al.* on measuring network security using Dynamic Bayesian Network (DBN), intended to model a DBN based framework that would add other ephemeral factors into considerations in threat detection efforts. These factors include the availability of exploit codes or vulnerabilities patches releases at the moment of the attack.

The authors state that threat analysis is not a static environment and should always take in consideration the ever-changing dynamic landscape, where information flow takes a big protagonist role. As information is shared, vulnerabilities can become widely spread on the Internet and easier to exploit. Consequently, in the best of cases, company vendors might release patches to prevent these attacks from spreading any further. Moreover, information from CVSS can also be misleading. For instance, different vulnerabilities that are labelled as low or medium threats could escalate to higher severities values if combined together. For their DBN design, M. *Frigault et al.* consider both a static (solid) domain and a dynamic (changing) domain. All numerical data is retrieved from CVSS, from static scores and from temporarily available dynamic scores, that are updated and maintained on regular basis by the NIST's National Vulnerabilities Database (NVD), and take into consideration updated information on vulnerabilities. The results from the case studies based on their DBN model are positive, and the authors conclude that DBNs can be derived from attack graphs and can be fed with both static and dynamic metrics [32].

A similar framework that implements attack graphs, and takes into consideration dynamic attributes, was developed by *S. Abraham and S. Nair* in their study on predictive frameworks for cybersecurity analytics using ATs. This model relies on MM, and in order to increase the accuracy of the results, it also implements the age of the vulnerability as a covariate. The aim of this proposed framework is to identify systems that might be a stake and need reinforcement in some or several parts of its constitution [33].

Another survey study on system dependability conducted by *D. M. Nicol et al.* dwells on the evaluation of system security, and how stochastic values need to be integrated in order to be able to quantify the security of a system.
The study suggests that this can be achieved by modelling the attacker's behaviours, and by creating a methodology for the evaluation of security configurations (or rather misconfiguration) requirements for the attack to be successful. Furthermore, they evaluate the effects of the attacks rather than the known attack signatures (the latter would cover zero-day vulnerabilities).
This early paper addresses the problem of uncertainty and the indeterministic nature of cyberattacks. For instance, how to make a relevant model of the attacker's behaviours that provide with quantitative evaluation results, and how to provide with the level of detail (or abstraction) that is relevant for each scenario. The authors suggest that breaking down the task in more manageable parts can be a good start. Emphasis is made in the potential of stochastic evaluation techniques, and its application on cybersecurity frameworks. The great desideratum is a fully integrated validation method, which can perform different security evaluations, including attributes such as availability, survivability, confidentiality and non-repudiation. Some of these attributes might be inferred with more traditional quantification methods, others with more stochastic approaches. It is necessary, the authors conclude, to organize and coordinate these efforts on an integrated framework [34].

Efforts in the implementation of BN for security measures are not exclusively reserved to cybersecurity applications. A study by *J. Shin et al.* suggested the development of an integrated BN based security risk model for monitoring nuclear facilities. The suggested model is based in two different parts. One part of the model evaluates the level of the facilities operator's compliance with regulatory guidelines and standards (*activity-quality*

analysis model), while the second part of the model simultaneously evaluates the possible vulnerabilities on the computer systems implemented, and the counter measures used to defend the system under evaluation (*architecture analysis* model).
The *activity-quality analysis* is derived from the observation of the level of attendance or application of regulations and standards. As described by the authors, the regulations are derived from manuals and existing security frameworks for the correct operation of the facilities. For the evaluation, a checklist is developed from the regulations and standards, and handed over to the facilities operators for their completion. After the evaluation of facts reflected on the checklist, the data obtained is indexed and labelled accordingly for quantitative evaluation. The results are then translated into BN models for inference and further evaluation.
The *architecture analysis* model is system-specific and includes a vulnerability analysis of the computer system along with an analysis of the counter measures applied. Different scenarios are taken into consideration. Some of these scenarios include DoS attacks, the introduction of malware into system networks, worms proliferation on the systems networks, data tempering, and any mitigations utilized, such as the use of antivirus in individual computer systems, network monitoring, device validation policies, vulnerabilities patching, and more.
Once both analysis are completed, the two parts of the model are combined together in order to make a complete inference on the state of the system. This resulting unified model is referred by the authors as the *cybersecurity risk* model, and the integration of both activity-quality analysis and architecture analysis model is done using BNs. The authors could successfully use the resulting integrated model to make back propagation queries on the BN, and to derived penetration testing scenarios for attack simulations on the system [35].

Security Frameworks and Threat Modelling

Along with AI techniques, frameworks are being developed in order to systemize and automate IDS. To abide automation and pattern recognition, formal languages are desired in these implementations. For instance, a modular based language called *The Correlated Attack Modeling Language* (CALM) was presented by *S. Cheung, U. Lindqvist* and *M. W. Fong* in 2003, and used on a prototype scenario detection engine [36].
CALM includes attack patterns along with a library of predicates used to describe system states and events. Each module in CALM represents an inference step, moreover different modules can be combined to represent multistep (composite) scenarios. The ultimate goal is to model abstract scenarios that can be shared among different platforms and frameworks. The authors recognized the following 3 tasks on the attack modeling process:

(i) Identify logical attacks within a scenario, decomposed into sensor detectable modules.
(ii) Characterization of logical attacks according to the detection capabilities.
(iii) Recognize and specify relationships among detected attacks.

The authors identify the following challenges when recognizing attack scenarios: the heterogeneous nature of cyberattacks; steps belonging to the same composite scenario might be distributed spatially and temporally; a scenario might be presented with variations; the system could be overwhelm with alarms as a results of false positives (the latter could be

intentional or fortuitous).
Hence, a modelling language must be able to express all three tasks, by complying with a set of properties: the language must be extensive, able to incorporate new attacks and sensors; expressive enough to represent different attacks; unambiguous so as to avoid confusion and enable mechanization; able to reduce the number of identified events and distinguish high-level and low-level security events; efficient on its implementation; finally independent of technology for portability and reuse [36].

A holistic security framework presented by *F. Flammini et al.*, called SENSORAIL, explores the application of AI technology, in combination with wireless sensor networks, for monitoring physical infrastructures, in this particular case railway stations. SENSORAIL dwells on the possible application of BNs and wireless sensors for the prevention of events using the DETECT framework [37]. DETECT *(Decision Triggering Event Composer and Tracker*) was developed aimed to an early, real-time threat detection by matching known attacks patterns and signatures. This framework uses soft computing approaches, such as data fusion and cognitive reasoning, as the core of its detection engine. DETECT uses model analysis and sequence of events recognition in order to recognize known threats patterns, and it can be embedded in existing PSIM (Physical Security Information Management) and SIEM (Security Information and Event Management) systems. A specific Event Description Language (EDL) has been developed for threat descriptions to be stored in an appropriate scenario repository feeding the model-based detection engine. The next sections will introduce DETECT in more details, focusing on essential aspects for integration of BN threat assessment models into the framework [17][37].

3.2 Integrating Bayesian Networks in the DETECT framework

3.2.1 Introduction to DETECT

The DETECT framework was developed mainly for Critical Infrastructure Protection (CIP). A *critical infrastructure* comprises physical assets and communication services that are critical or high-priority for a private or governmental institution. Other fields of applications for the DETECT framework include environment monitoring and control of distributed systems.

The basic idea behind DETECT is that attack scenarios can be inferred from a set of basic events, that can be correlated to build a threat signature, and warn about the threat when it is detected in a specific logical, spatial and temporal combination. To that aim, DETECT includes an attack scenario repository. DETECT aims to early detection, decision support and possible automatic counteraction of threats. The system has been integrated and experimented within existing security management systems for *critical infrastructures*.

For this research, BN are used within the general DETECT framework in order to assess cyberthreats in real time. DETECT processes integrated information possibly enriched with reliability indicators. This feature makes it suitable for online threat detection in presence of uncertainties. The DETECT framework has been demonstrated to be capable to detect complex event-driven scenarios, outlined by heterogeneous events [38][39][40].

3.2.2 The DETECT framework Architecture

In order to detects anomalies, possible threats and vulnerabilities, DETECT counts with a complex model-base logic and online detection engine process. Event occurrences might take place in lapses of time and correlate to other events spatially and temporally. Composite events are envisioned by DETECT based on its complex engine, that does no regard events separately, but in conjunction with previous and posterior events. DETECT's intrinsic event driven architecture recognizes combination of events, and how they are related to each other.

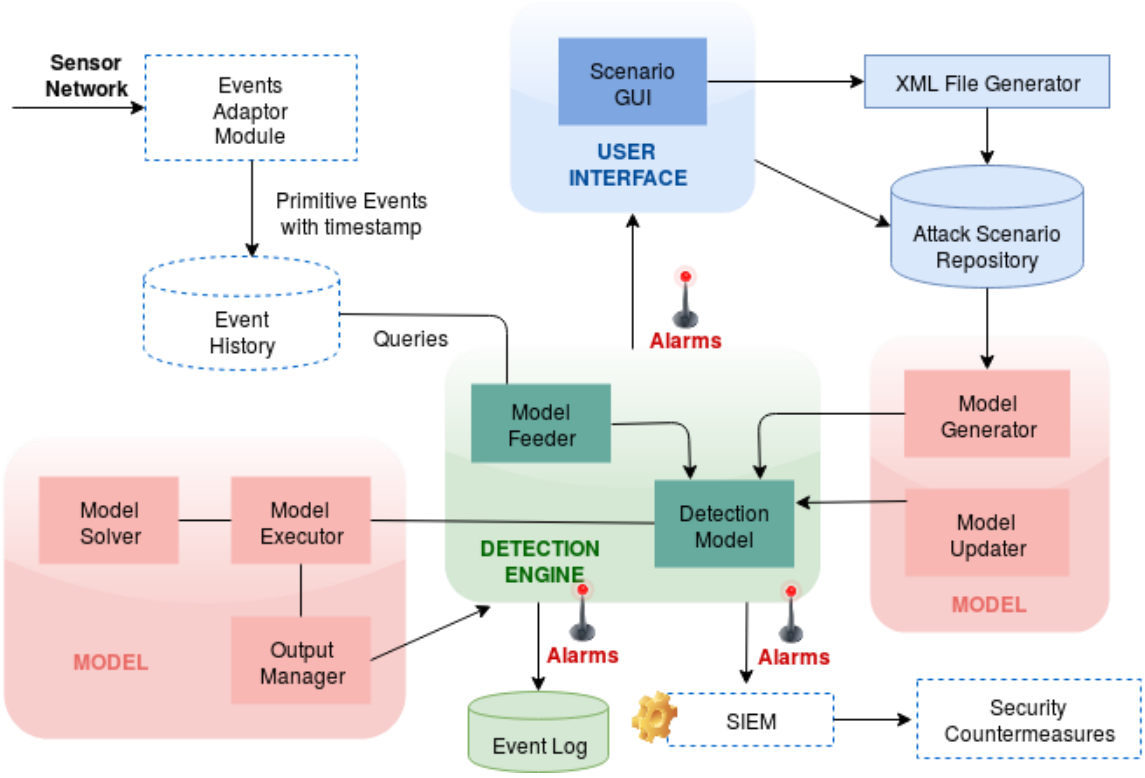

Fig. 3.2.2 – The modular architecture of the DETECT Framework

The architecture of the DETECT framework includes:

***Event History*:** contains a list of all identifiable events that are detected by the system under scrutiny. This database can also be provided by external sources.

***Event Adaptor Module*:** pre-processes the events from the *Event History*. This can also be provided by external sources.

***User Interface*:** provides with an intuitive GUI for the designing and sketching of attack scenarios, control of the detection process, and view of the monitoring status of the system under scrutiny.

***XML File Generator*:** exports the attack scenarios blueprints generated on the Scenario GUI as XML files.

***Attack Scenario Repository*:** indexes all XML files generated for data processing for posterior use.

***Model Generator*:** is responsible for parsing all files from the *Attack Scenario Repository* to EDL, in order to build the correct detection models with the corresponding structures and parameters.

***Model Updater*:** provides with real time update of parameters used on the model.

***Detection Model*:** is one of the main part of the *Detection Engine*, the central unit of the DETECT architecture. This module is in charge of detecting the potentially harmful events that occur on the monitored system. The engine is designed to implement both a deterministic and a heuristic detection model. Nonetheless, the deterministic detection model approach is the only one that has been completely implemented with success.

Model Feeder: controls the representation of the input that is taken from querying on system events from the *Event History* database.

***Model Executor*:** prompts the execution of the model, and activation of the *Model Solver*.

***Model Solver*:** is the module in charge of detecting an executing the model. All logical assumptions inferred from the *Model Feeder* are lodge here. The *Model Solver* detects the composite events taking place.

***Output Manager*:** manages the output from the model before it is sent to the *Detection Engine*.

Event Log: Saves information about the discovered threats. Metadata saved on the *Even Log* includes detailed information such as time of detection, alarm level severity, all events detected for composite threat detection and more.

Previous studies have suggested incorporations of new modules to the DETECT framework. *A. Gaglione* suggested in *Threat Analysis and Detection in Critical Infrastructure Security* the incorporation of advance features including: an *Attack Scenario Management* to store/modify/delete/load scenarios in a persistent manner, *Detection Model Traceability* to track down composite events in an attack scenario after occurrence, and an *Alarm Level Management* module for thoroughly configuring alarm levels and signalling [38].

The architecture of DETECT allows for alarm hierarchies based on risk levels. Alarms are associated to each event that form part of a set of events that are recognised as composite events by the *Detection Engine*. Alarms are sent from the main engine to the SIEM system for operator decision support and/or immediate application of countermeasures. Warnings and alarms are shown on the *User Interface* for acknowledgement by security operators and CERT (Computer Emergency Response Team); they are also saved to the *Event Log* for late retrieval, investigation and any other forensic activities.

3.2.3 Bayesian Networks for online threat detection in DETECT

Previous studies have identified the main requirements for security monitoring systems: threats must be represented by using appropriate modelling formalisms; parameters must be contextualized; online detection must be updated in real time; relevant signalizing of the threats must be implemented using a pertinent alert system; finally, threats must be classified and integrated on the systems database. As shown in the previous section, DETECT provides security operators with all those functionalities [37]. Moreover, this security framework has been used before with deterministic models, in combination with both wireless sensors and distributed systems with notorious success [38][39][40].
This project intends to implement a BN based detection model, and fully integrate it with the rest of the components of the DETECT framework. An unerring security system should provide with identification of threats, vulnerabilities and shortcomings in the system, and also be able to determine if a system is under attack. If the latter is detected, the security system should be able to provide with a course of action to countermeasure the attack. This should be achieved by automatic detection of threat scenarios in real time, and correct association and interconnection of different ongoing composite events detected by the system. A cybersecurity monitoring system should provide operators with identification of threats, vulnerabilities and shortcomings in the system, being able to autonomously determine if the monitored system is under attack. If an attack is detected, the security system should be able to provide with a course of action to countermeasure the attack. However, effective decisions and even more autonomous response must be supported by probabilistic analysis in order to assess and justify false positive and false negative detection probabilities. To that aim, we have worked on the idea of creating a DETECT module addressing BN for stochastic inference, along with other soft computing concepts, in order to safely detect possible attacks and provide operators with indications of threat detection probability. In such a probabilistic model, BN inferences can be used offline to support risk assessment, and online to support (early) detection of ongoing attacks. Therefore, BN utility is manifold, including but not limited to the assessment of online threats, to prompt the user to execute the correct course of action to counter measure an ongoing attack; BN models can also be used to mitigate risks by discovering and "measuring" the impact of specific security weaknesses.

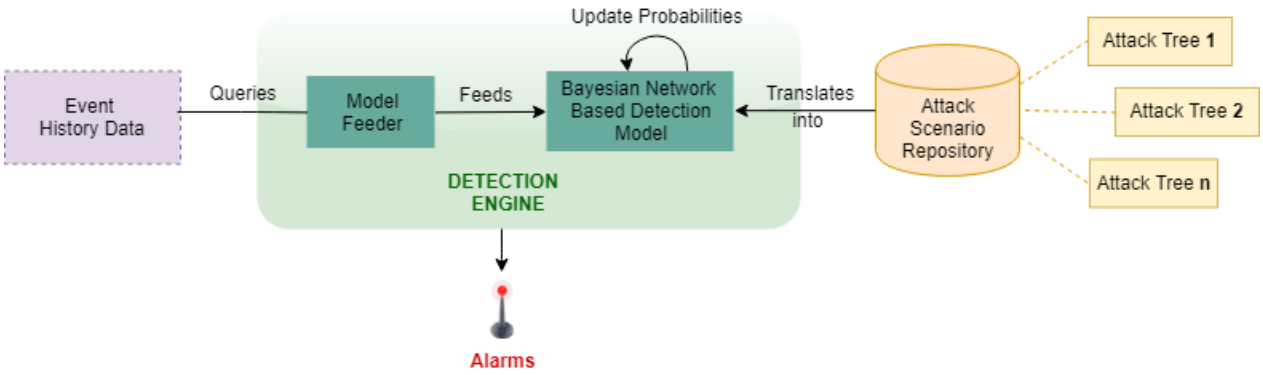

Fig. 3.2.3 – Model for the application of Bayesian Networks for online threat detection in DETECT.

For the creation of a BN based detection model for online threats, a simplified implementation will be considered, based on the compositional DETECT architecture. A

modular implementation on a simplified model provides with both encapsulation and isolation, reducing the possible introduction of error by other parts of the system while simplifying the course of action, and allowing for easy extension and further developments in the future.

The scenarios will be represented with an event description approach, being XML or UML diagrams. For my case study, I decided to model attack scenarios by creating AT diagrams, that will conform an attack scenario repository. These models, first presented by *B. Schneier* in 1999 [41], present a visual overall on the security of a system, and have been proven to be easily translated into BNs [42][43]. Threat models are described in XML to be parsed as machine-readable data.

In online threat assessment, probabilities are updated in real time as events unfold. When cybersecurity relevant events are detected, the *Model Feeder* queries the *Event History* inferring the correlation of the unfolding events inside a defined time window. In our BN assumption, a basic event that is part of a more complex threat scenario can change its state from unknown/estimated (probability less than 1.0) to True (probability equal to 1.0) if it is determined that the event is taking place, and back to unknown/estimated once new inference update determines otherwise. Examples of events can be firewall alerts, intrusion detections, wrong user login attempts, authentication issues, unauthorized behaviours, access to malicious websites, antivirus alarms, vulnerability scanner alerts, software update alerts, email phishing warnings, etc. When detected, these cybersecurity events will cause the corresponding events on the BN detection model to be set to True. In case an event is only indirectly related to security, with a low correlation to threats, it can still be used to update model parameters, e.g. by increasing expected threat occurrence probability, even though no BN node will be set to 'True'.

The final aim of BN detection model is to enable automatic recognition of threats in order to output an alarm. In order to achieve that, as events unfold, the probabilities on the BN are dynamically updated, and provided that the probabilities reach predefined thresholds, warning/alarms/countermeasures are triggered depending on the level of trustworthiness. Therefore, it is essential to associate the appropriate thresholds in the SIEM system. These indicators will be used to inform security managers and operators in security control rooms, also known as Security Operations Centers (SOC), and/or trigger corresponding security countermeasures from the SIEM system. Proper setting of thresholds can be critical and application dependant, but fine-tuning and adequate learning periods can help achieve good trade-offs for best usability and results.

Nevertheless, a number of issues need to be addressed. For instance, parameter context should be taken into consideration. It is necessary to discern the right number of events that conform a composite event. To do this it is important to verify that the lapse of time between each event is not too far apart from each other. The model should assure, by following pertinent guidelines, that all events are part of the same composite event that conform the attack scenario. To simplify, events that are outside a relevant window period, will be reset automatically or by security operators/managers; however, if the events take place inside a relevant window period (e.g. 24h), the detection engine should output the corresponding alarms after making an inference.

On the model proposed in this project, BNs will be obtained from a model-to-model (M2M) transformation from ATs. However, such a transformation only address model structure, while parameters/probabilities should be set directly in BNs. Hence, the CPTs of each generated BN will be populated based on available data. In real-world scenarios, these probabilities will be obtained from diverse sources ranging from known detector reliability to expert judgment and historical data or statistics. Since actual data is not essential to validate the methodology, in this study we describe and test the BN approach with some realistic assumptions and pseudo-data. Regarding false positive and false negative alarms generated by basic detectors, in BNs those uncertainties can be taken into account together with all other uncertainties related to cybersecurity model structure and parameters. In fact, one strength of BN is that they can easily model the so called "noisy" AND-OR in composite events, implementing some aspects of fuzzy logic. Once the BN detection model for a certain threat has been developed, it should be thoroughly tested for sensitivity analysis, data perturbation, value distortion of probabilities, etc., before it can be included into a fully functional DETECT module.

3.2.4 Attack Trees

Attack Trees (AT) were first introduced by *B. Schneier* in 1999. they describe a modelling format based on a tree abstract data type. The root of the tree indicates the attack that is to be performed, each internal node indicates a series of steps or events that need to be done for the attack to be possible, and the leaves, or nodes without children, indicate a series of individual events [41].

Even though the model was originally designed for computer systems, AT models and have been implemented in other fields. There are different applications for ATs, and similar models based on ATs. For instance, there are publications on the use of ATs in ICS systems [44], homeland security [45], and nuclear power plants [46] among others.

The intrinsic flexibility and malleability of ATs make them one of the most implemented models for the representation of attack scenarios [43]. Moreover, there are different variations of ATs. For instance, Fault Trees (FT) are extremely similar in practice, and they have been used in conjunction with BN before [42].

For the model implementation of ATs in conjunction with BN that I intend to present, the following formal definition of an AT is taken into consideration. The definition is based on formal descriptions made by *Schneier, Mauw and Oostdijk* [47], and *Gribaudo et al.* [43] in the quoted papers, and adapted for the convenience of the proposed model.

Definition of an Attack Tree:
Let AT be an attack tree then:

$$AT = \langle V,E \rangle \tag{1}$$

Where $\langle V,E \rangle$ is a tree, with a set of vertices or nodes V that are connected with a set of directed edges E. Where $\{v_i, v_{i+1}, \dots, v_n\} \in V$ and $\forall\, e_i \in E \mid e_i = (v_p, v_c)$, meaning there is an edge that links the parent node v_p with a child node v_c, with both v_p and $v_c \in V$.

Definition of Parent Nodes Subset: Given an AT = <V,E> as defined above, and a node $v_i \in V$, there is a subset of V, named $V_{v_i}^{p} \subset V$, containing all parent nodes for v_i, such that $\forall\ v_p \in V_{v_i}^{p}\{(v_p, v_i)\}$.

Definition of Child Nodes Subset: Given an AT = <V,E> as defined above, and a node $v_i \in V$, there is a subset of V, named $V_{v_i}^{c} \subset V$, containing all children nodes for v_i, such that $\forall\ v_c \in V_{v_i}^{c}\{(v_i, v_c)\}$.

Definition of Leaf Node: Given an AT = <V,E> as defined above. A node $v_{leaf} \in V$ is said to be a leaf node, if its *child node subset* is empty$V_{v_{leaf}}^{c} = \{\ \}$

Definition of Middle Node: Given an AT = <V,E> as defined above. A node $v_{middle} \in V$ is said to be a middle node, if its *child node subset* is not empty$V_{v_{middle}}^{c} \neq \{\ \}$

Definition of Root Node: Given an AT = <V,E> as defined above. A node $v_{root} \in V$ is said to be a root node, if its *parent node subset* is empty$V_{v_{root}}^{p} \neq \{\ \}$

In an AT, all leaf nodes represent a single event that is the start of a path to the execution of an attack, all middle nodes represent an intermediary step of an attack scenario, and the root node, that is unique to each attack tree, represents the final step necessary for an attack to be deemed as successful.

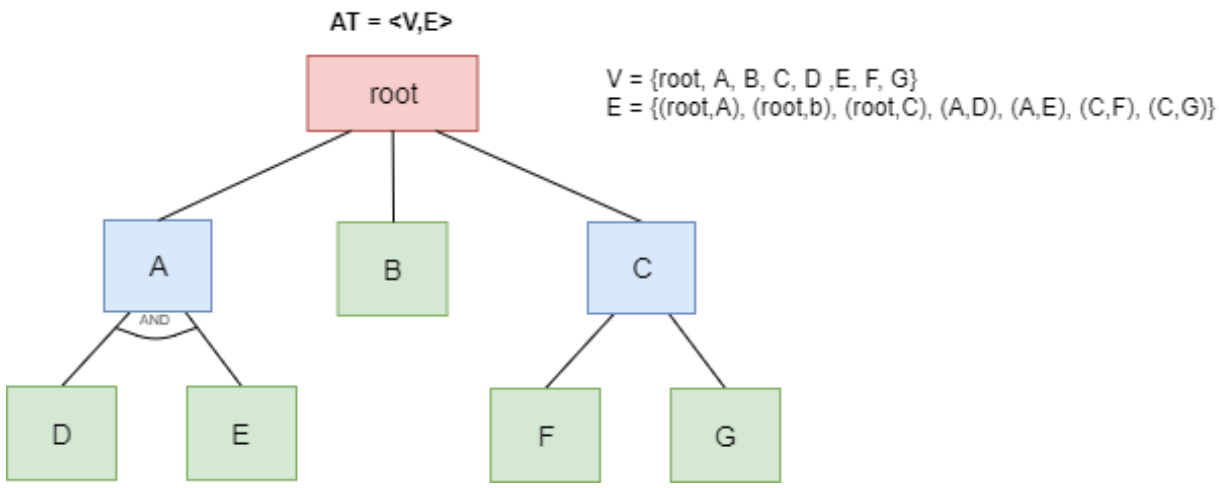

Fig. 3.2.4 – Model for the proposed implementation for an Attack Tree. The red node is the root node, all middle nodes are coloured blue, and all leaves nodes are coloured green.

Operation functions on the AT are defined graphically in proposed AT models. These functions define how a node in the tree is to be accomplished during the execution of an attack. The operation functions taken into consideration in the presented approach include:

- **AND:** All steps indicated on the child node must be accomplished. This is modelled as a line that unites all arcs involved together.
- **OR:** At least one of the steps on the child nodes must be accomplished.
- **XOR:** Exclusive **OR**, at most one of the steps on the child nodes must be accomplished.

For simplicity sake, I have reduced the logical operations to the three presented in the previous page. The function namely **SEQ,** which indicates that a series of nodes must be accomplished on a specific sequence, is represented as a sequence of **AND** nodes that are graphically represented on the given order. The default operation function is the inclusive **OR** that should be tacitly understood when no other operation function is directly stated as a label in between the arcs**.**

3.2.5 Bayesian Networks

The utility of the use of Bayesian Networks (BN) in the proposed model is based on the possibility to infer the probabilities to different paths in the network by observing all circumventing nodes. The logical underpin of BN is based on the well-known Bayes Theorem (1763), which describes how to implement conditional probability axioms to update probabilities as conditions are proven to be true [26].

The Bayes Theorem states that:

$$\Pr(A \mid B) = \frac{Pr(A \wedge B)}{Pr(B)} \quad \text{hence} \quad \Pr(B \mid A) = \frac{Pr(AIB) * Pr(B)}{Pr(A)} \tag{2}$$

The implementation of BN would allow us to make stochastic inferences using this methodology in complex models. A BN graphical model will be represented as directed acyclic graphs (DAGs), in which each node represents a set of variables, and the arcs represent their conditional dependencies or possible state transitions. Probability values can be assigned as weight to the arcs of the graph.

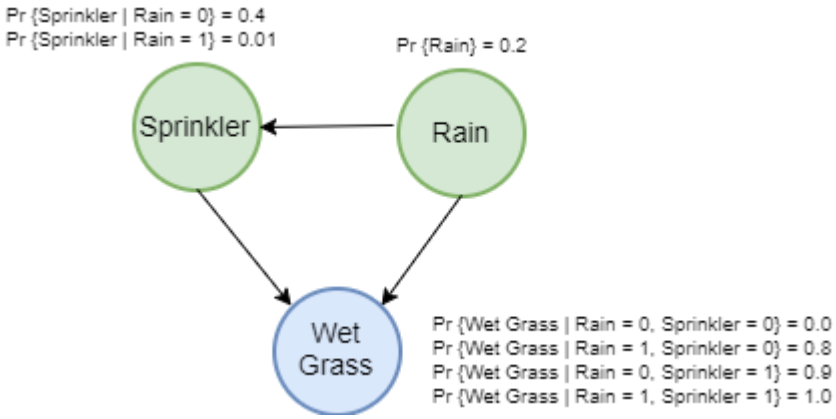

Fig. 3.2.5 – Example of a graphical representation of a Bayesian Network with corresponding probability tables using the classic *wet grass* example.

Mathematically speaking, a BN is simply a set of joined conditional probability distributions. Every node on the BN is associated with a variable X_i. The relations among nodes, which are graphically represented as edges or arcs, represent the connection with all the parent nodes of the given variable. Every node can be associated with the distribution of probabilities, which are represented by a CPT, whose probabilities are conditionally given by all the parent nodes of X_i. This can be denoted as p(X | parents(X)). Following this simplified explanation, an

entire BN, that ends on a target node, can then be represented by a single joined probability distribution [26]:

$$\boldsymbol{p}\ (\boldsymbol{X_1} \dots \boldsymbol{X_n}) = \prod_{\mathbf{1}}^{\boldsymbol{n}} \boldsymbol{p}\ (\boldsymbol{Xi} \mid \boldsymbol{parents}\ (\boldsymbol{Xi})) \qquad (3)$$

3.2.6 Model-to-model (M2M) transformation proposal: from Attack Trees to Bayesian Networks

The M2M transformation proposal that I present in this paper is based on a combination of the approaches by *A. Bobbio et al.* implemented for mapping FTs [42], together with the model presented by *G. Gribaudo et al.* for the analysis of combined ATs using BNs [43]. Both proposed models have been used as a source of inspiration. Nonetheless, I have modified the notation and graphical representation to better suit the implementation for the proposed model in DETECT.

Implemented M2M transformation:

The M2N transformation proposal is described on the flow chart shown in **Fig. 3.2.6.a**. We can identify 4 main levels:

1- On the first level of the process, each leaf node in the existing AT is translated as a root node in a new BN. The calculated probability for each one of the leaves in the AT is then assigned to the equivalent node in the BN.

2- On level two, for each middle node in the AT, an analogous node is created in the BN. Each one of these nodes is then connected to the matching leaf nodes in the original AT model. Linear, diverging and converging connections within the BN nodes indicate the dependency relation among the nodes in the BN.

3- On the third level, the root node in the AT is translated as a non-root node in the BN. Once again, this non-root node is then connected to the matching middle and leaf nodes in the original AT model. The final probability of this non-root node will be inferred based on all the observed events in the network. Once again, linear, diverging and converging connections within the BN nodes indicate the dependency relation among the nodes in the BN.

4- Lastly, on the fourth and last level of the process, each of the node's CPTs must be populated. Operation functions (AND, OR, XOR) assigned in the AT are translated to the BN models with the implementation of equivalent CPT tables on each node in the BN.

As the consequence, the resulting BN is closed to an inverted version of the given AT, as depicted in **Fig. 3.2.6.b**

Hence the versatility of AT for modelling attack systems can be combined with the stochastic inference powers of BN. Once we have an AT model translated into a BN, we can now model each path or branch using event algebras or UML behavioural diagrams to have a formal or semi-formal representation of each scenario. The objective is to define an algorithm to automate the process of building a BN detection model based on the modelled threats. CPTs can be later be refined in order to take into account any *fuzzy correlation*s and take advantage

of the higher modelling power of the BN models. Based on the resulting model, we can start to better define the events/states/conditions need to provide with situation awareness. These factors need to be detected to trigger an early warning during an ongoing attack. This will be exemplified in the following chapters by the implementation of the proposed model in the analysis of case studies of common cyberattacks presented in Chapter 4.

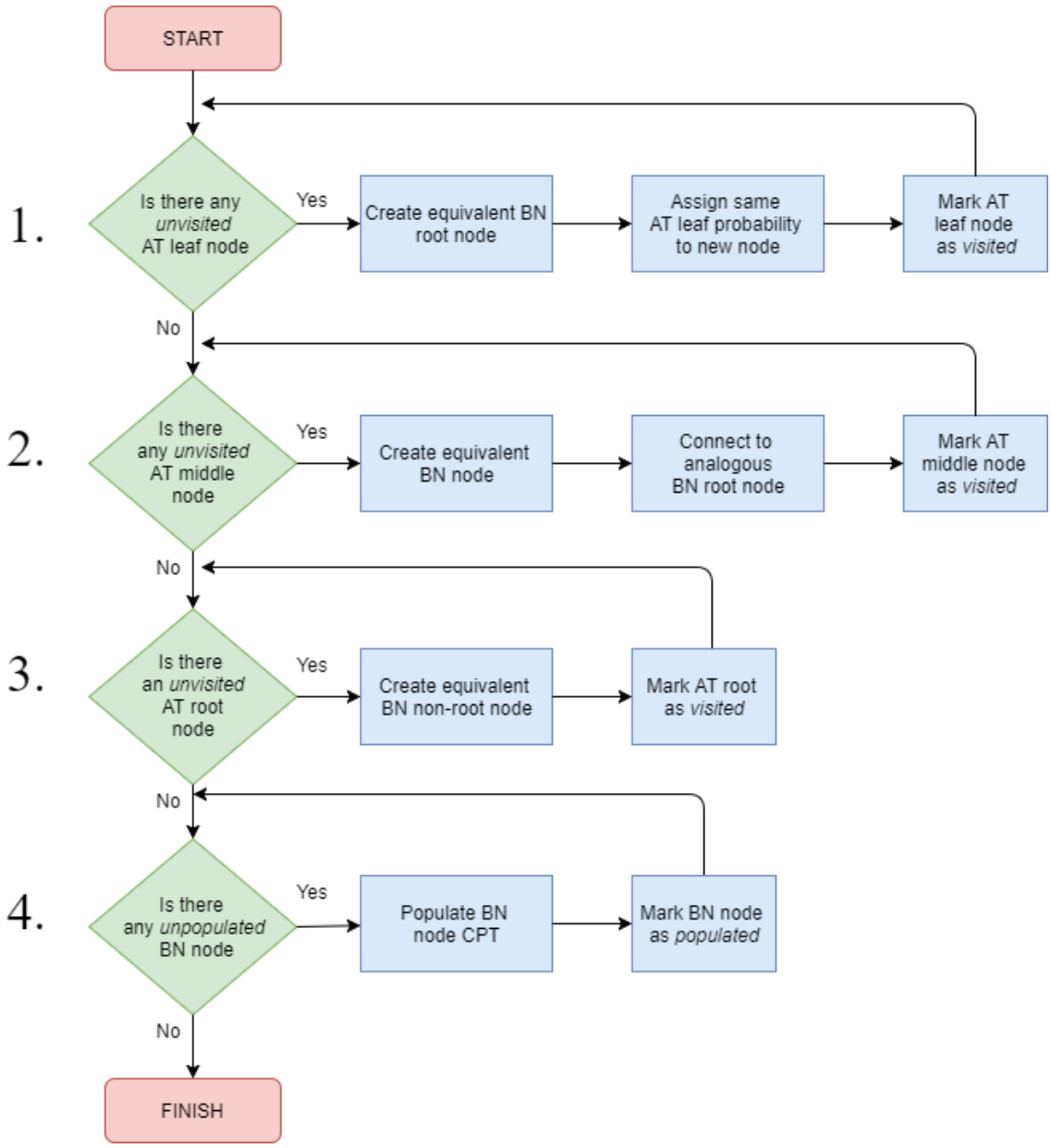

Fig. 3.2.6.a – Flow diagram depicting the proposed M2M transformation procedure.

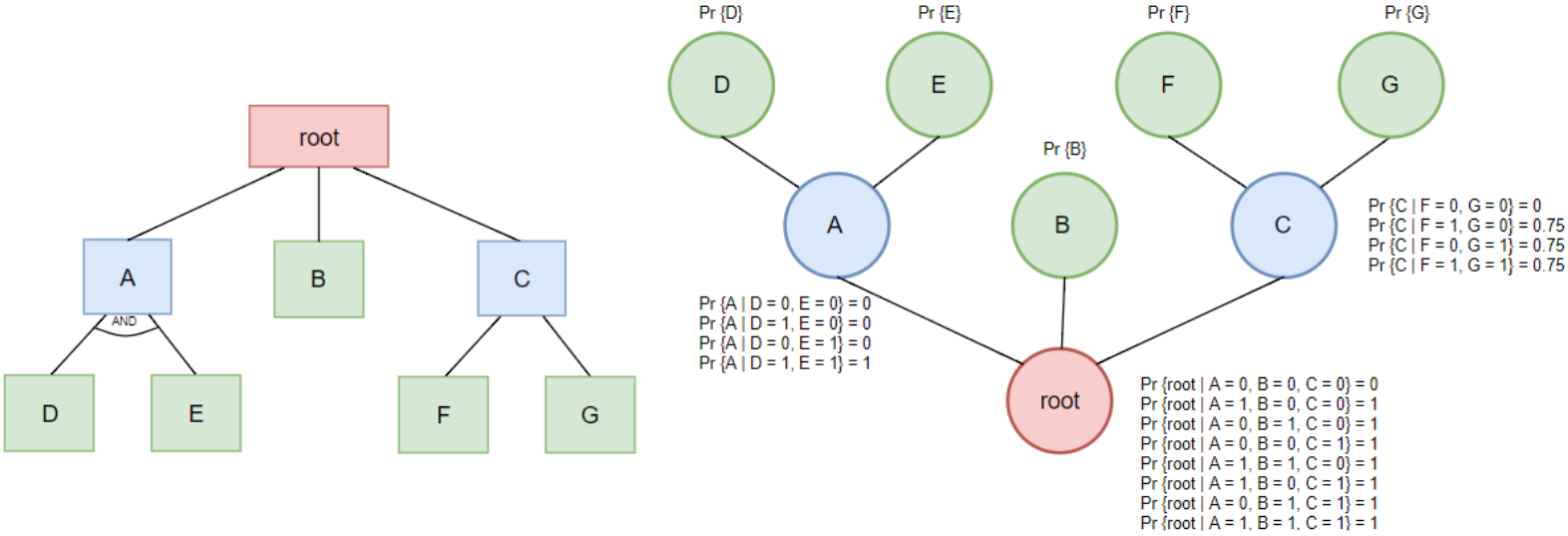

Fig. 3.2.6.b – Example of the implementation of the proposed M2M translation applied, showing an Attack Tree and its corresponding Bayesian Network equivalence.

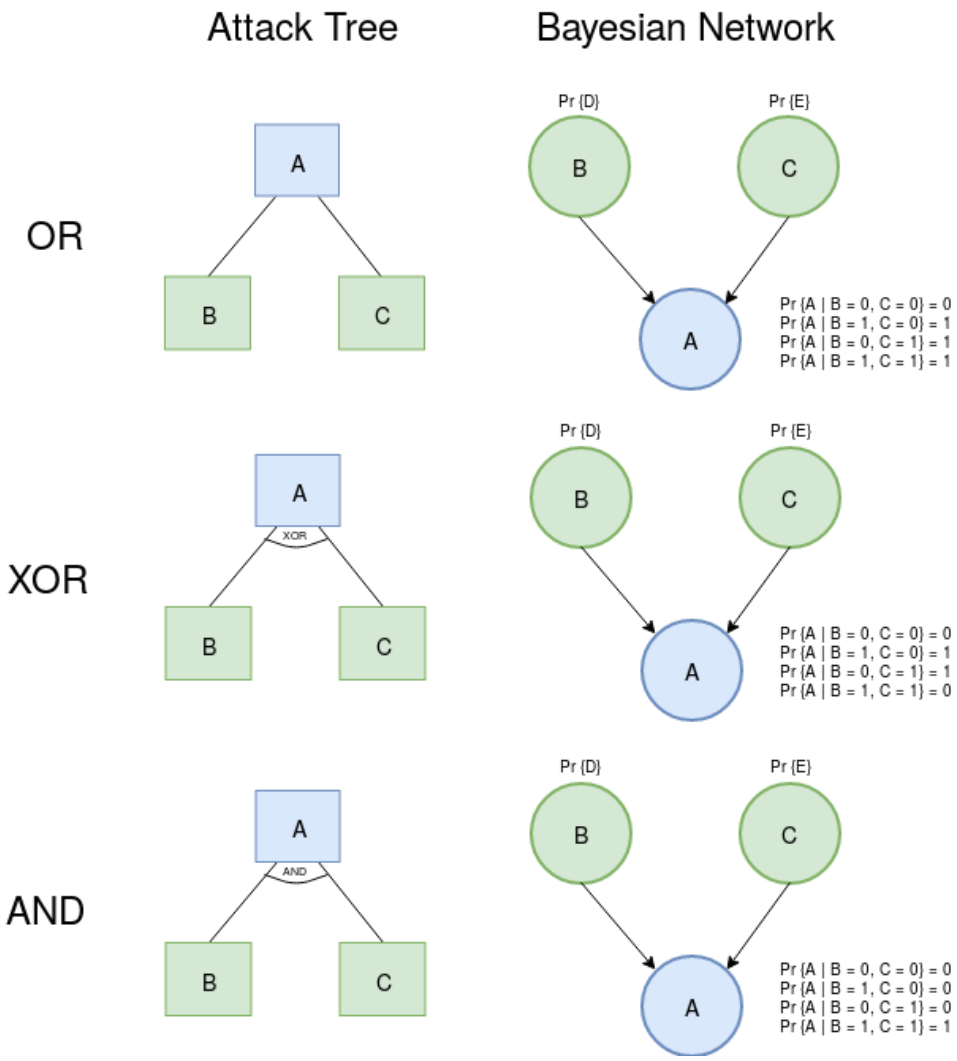

Fig. 3.2.6.c – Example of the implementation of the proposed M2M translation on OR, XOR and AND operation functions. Notice the only visible difference is on the CPT tables of each BN model presented.

3.2.7 Data population of the probability tables

The next step for the finalization of the model, is to populate the CPTs of the resulting BN. Initial probability data can be inferred from publicly available datasets or privately generated data. Once the data has been retrieved, it is necessary to translate into feasible redundant probabilistic values. Depending on the scenario, the nature of the system under protection, and many other variables, the results might vary. It is important to remark, that the probabilities will be refined as more precise data is collected. This can be achieved with data collection on attacks, or other data collection techniques such as honey pots and data harvested from simulations. Using ML techniques, the idea is to progressively tailor the probabilistic inference to each scenario using values that are conjectured from empirical evidence.

As a working example, a study published in 2017 by *Symantec Corporation* titled the *Internet Security Threat Report* (ISTR) was used to estimate the probability rates used for the case studies presented in Chapter 4.2. The study revealed the following information from data collected in the year 2017 [48]:

- *Email phishing rate is 1 in 2995 emails.*
- *Email malware rate is 1 in 412 emails.*
- *From more than 1 billion requests analysed every day, 1 in 13 web-requests lead to malware.*
- *76% of websites contain vulnerabilities, out of which 9% are critical vulnerabilities.*
- *Out of 8718 vulnerabilities discovered in 2017, 4262 were zero-day vulnerabilities.*

This and similar data can be customized to a specific organization and updated dynamically by counting the number of emails sent, websites accessed, etc. For instance, if you trust the above statement "*Email phishing rate is 1 in 2995 emails*", and in your organization you have 1200 emails sent at a certain time, then you can get your custom value for the email phishing probability as:

$$1 - \prod_{1}^{1200} (1 - \frac{1}{2995}) \tag{4}$$

In this formula $(1 - \frac{1}{2995})$ would be the probability of not having phishing in a single email, whereas the production refers to the probability of not having phishing in any of the 1200 emails (assuming they are not correlated). One minus the production is then the probability of having phishing after 1200 emails.

In other words, it is possible to update in real-time that probability by counting the number of emails received in the organization at any time. The same holds for the other parameters like website access. The SIEM system can be configured to monitor those parameters and provide updates to DETECT and hence to the BN detection model.

3.2.8 Transformation of Bayesian Networks to machine-readable XML code

In accordance to the proposed architecture that is based the DETECT framework, it is a requisite to translate the proposed models into a machine-readable format. Following this, the models can be stored in the *Attack Scenario Repository*, and automatically translated into a BN model which can be appended to the proposed *Detection Engine*.

My proposal for achieving this is using XML language. Apart from its reliability, malleability and easy parsing; it has been implemented in DETECT and other security frameworks as shown by *A. L. Busczak and E. Guven*'s survey [16].

The transformation procedure further described below and depicted in **Fig. 3.2.8** .

Transformation procedure:

For each resulting BN that is to be attached to the *Attack Scenario Repository* a new *bayes_network* XML element will be created. There are 4 big levels we can identified in the process:

1- On the first level, for each node on the BN define an element *node*, with an attributes *type* indicating if the node is a leaf, middle or root node, an attribute *id* indicating a unique label to identify the node. A description label can be entered as a simple text inside the node element.

2- On the second level, for each node relation in the BN define an XML element *relation*, with the attributes *parent* and *child*, indicating the nodes involved in the relation using each of the nodes unique *id* attribute, and an attribute *configuration* to explicitly define the configuration of the relation (AND, OR or XOR).

3- On the third level, for each node that is a leaf node, create and populate the corresponding CPT. Define element *probability* with attribute *node* indicating the leaf node using its unique *id* attribute of the node. The, for each state on the probability table create an XML element *state* with a *label* attribute indicating an identifier for the state. Finally, enter the probability value as a float between 0.0 and 1.0 inside the element.

4- Lastly, for each non-deterministic middle node on the BN, define an XML element *probability*, with the attribute *node* indicating the middle node using its unique *id* attribute. Inside the *probability* element define an XML element *conditional* for each node that conditions the probability, with an attribute *node* indicating the conditional node using its unique *id* attribute. For each state on the probability table create an XML element *state* with a *label* attribute indicating an identifier for the state. Finally, enter the probability value as a float between 0.0 and 1.0 inside the element.

Each XML file will be saved in the *Attack Scenario Repository*. An algorithm can be defined to parse each XML file into a working BN that will be implemented in the *Detection Engine* in the proposed model.

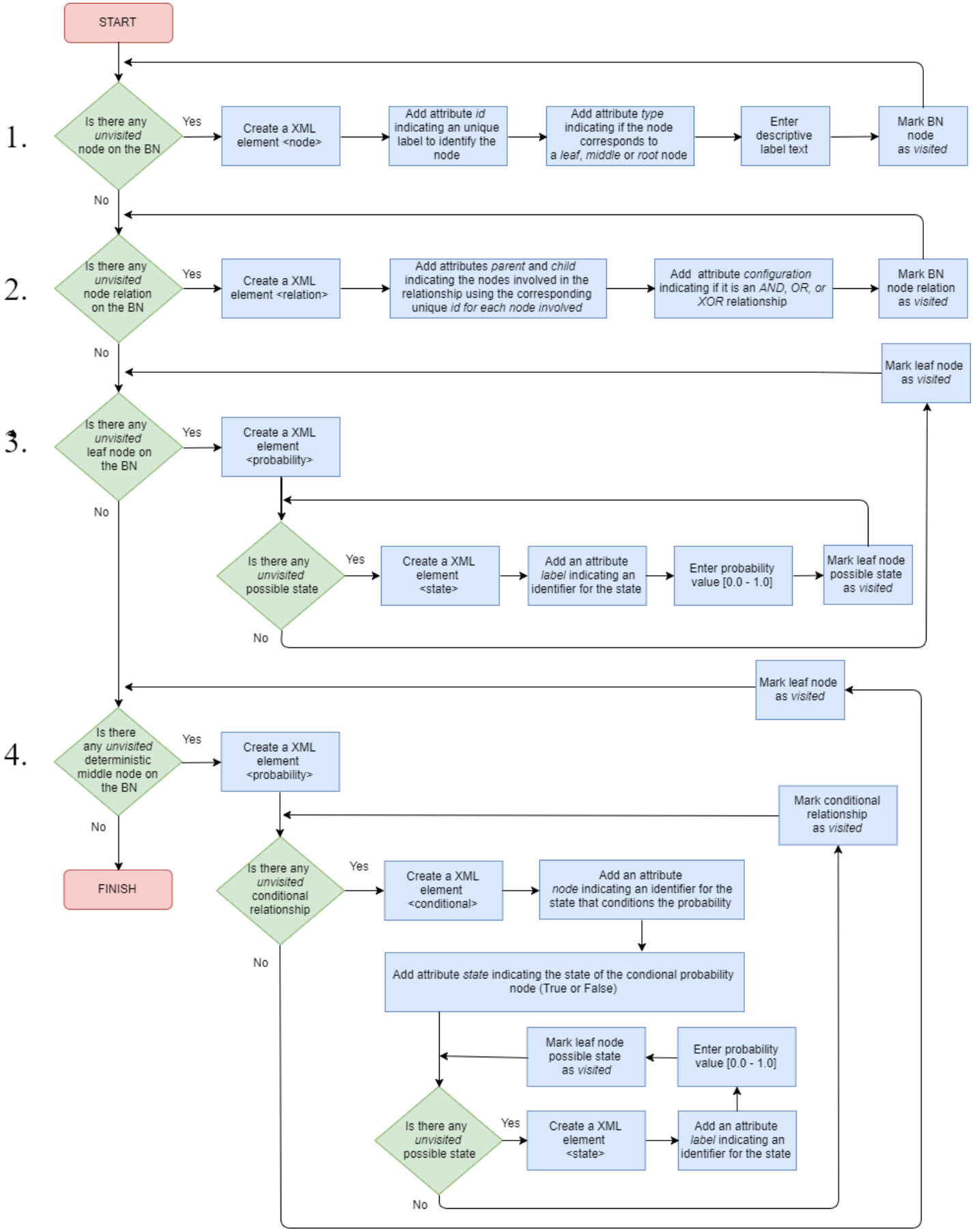

Fig. 3.2.8 – Diagram depicting the proposed BN to machine-readable XML code transformation procedure.

The following code snippet presented reports the example BN converted from AT shown in **Fig. 3.2.6.b** parsed into machine-readable XML code:

```
<?xml version="1.0" encoding="UTF-8"?>

<bayes_network>

  <!-- Define Nodes-->
  <node type="leaf" id="B">B Description</node>
  <node type="leaf" id="D">D Description</node>
  <node type="leaf" id="E">E Description</node>
  <node type="leaf" id="F">F Description</node>
  <node type="leaf" id="G">G Description</node>
  <node type="middle" id="A">A Description</node>
  <node type="middle" id="C">C Description</node>
  <node type="root" id="root">Root Description</node>

  <!-- Assign Node Relations -->
  <relation parent="D" child="A" configuration="AND"></relation>
  <relation parent="E" child="A" configuration="AND"></relation>
  <relation parent="F" child="C" configuration="OR"></relation>
  <relation parent="G" child="C" configuration="OR"></relation>
  <relation parent="A" child="root" configuration="OR"></relation>
  <relation parent="B" child="root" configuration="OR"></relation>
  <relation parent="C" child="root" configuration="OR"></relation>

  <!-- Populate Leave Nodes CPTs -->
  <probability node = "D">
    <state label='True'>0.5</state>
    <state label='False'>0.5</state>
  </probability>

  <probability node = "E">
    <state label='True'>0.1</state>
    <state label='False'>0.9</state>
  </probability>

  <probability node = "F">
    <state label='True'>0.6</state>
    <state label='False'>0.4</state>
  </probability>

  <probability node = "G">
    <state label='True'>0.8</state>
    <state label='False'>0.2</state>
  </probability>

  <!-- Populate Middle Nodes CPTs -->
  <probability node="C">

    <conditional node= "F" state="True">
      <state label='True'>0.75</state>
      <state label='False'>0.25</state>
    </conditional>

    <conditional node= "F" state="False">
      <state label='True'>1.0</state>
      <state label='False'>0.0</state>
    </conditional>
```

```
    <conditional node= "G" state="True">
      <state label='True'>0.75</state>
      <state label='False'>0.25</state>
    </conditional>

    <conditional node= "G" state="False">
      <state label='True'>1.0</state>
      <state label='False'>0.0</state>
    </conditional>

  </probability>
</bayes_network>
```


‘

4. Case Studies: Attack Scenarios

Based on my brief research on most common cyberattacks in 2017 presented on ***Appendix 2*** I have derived models of different attack scenarios in order to test and demonstrate the proposed implementation presented. Each scenario is modelled as an Attack Tree and translated to a Bayesian Network model using the M2M proposed in Chapter 3.2. Since actual data is not essential to validate the methodology, CPTs are populated using realistic assumptions and pseudo-data. Each of the models is then tested for perturbation and relevant results.

4.1 Scenario 1: Attacker gets Access to Internal System

Brief description of the scenario and AT

This is a scenario for an authentication violation attack. This sort of attack ranks second on OWASP's top 10 vulnerabilities in 2017. This is an online threat detection scenario of a possible Authentication Violation cyberattack. The AT presented below displays a simplified version of the scenario for demonstration purposes.

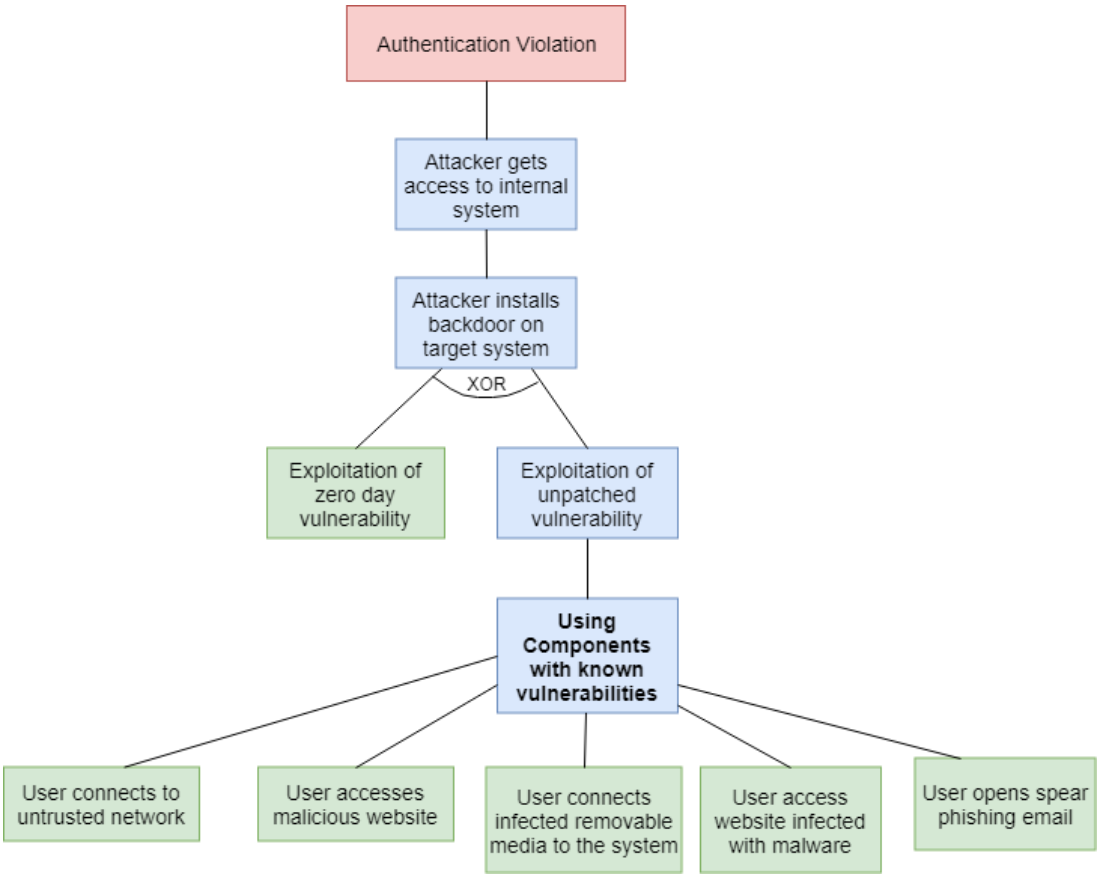

Fig 4.1.a – Scenario for an Authentication Violation attack modelled as an Attack Tree

Values for Static Assessment

In order to make a static assessment, it is necessary to assign some probabilistic values to the leave nodes as well as sensor assumption for the detection of the events. These values can be obtained from reputable sources online or could be tailored to a company or organization data

obtained over a reasonable period of time. For the purpose of this paper, a study published in 2017 by *Symantec Corporation* titled the *Internet Security Threat Report* (ISTR) was used to estimate the probability rates. [48] From this data collected for an entire year, and with some assumptions for the sake of simplicity, we have estimated the daily probabilities for presenting the scenario as shown in **Table 4.1.a** and **Table 4.1.b** . **Table 4.1.a** also displays possible detectors to be used in order to recognize if any part of the scenario has taken place to update BN parameters accordingly.

Leaf node	Identifying Acronym	Estimated probability	Possible Detection Sensors
Exploitation of Zero-Day Vulnerability	ZDV	0.03	-Anomaly detection based IDS -User Level Endpoint Monitoring
User Connects to Untrusted Network	UN	0.24	-IDS -SSL Certificate missing/rejected -Netflow Analysis -Firewall
User Accesses Malicious Website	MW	0.08	-IDS -SSL Certificate missing/rejected -Unexpected flow of data
User Connects Infected Removable Media to the System	IM	0.02	-IDS -Antivirus -System Event Logs
User Accesses Website Infected with Malware	IW	0.09	-IDS -Web Browser Plugin
User Opens Spear Phishing Email	SPE	0.03	-Human -User Level Endpoint Monitoring

Table 4.1.a – Assigned Probabilities and sensor assumptions for the scenario

Middle Node	Identifying Acronym	Estimated Probability
Exploitation of unpatched vulnerability	EUV	0.60
Attacker installs backdoor on target system	BD	0.85
Attacker gets access to internal system	AIC	0.90

Table 4.1.b – Assigned Probabilities for non-deterministic middle nodes

Conversion to BN and machine-readable XML code

Following the proposed M2M transformation, we convert the given AT into a BN, and populate the CPT with the proposed values presented. The results are displayed on the following BN:

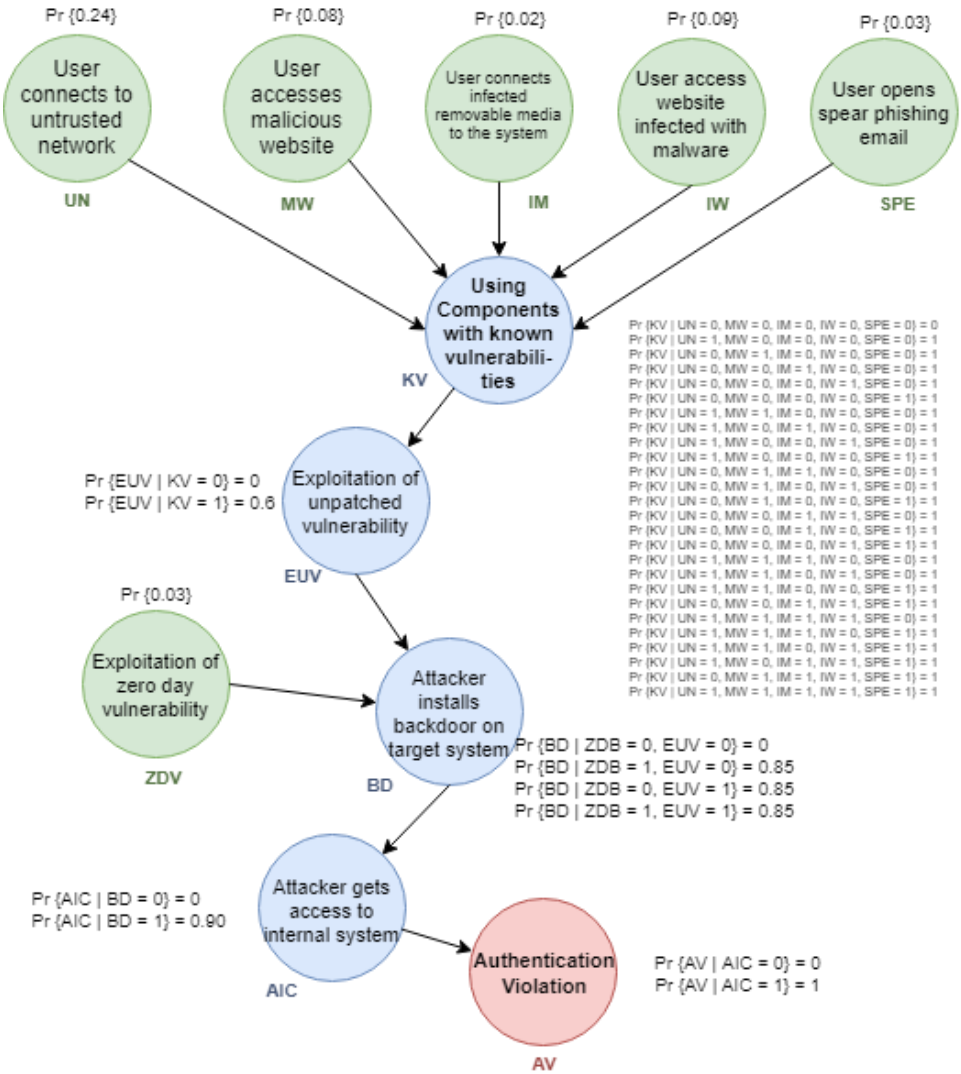

Fig 4.1.b – Resulting BN using the proposed M2M transformation model. Notice that only probabilities for True are represented on each leaf node.

The machine-readable XML code for this scenario can be found on ***Appendix 3***.

BN perturbation test

In order to make an assessment on the sensibility of the BN generated, two different sensitivity tests were performed.

The first perturbance test applies relative variations. To do this I modified all leave nodes one at the time, by increasing/decreasing the estimated probabilities values by 5%, 10%, 15%, 20%, 25% and up to 50% relative to the originally estimated value (multiplication).

Such that:

$$\textit{test value} = \textit{estimated value} + (\textit{estimated value} * \textit{percentage}\ \%) \qquad (5)$$

Relative variation perturbation test results for all leave nodes are presented in **Table 4.1.c** , **Fig 4.1.d** , results for middle nodes are presented in **Table 4.1.d** , **Fig 4.1.e** .

The second perturbance test applies absolute variations. The approach is similar to the first test, but this time the estimated probability values are increased/decreased by 5%, 10%, 15%, 20%, 25% and up to 50% (simple addition). Such that:

$$\textit{test value} = \textit{estimated value} + \textit{percentage}\ \% \qquad (6)$$

Absolute variation perturbation test results for all leave nodes are presented in **Table 4.1.e** **Fig 4.1.f** , results for middle nodes are presented in
Table 4.1.f **Fig 4.1.g** Test results are further discussed in Chapter 5 and Chapter 6.

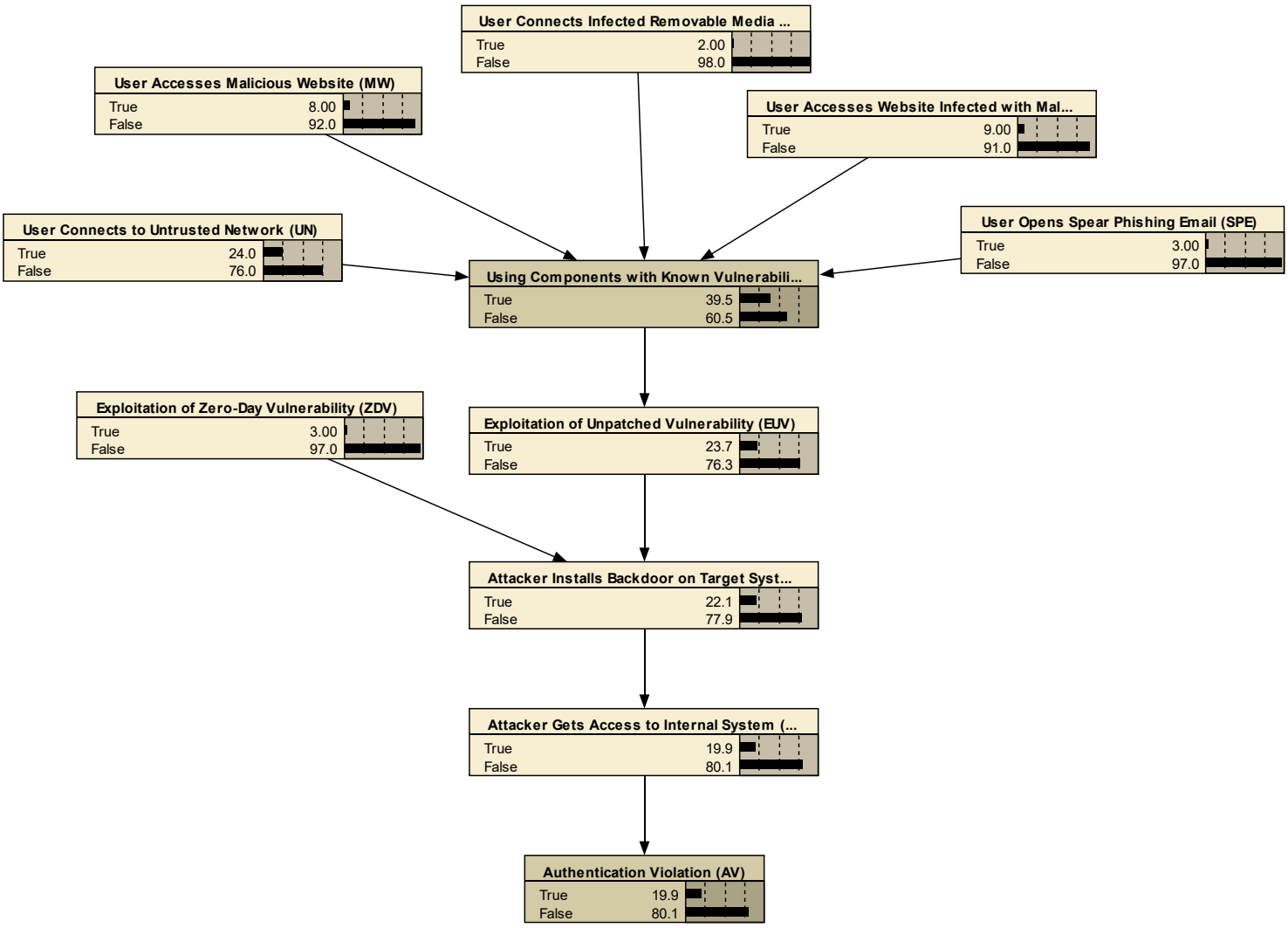

Fig 4.1.c – Resulting BN for scenario 1, modelled using Netica.

Node	ZDV		UN		MW	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	1.5	19	12	15.6	4	18.7
-25%	2.25	19.5	18	17.8	6	19.3
-20%	2.4	19.5	19.2	18.2	6.4	19.4
-10%	2.7	19.7	21.6	19	7.2	19.7
0%	3	19.9	24	19.9	8	19.9
10%	3.3	20.1	26.4	20.7	8.8	20.1
20%	3.6	20.2	28.8	21.6	9.6	20.4
25%	3.75	20.3	30	22	10	20.5
50%	4.5	20.8	36	24.1	12	21.1

Node	IM		IW		SPE	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	1	19.6	4.5	18.6	1.5	19.5
-25%	1.5	19.8	6.75	19.2	2.25	19.7
-20%	1.6	19.8	7.2	19.4	2.4	19.7
-10%	1.8	19.8	8.1	19.6	2.7	19.8
0%	2	19.9	9	19.9	3	19.9
10%	2.2	19.9	9.9	20.2	3.3	20
20%	2.4	20	10.8	20.4	3.6	20.1
25%	2.5	20	11.25	20.6	3.75	20.1
50%	3	20.2	13.5	21.2	4.5	20.3

Table 4.1.c – Relative variation perturbation test results for each leaf node on the BN for Scenario 1.

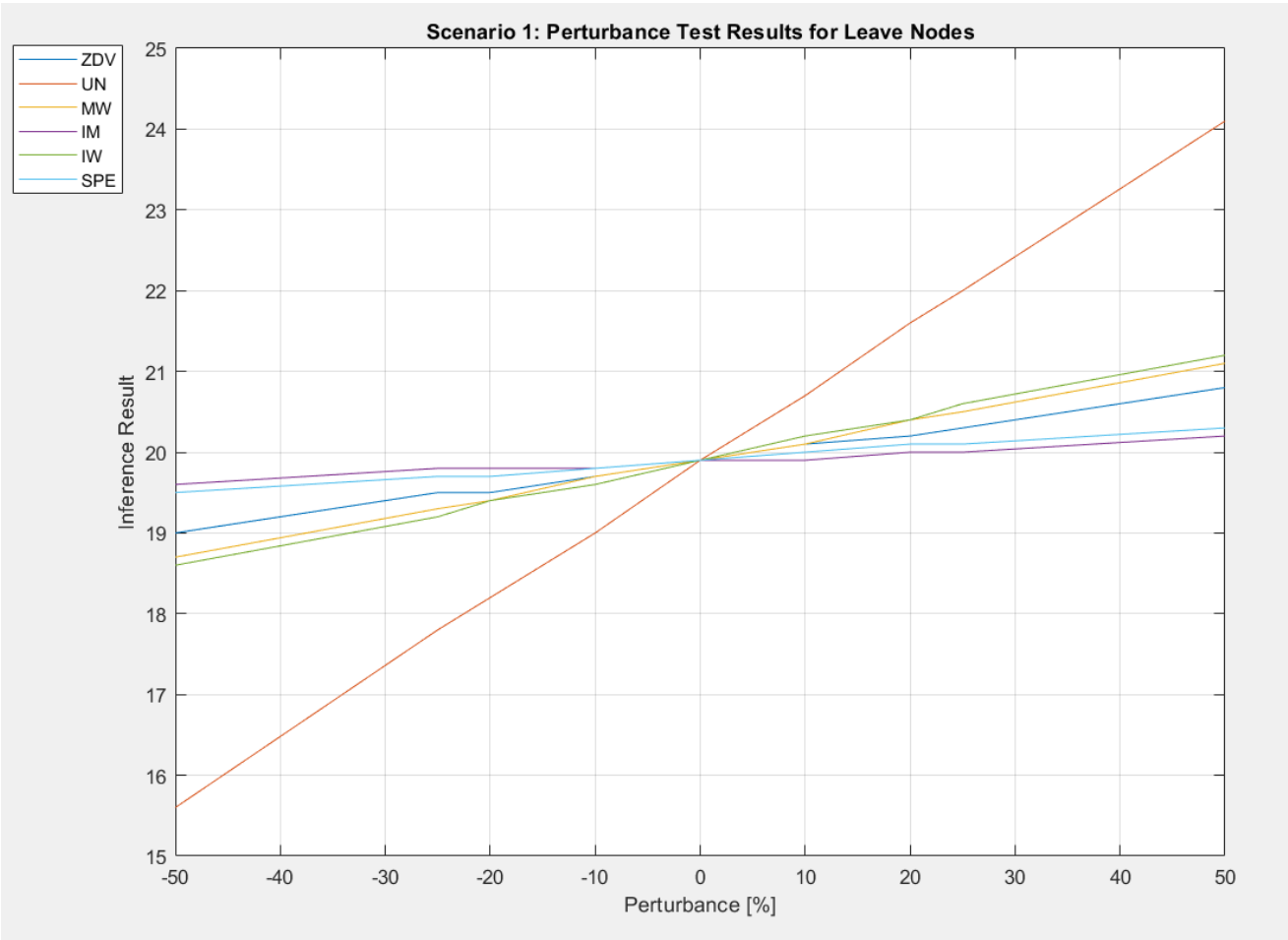

Fig 4.1.d – Relative variation perturbation test results for Scenario 1 plotted in MATLAB.

Node	EUV		BD		AIC	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	30	11.1	42.5	9.94	45	9.94
-25%	45	15.5	63.75	14.9	67.5	14.9
-20%	48	16.4	68	15.9	72	15.9
-10%	54	18.1	76.5	17.9	81	17.9
0%	60	19.9	85	19.9	90	19.9
10%	66	21.6	93.5	21.9	99	21.9
20%	72	23.4	100	23.4	100	22.1
25%	75	24.3	100	23.4	100	22.1
50%	90	28.7	100	23.4	100	22.1

Table 4.1.d – Relative variation perturbation Test results for each non-deterministic middle node on the BN.

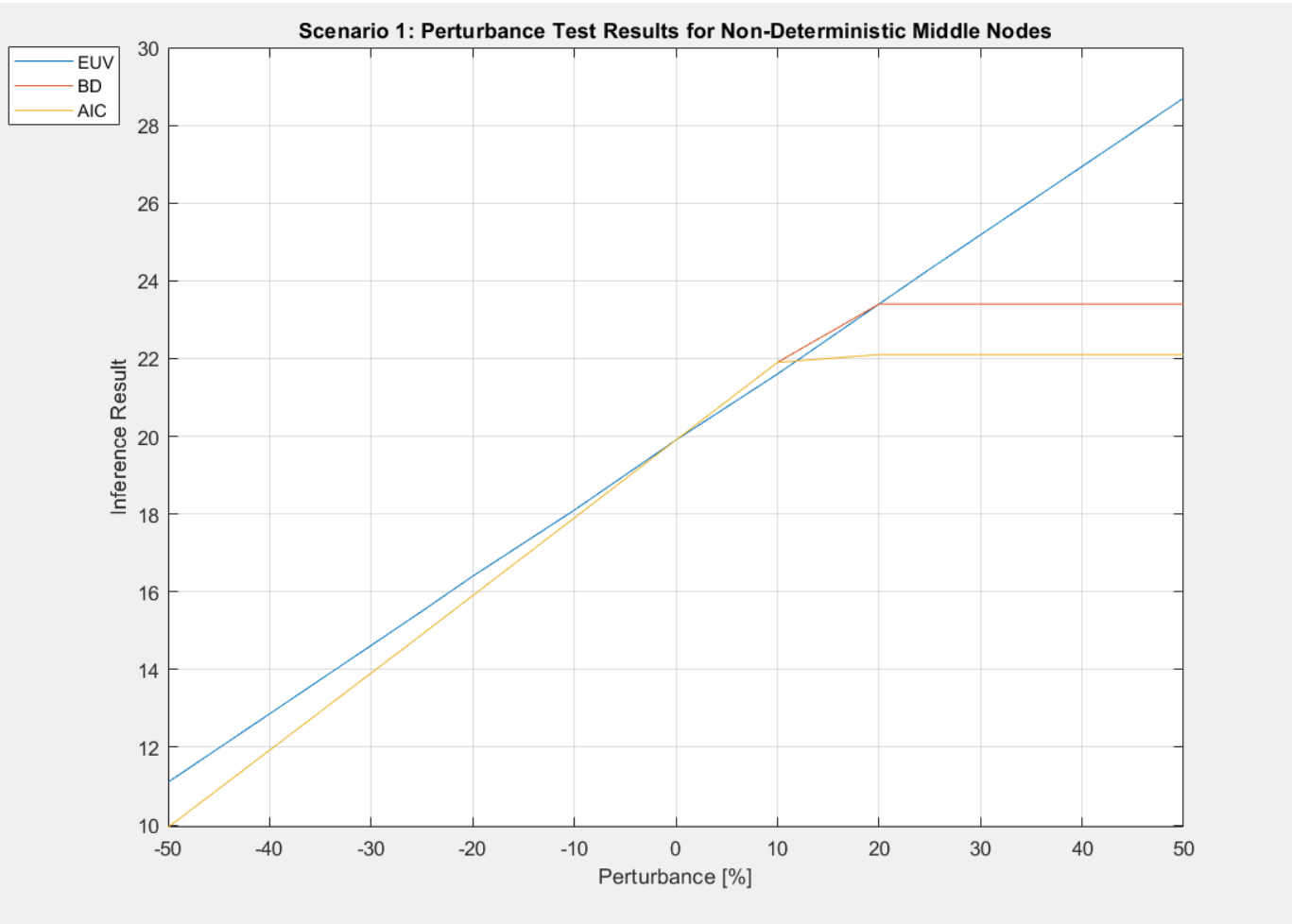

Fig 4.1.e – Relative variation perturbation test results for Scenario 1 plotted in MATLAB.

Node	ZDV		UN		MW	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	0	18.1	0	11.4	0	17.5
-25%	0	18.1	0	11.4	0	17.5
-20%	0	18.1	4	12.8	0	17.5
-10%	0	18.1	14	16.3	0	17.5
0%	3	19.9	24	19.9	8	19.9
10%	13	25.7	34	23.4	18	22.8
20%	23	31.6	44	27	28	25.7
25%	28	34.5	49	28.7	33	27.2
50%	53	49.1	74	37.6	58	34.5

Node	IM		IW		SPE	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	0	19.3	0	17.2	0	19.1
-25%	0	19.3	0	17.2	0	19.1
-20%	0	19.3	0	17.2	0	19.1
-10%	0	19.3	0	17.2	0	19.1
0%	2	19.9	9	19.9	3	19.9
10%	12	22.6	19	22.8	13	22.7
20%	22	25.4	29	25.8	23	25.4
25%	27	26.8	34	27.3	28	26.8
50%	52	33.6	59	34.7	53	33.8

Table 4.1.e – Absolute variation perturbation test results for each leaf node on the BN for Scenario 1.

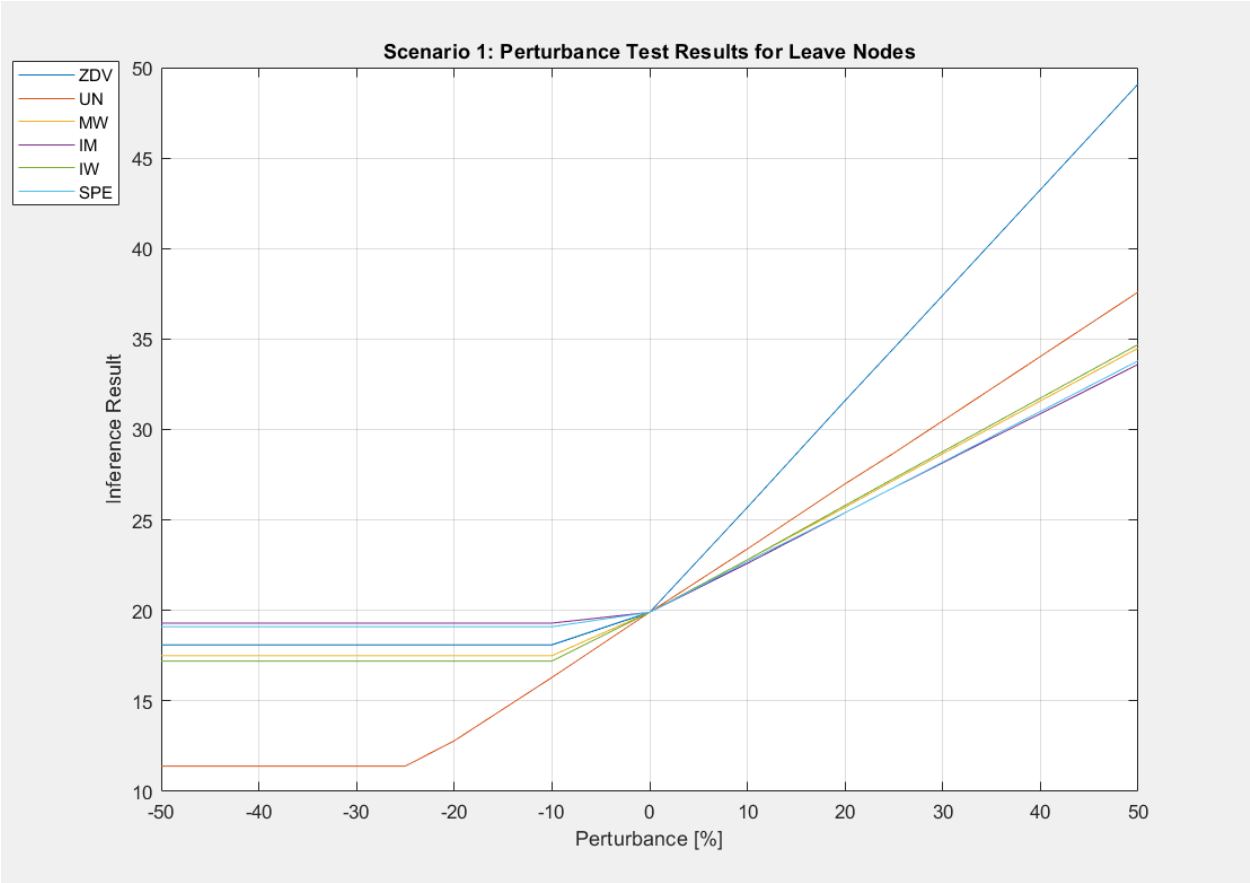

Fig 4.1.f - Absolute variation perturbation test results for Scenario 1 plotted in MATLAB.

Node	EUV		BD		AIC	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	10	5.23	35	8.19	40	8.84
-25%	35	12.6	60	14	65	14.4
-20%	40	14	65	15.2	70	15.5
-10%	50	17	75	17.5	80	17.7
0%	60	19.9	85	19.9	90	19.9
10%	70	22.8	95	22.2	100	22.1
20%	80	25.8	100	23.4	100	22.1
25%	85	27.2	100	23.4	100	22.1
50%	100	31.6	100	23.4	100	22.1

Table 4.1.f -Absolute variation perturbation test results for each leaf node on the BN for Scenario 1.

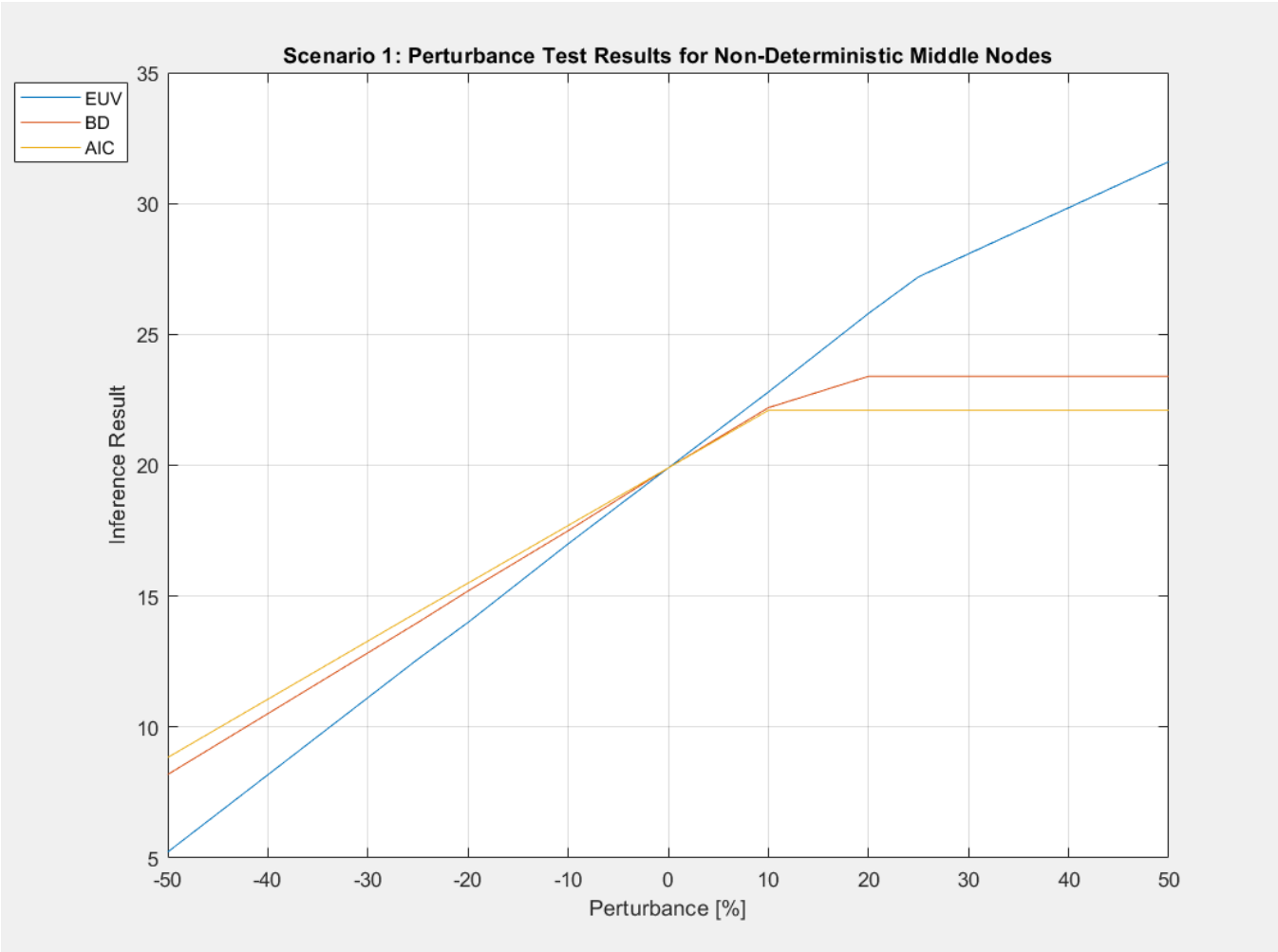

Fig 4.1.g - Absolute variation perturbation test results for Scenario 1 plotted in MATLAB.

4.2 Scenario 2: Obtaining Username and Password

Brief description of the scenario and AT

This is the second scenario for an authentication violation attack. This time around, the scenario is centred on the attacker obtaining username and password from the target system. This scenario introduces Cross-Site Requested Forgery node, included on OWASP top 10 vulnerability rank for 2017.

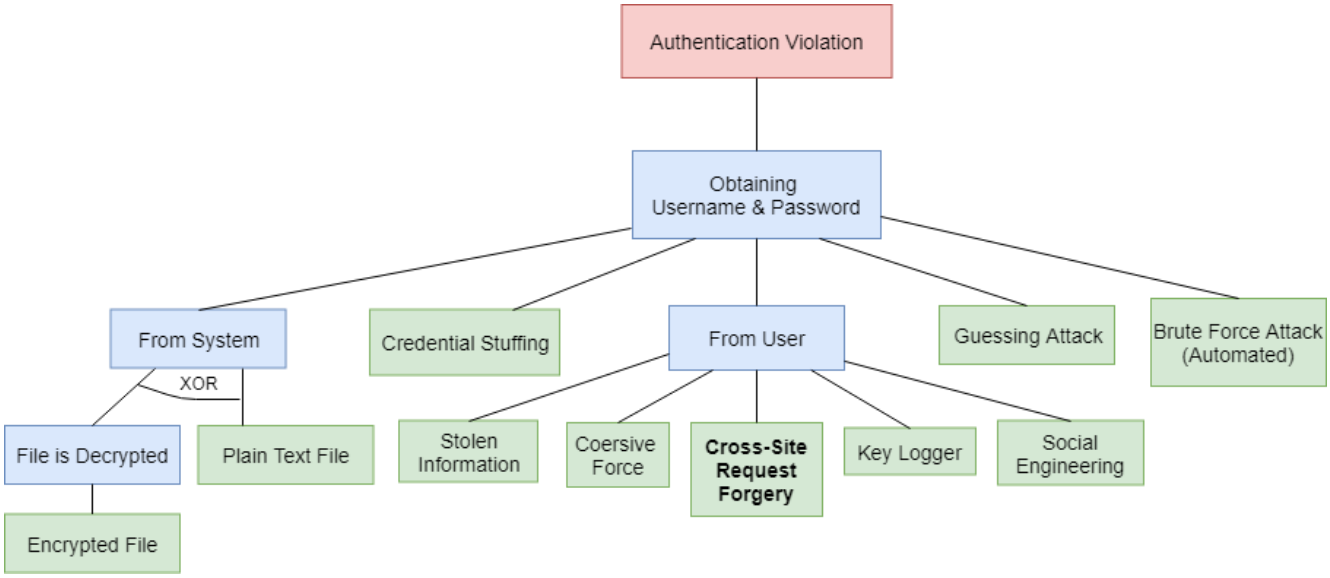

Fig 4.2.a – Scenario for an authentication violation attack modelled as an Attack Tree.

Values for Static Assessment

Following the same logic as Scenario 1, and for the sake of simplicity, we assigned hypothetical values that could have been reelevated from simulations, honeypots or using other techniques, in this case on a small or medium size company.

Leaf node	Identifying Acronym	Estimated probability	Possible Detection Sensors
Encrypted File	EF	1.0	-Human -User Level Endpoint Monitoring
Plain Text File	PTF	0.0	-Human -User Level Endpoint Monitoring
Credential Stuffing	CS	0.15	-Unexpected flow of data -IDS -Antivirus -System Event Logs
Guessing Attack	GA	0.05	-Unexpected flow of data -IDS -Antivirus -System Event Logs

Automated Brute Force Attack	BFA	0.03	-Unexpected flow of data -IDS -Antivirus -System Event Logs
Stolen Information	SI	0.02	-Human -User Level Endpoint Monitoring
Coercive Force	CF	0.01	-Human -User Level Endpoint Monitoring
Cross-Site Request Forgery	XSF	0.12	-Human -User Level Endpoint Monitoring
Key Logger	KL	0.02	-Anomaly detection based IDS -System Event Logs
Social Engineering	SE	0.20	-Human -User Level Endpoint Monitoring

Table 4.2.a – Assigned probabilities and sensor assumptions for the scenario

Middle Node	Identifying Acronym	Estimated Probability
File is Decrypted	DF	0.05

Table 4.2.b – Assigned probabilities for non-deterministic middle nodes

Conversion to BN and machine-readable XML code

Following the proposed M2M transformation, we convert the given AT into a BN, and populate the CPT with the proposed values presented above. The results are displayed in **Fig 4.2.b** . The machine-readable XML code for this scenario can be found on ***Appendix 4***.

BN perturbation test

The same methodology implemented on Scenario 1 was used to measure the perturbation sensibility of Scenario 2. Relative variation perturbation test results for all leave nodes are presented in **Table 4.2.c** , **Fig 4.2.d** , results for middle nodes are presented in **Table 4.2.d** , **Fig 4.2.e** . Absolute variation perturbation test results for all leave nodes are presented in **Table 4.2.f Fig 4.2.f** , results for middle nodes are presented in **Table 4.2.e Fig 4.2.g** .
Test results are further discussed in Chapter 5 and Chapter 6.

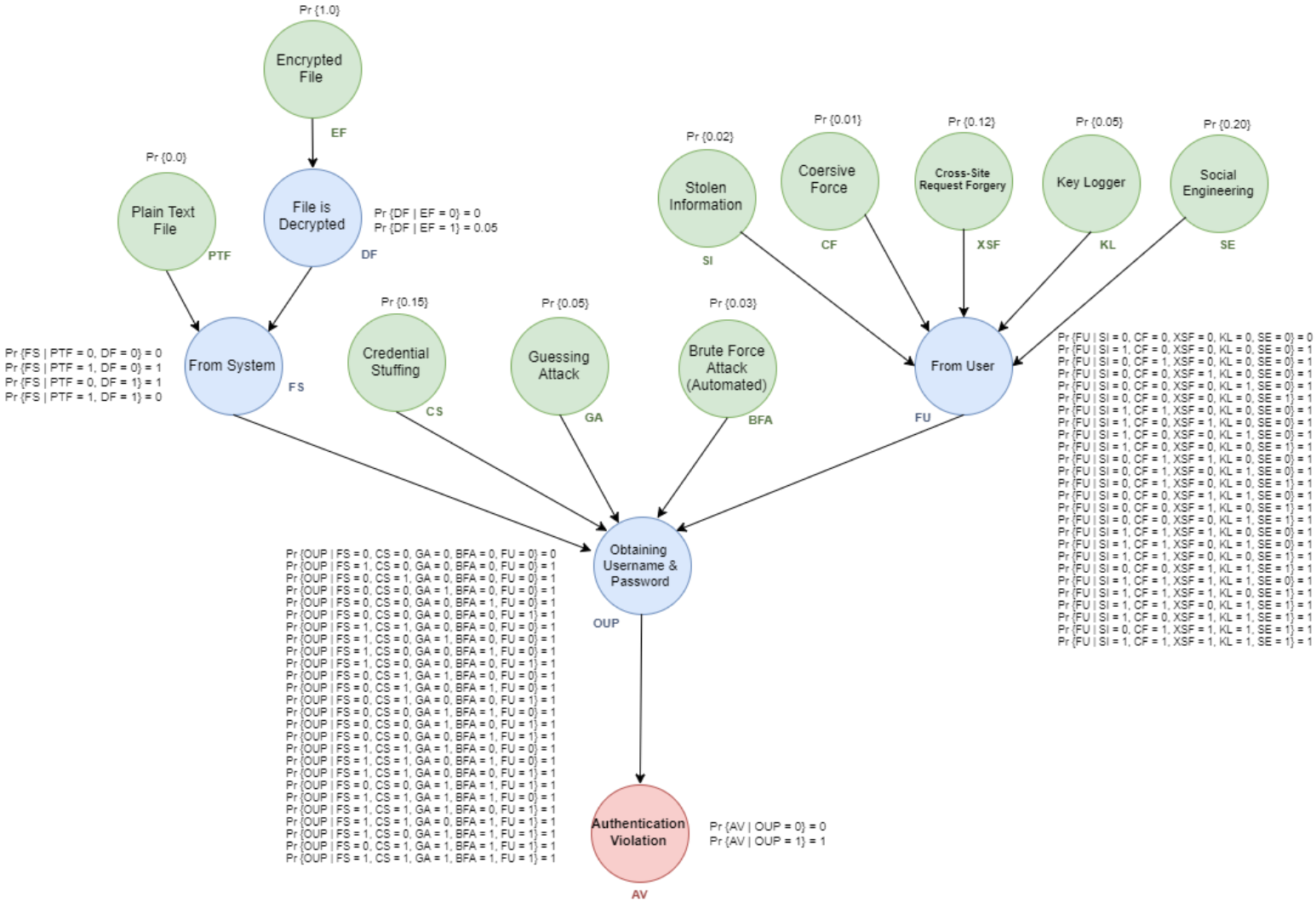

Fig 4.2.b – Resulting BN using the proposed M2M transformation model.

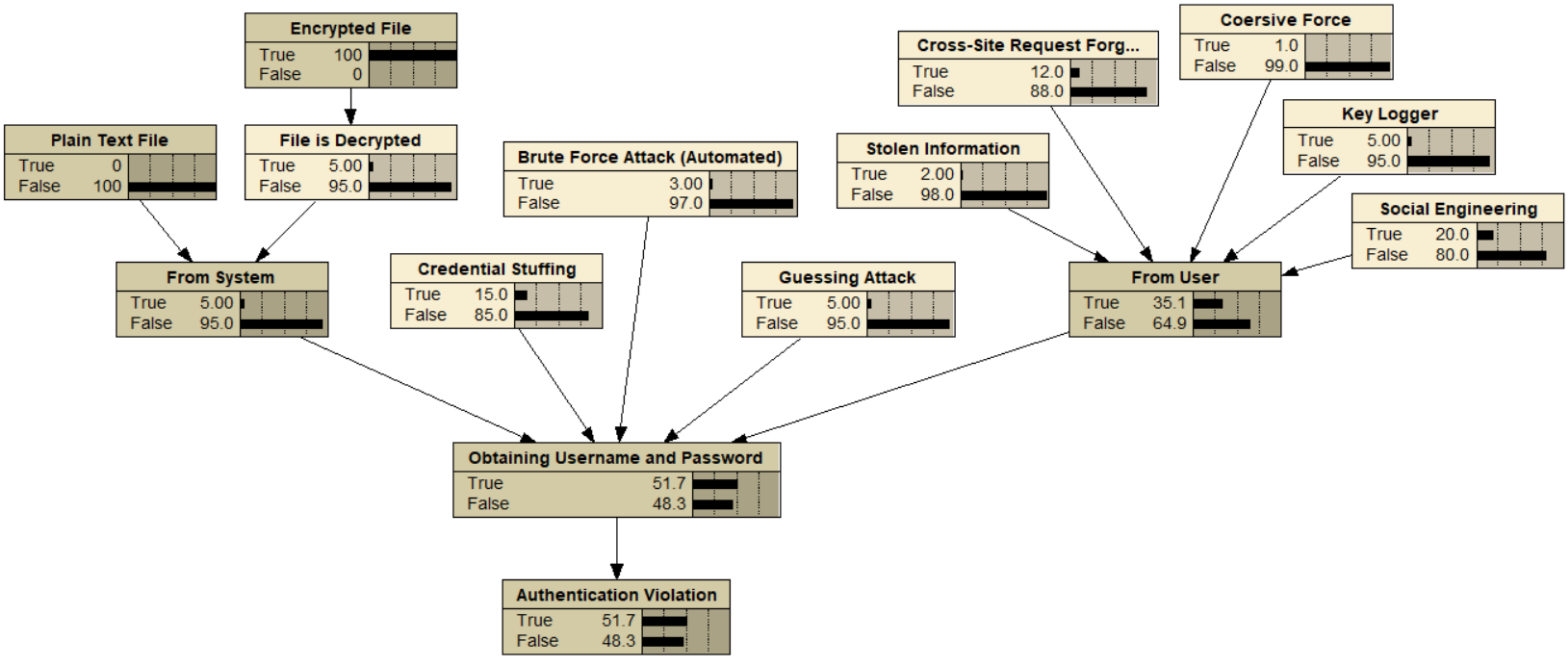

Fig 4.2.c – Resulting BN for scenario 2, modelled using Netica.

Node	EF		PTF		CS		GA		BFA	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results	Value	Results
-50%	50	50.4	0	51.7	7.5	47.5	2.5	50.4	1.5	51
-25%	75	51.1	0	51.7	11.25	49.6	3.75	51.1	2.25	51.3
-20%	80	51.2	0	51.7	12	50	4	51.2	2.4	51.4
-10%	90	51.5	0	51.7	13.5	50.9	4.5	51.5	2.7	51.6
0%	100	51.7	0	51.7	15	51.7	5	51.7	3	51.7
10%	100	51.7	0.01	52.2	16.5	52.6	5.5	52	3.3	51.9
20%	100	51.7	0.02	52.6	18	53.4	6	52.2	3.6	52
25%	100	51.7	0.025	52.9	18.75	53.8	6.25	52.4	3.75	52.1
50%	100	51.7	0.05	54	22.5	56	7.5	53	4.5	52.5

Node	SI		CF		XSF		KL		SE	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results	Value	Results
-50%	1	51.2	0.5	51.5	6	48.4	2.5	50.4	10	45.7
-25%	1.5	51.5	0.75	51.6	9	50.1	3.75	51.1	15	48.7
-20%	1.6	51.5	0.8	51.6	9.6	50.4	4	51.2	16	49.3
-10%	1.8	51.6	0.9	51.7	10.8	51.1	4.5	51.5	18	50.5
0%	2	51.7	1	51.7	12	51.7	5	51.7	20	51.7
10%	2.2	51.8	1.1	51.8	13.2	52.4	5.5	52	22	52.9
20%	2.4	51.9	1.2	51.8	14.4	53	6	52.2	24	54.1
25%	2.5	52	1.25	51.8	15	53.4	6.25	52.4	25	54.7
50%	3	52.2	1.5	52	18	55	7.5	53	30	57.8

Table 4.2.c – Relative variation perturbation test results for each leaf node on the BN for Scenario 2.

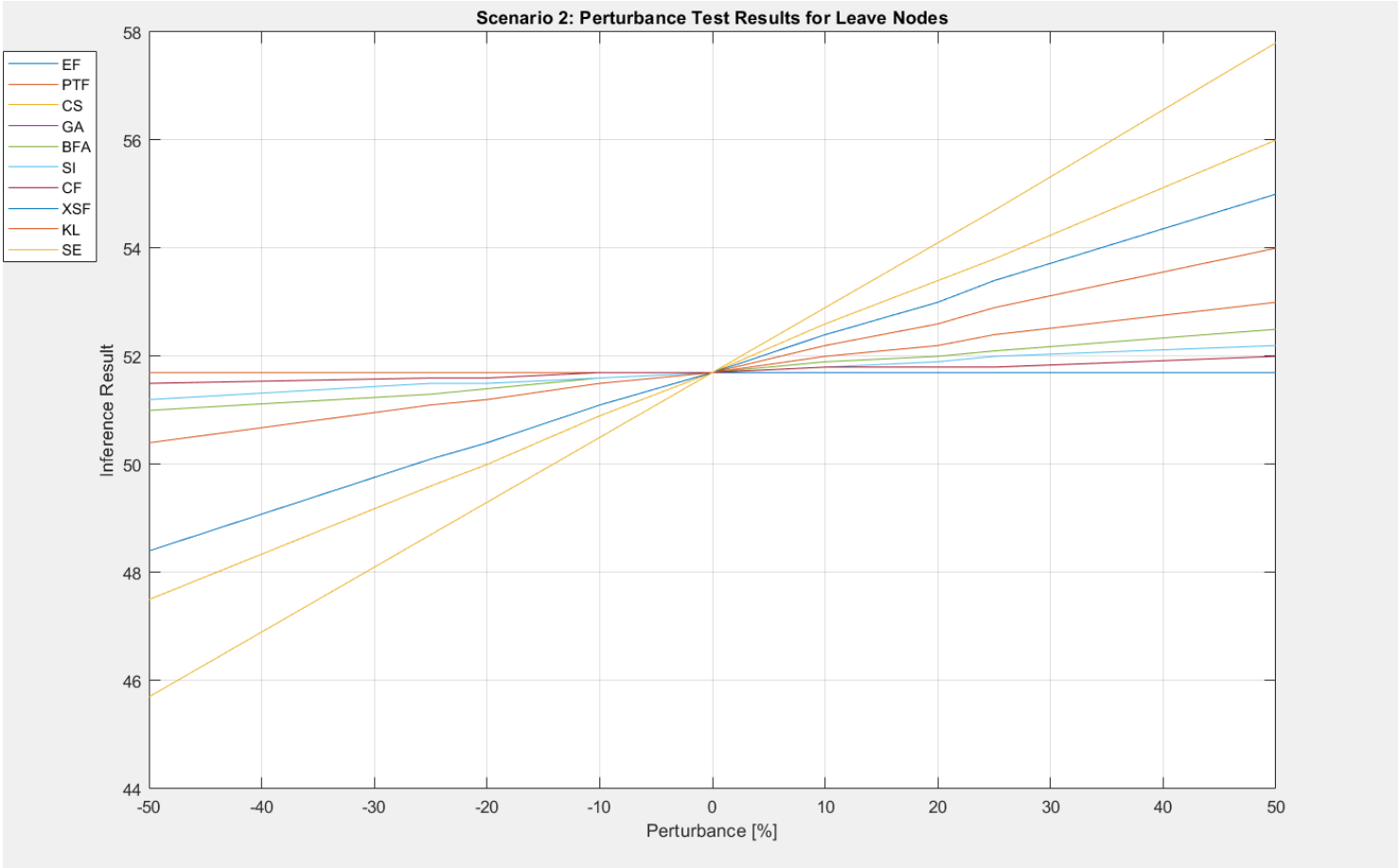

Fig 4.2.d – Relative variation perturbation test results for Scenario 2 plotted in MATLAB.

Node	DF	
Perturbance Percentage	Value	Results
-50%	2.5	50.4
-25%	3.75	51.1
-20%	4	51.2
-10%	4.5	51.5
0%	5	51.7
10%	5.5	52
20%	6	52.2
25%	6.25	52.4
50%	7.5	53

Table 4.2.d – Relative variation perturbation test results for each non-deterministic middle node on the BN

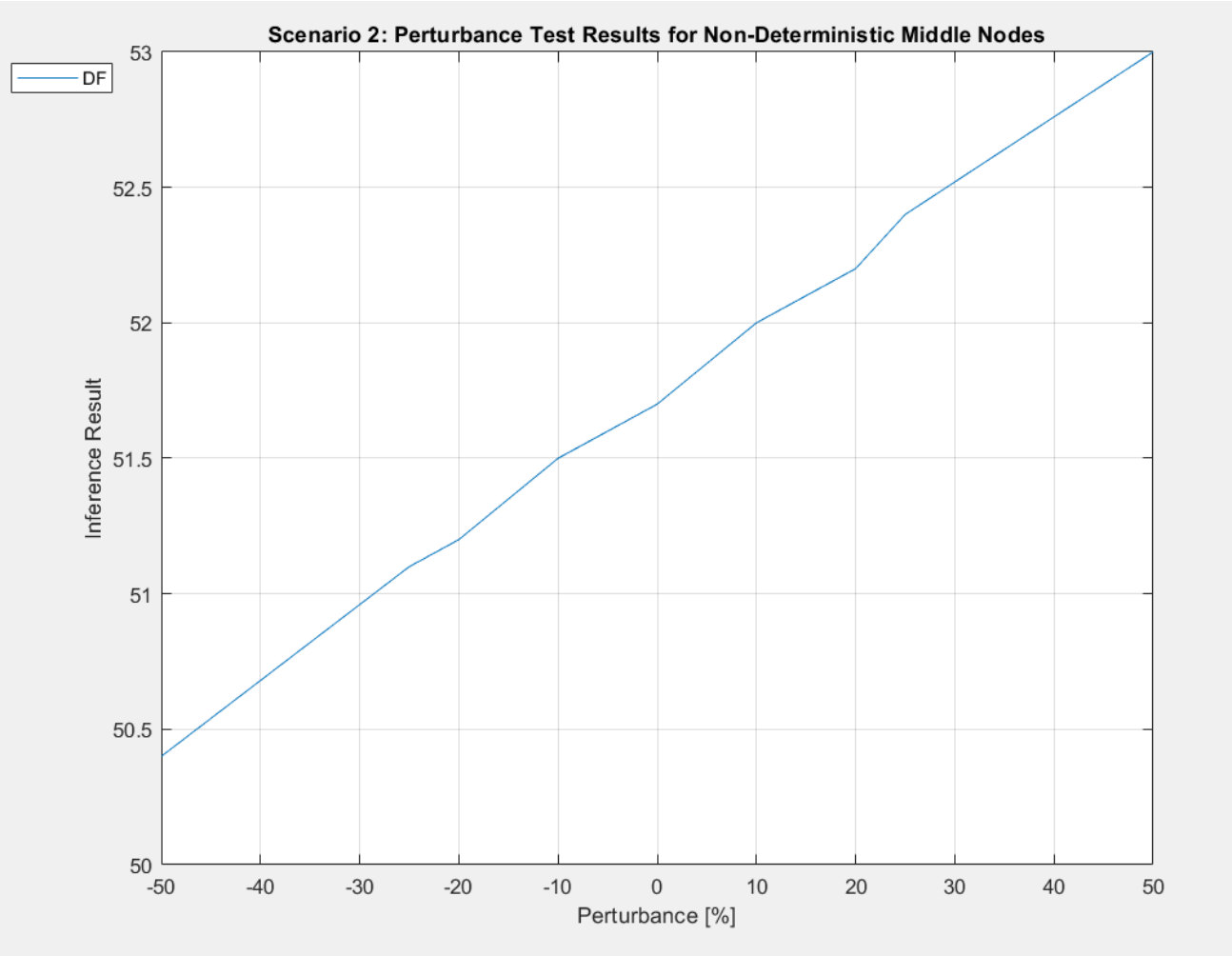

Fig 4.2.e – Relative variation perturbation test results for Scenario 2 plotted in MATLAB.

Node	EF		PTF		CS		GA		BFA	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results	Value	Results
-50%	50	50.4	0	51.7	0	43.2	0	49.2	0	50.2
-25%	75	51.1	0	51.7	0	43.2	0	49.2	0	50.2
-20%	80	51.2	0	51.7	0	43.2	0	49.2	0	50.2
-10%	90	51.5	0	51.7	5	46	0	49.2	0	50.2
0%	100	51.7	0	51.7	15	51.7	5	51.7	3	51.7
10%	100	51.7	10	56.3	2	57.4	15	56.8	13	56.7
20%	100	51.7	20	60.9	35	63.1	25	61.9	23	61.7
25%	100	51.7	25	63.2	40	65.9	30	64.4	28	64.2
50%	100	51.7	50	74.6	65	80.1	55	77.1	53	76.6

Node	SI		CF		XSF		KL		SE	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results	Value	Results
-50%	0	50.7	0	51.2	0	45.1	0	49.2	0	39.6
-25%	0	50.7	0	51.2	0	45.1	0	49.2	0	39.6
-20%	0	50.7	0	51.2	0	45.1	0	49.2	0	39.6
-10%	0	50.7	0	51.2	2	46.2	0	49.2	10	45.7
0%	2	51.7	1	51.7	12	51.7	5	51.7	20	51.7
10%	12	56.6	11	56.6	22	57.2	15	56.8	30	57.8
20%	22	61.6	21	61.5	32	62.7	25	61.9	40	63.8
25%	27	64	26	63.9	37	65.4	30	64.4	45	66.8
50%	52	76.5	51	76.1	62	79.2	55	77.1	70	81.9

Table 4.2.e - Absolute variation perturbation test results for each leaf node on the BN for Scenario 2.

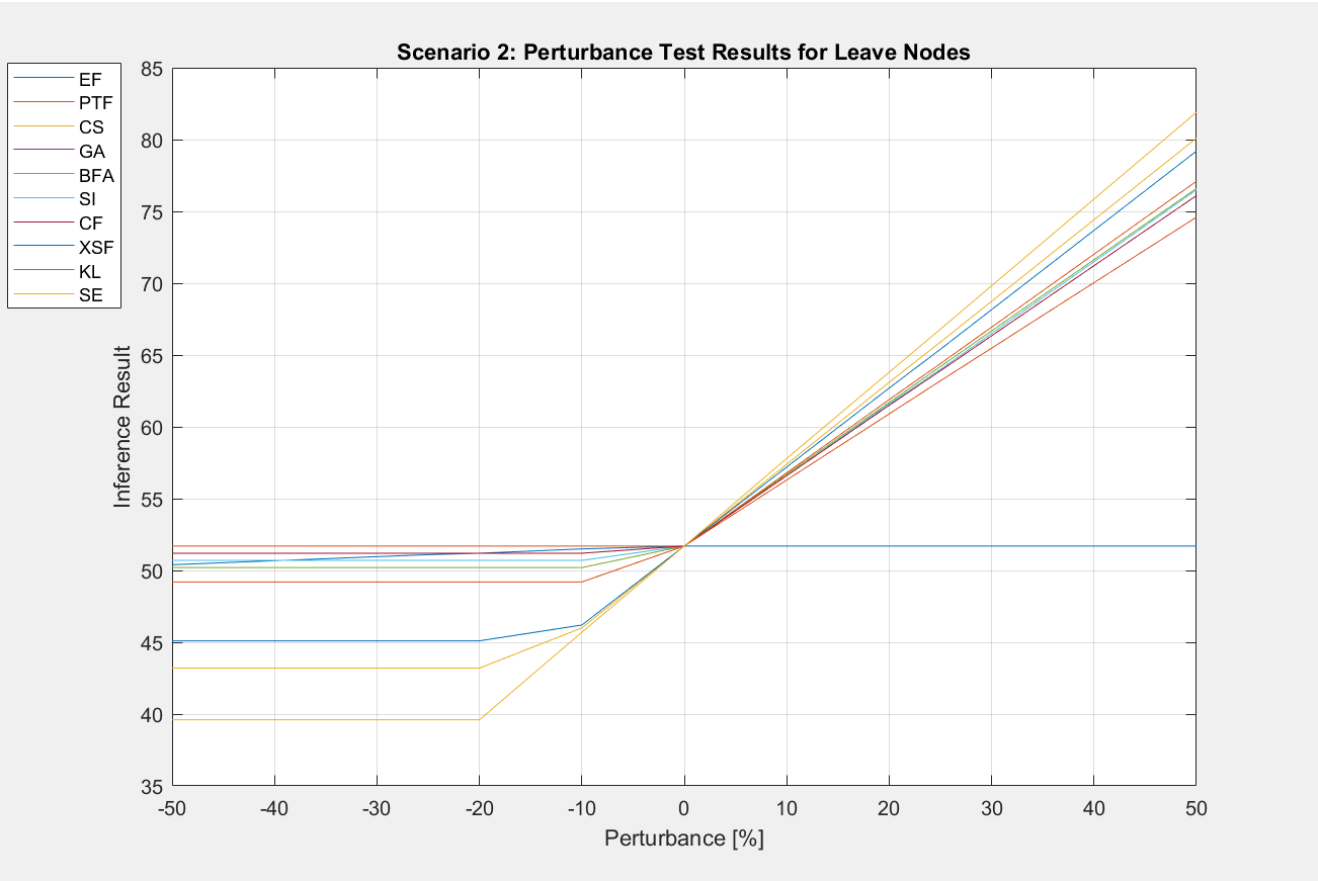

Fig 4.2.f - Absolute variation perturbation test results for Scenario 2 plotted in MATLAB.

Node	DF	
Perturbance Percentage	Value	Results
-50%	0	49.2
-25%	0	49.2
-20%	0	49.2
-10%	0	49.2
0%	5	51.7
10%	15	56.8
20%	25	61.9
25%	30	64.4
50%	55	77.1

Table 4.2.f - Absolute variation perturbation test results for each non-deterministic middle node on the BN

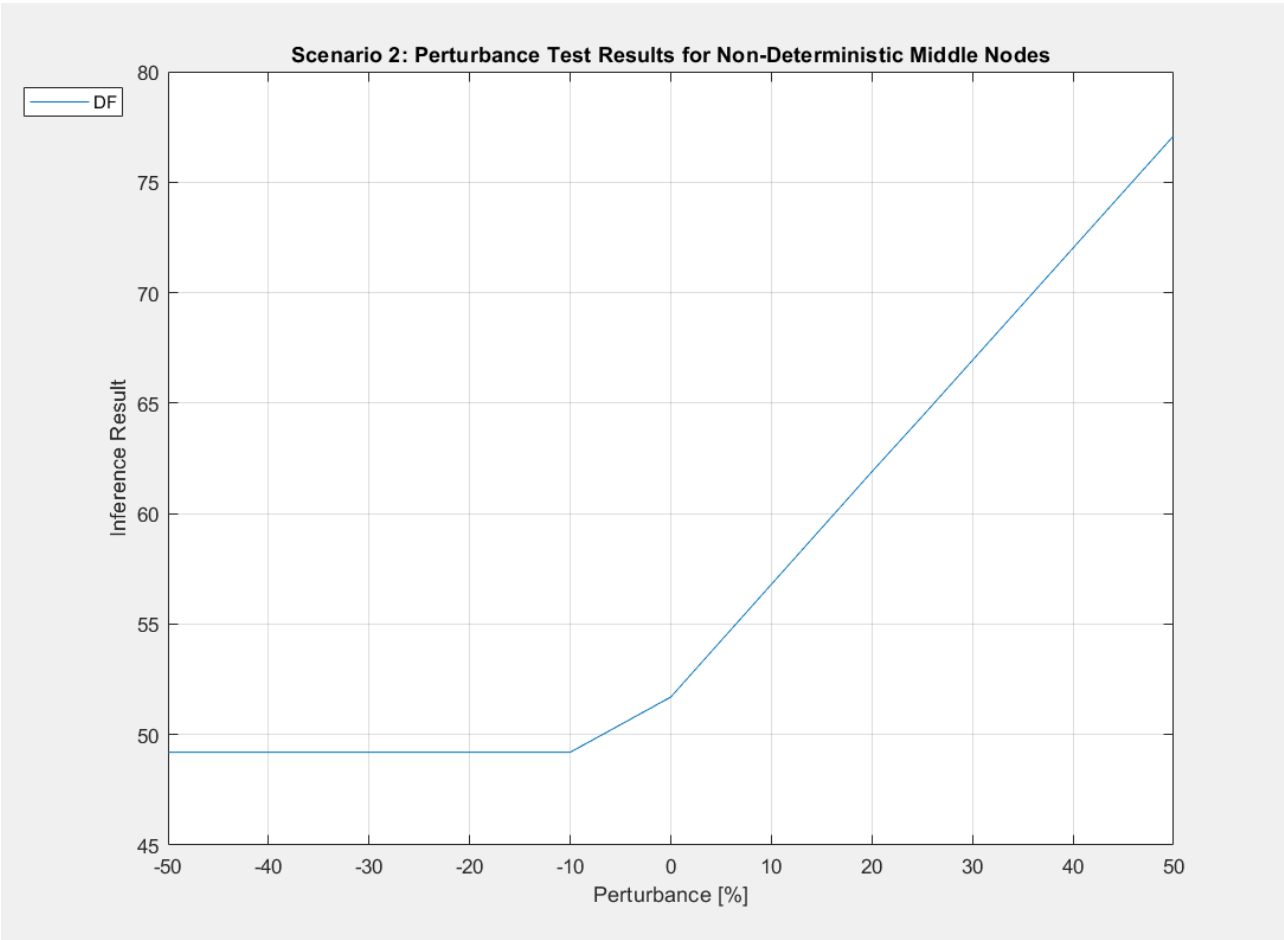

Fig 4.2.g - Absolute variation perturbation test results for Scenario 2 plotted in MATLAB.

4.3 Scenario 3: Intercepting login information and session hijacking

Brief description of the scenario and AT

This is the third scenario for an authentication violation attack. This time around, the scenario is centred on the attacker intercepting login information or hijacking a session. This scenario includes five different vulnerabilities that were included in OWASP'S 2017 list.

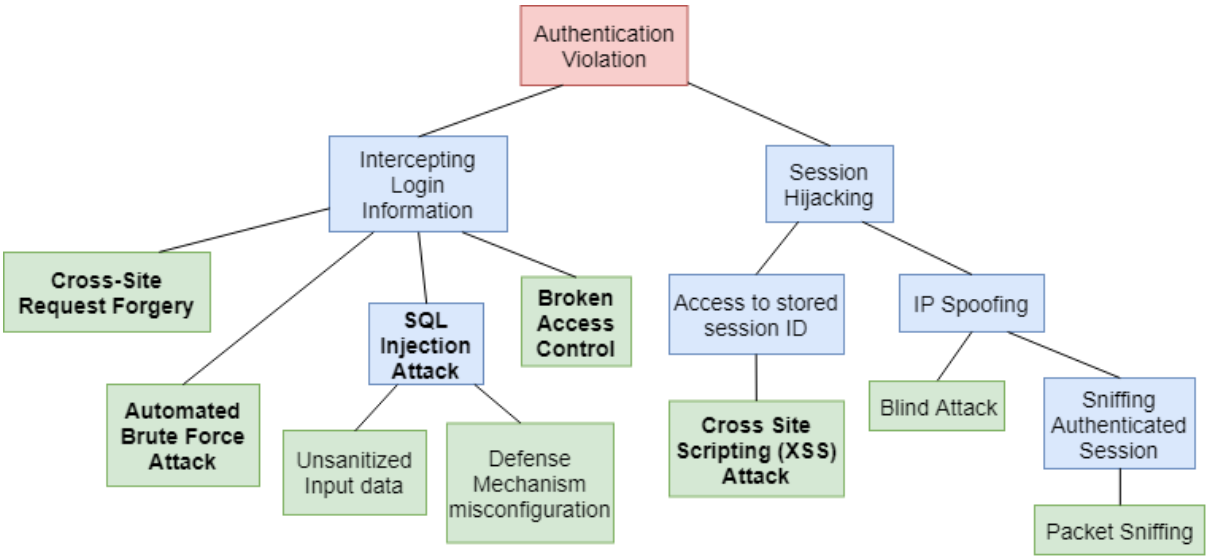

Fig 4.3.a – Scenario for an authentication violation attack modelled as an Attack Tree

Values for Static Assessment

Following the same logic as the previous two scenarios, and for the sake of simplicity, we assigned hypothetical values that could have been reelevated from simulations, honeypots or using other techniques, in this case on a small or medium size company.

Leaf node	Identifying Acronym	Estimated probability	Possible Detection Sensors
Cross-Site Request Forgery	XSF	0.12	-Human -User Level Endpoint Monitoring
Automated Brute Force Attack	BFA	0.03	-Anomaly detection based IDS -System Event Logs
Unsanitized Input Data	UID	0.05	-Human -User Level Endpoint Monitoring
Defence Mechanism Misconfiguration	DMM	0.18	-Human -User Level Endpoint Monitoring
Broken Access Control	BAC	0.10	-Human -User Level Endpoint Monitoring
Cross-Site Scripting Attack	XSS	0.20	-Anomaly detection based IDS -Antivirus -IDS

Blind Attack	BA	0.30	-Unexpected flow of data
Packet Sniffing	PS	0.65	-Unexpected flow of data

Table 4.3.a – Assigned probabilities and sensor assumptions for the scenario

Middle Node	Identifying Acronym	Estimated Probability
SQL Injection Attack	SIA	0.30
Intercepting Login Information	ILI	0.45
Sniffing Authentication Session	SAS	0.25
IP Spoofing	IPS	0.20
Access to stored session ID	ASID	0.50
Session Hijacking	SH	0.75

Table 4.3.b – Assigned probabilities for non-deterministic middle nodes

Conversion to BN and machine-readable XML code

Following the proposed M2M transformation, we convert the given AT into a BN, and populate the CPT with the proposed values presented above. The results are displayed in **Fig 4.3.c** . The machine-readable XML code for this scenario can be found on ***Appendix 5***.

BN perturbation test result

The same methodology implemented on Scenario 1 and Scenario 2 was used to measure the perturbation sensibility of Scenario 3. Relative variation perturbation test results for all leave nodes are presented in **Table 4.3.c** , **Fig 4.3.d** , results for middle nodes are presented in **Table 4.3.d** , **Fig 4.3.e** . Absolute variation perturbation test results for all leave nodes are presented in **Table 4.3.e** **Fig 4.3.f** , results for middle nodes are presented in **Table 4.3.f** **Fig 4.3.g** . Test results are further discussed in Chapter 5 and Chapter 6.

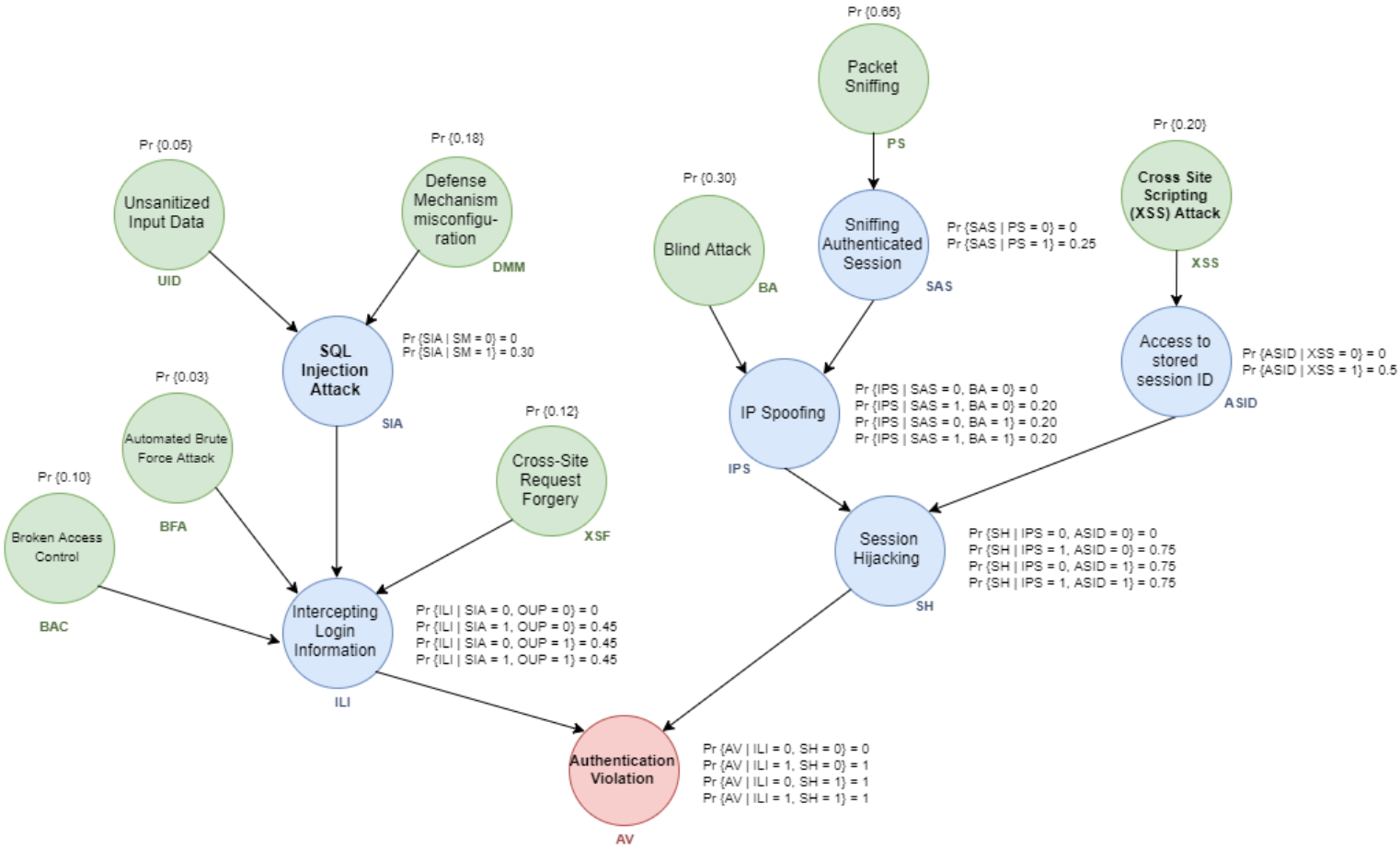

Fig 4.3.b – Resulting BN using the proposed M2M transformation model.

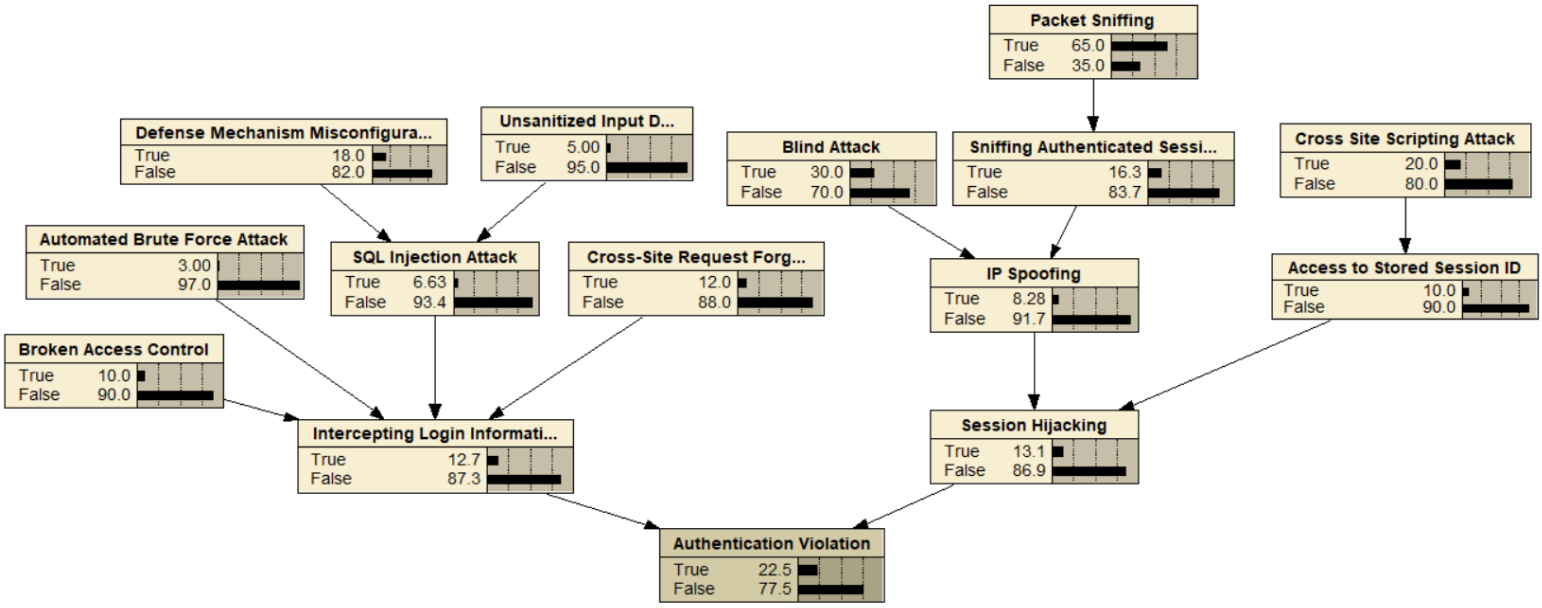

Fig 4.3.c – Resulting BN for scenario 3, modelled using Netica.

Node	XSF		BFA		UID		DMM	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results
-50%	6	20.9	1.5	22.1	2.5	22.3	9	21.8
-25%	9	21.7	2.25	22.3	3.75	22.4	13.5	22.2
-20%	9.6	21.8	2.4	22.3	4	22.4	14.4	22.2
-10%	10.8	22.2	2.7	22.4	4.5	22.4	16.2	22.3
0%	12	22.5	3	22.5	5	22.5	18	22.5
10%	13.2	22.8	3.3	22.6	5.5	22.5	19.8	22.6
20%	14.4	23.1	3.6	22.6	6	22.5	21.6	22.7
25%	15	23.3	3.75	22.7	6.25	22.6	22.5	22.8
50%	18	24.1	4.5	22.8	7.5	22.6	27	23.1

Node	BAC		XSS		BA		PS	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results
-50%	5	21.2	10	19.9	15	21.2	32.5	21.9
-25%	7.5	21.8	15	21.2	22.5	21.8	48.75	22.2
-20%	8	21.9	16	21.5	24	22	52	22.2
-10%	9	22.2	18	22	27	22.2	58.5	22.4
0%	10	22.5	20	22.5	30	22.5	65	22.5
10%	11	22.7	22	23	33	22.7	71.5	22.6
20%	12	23	24	23.5	36	23	78	22.7
25%	12.5	23.1	25	23.8	37.5	23.1	81.25	22.8
50%	15	23.8	30	25	45	23.7	97.5	23.1

Table 4.3.c – Relative variation perturbation test results for each leaf node on the BN for Scenario 3.

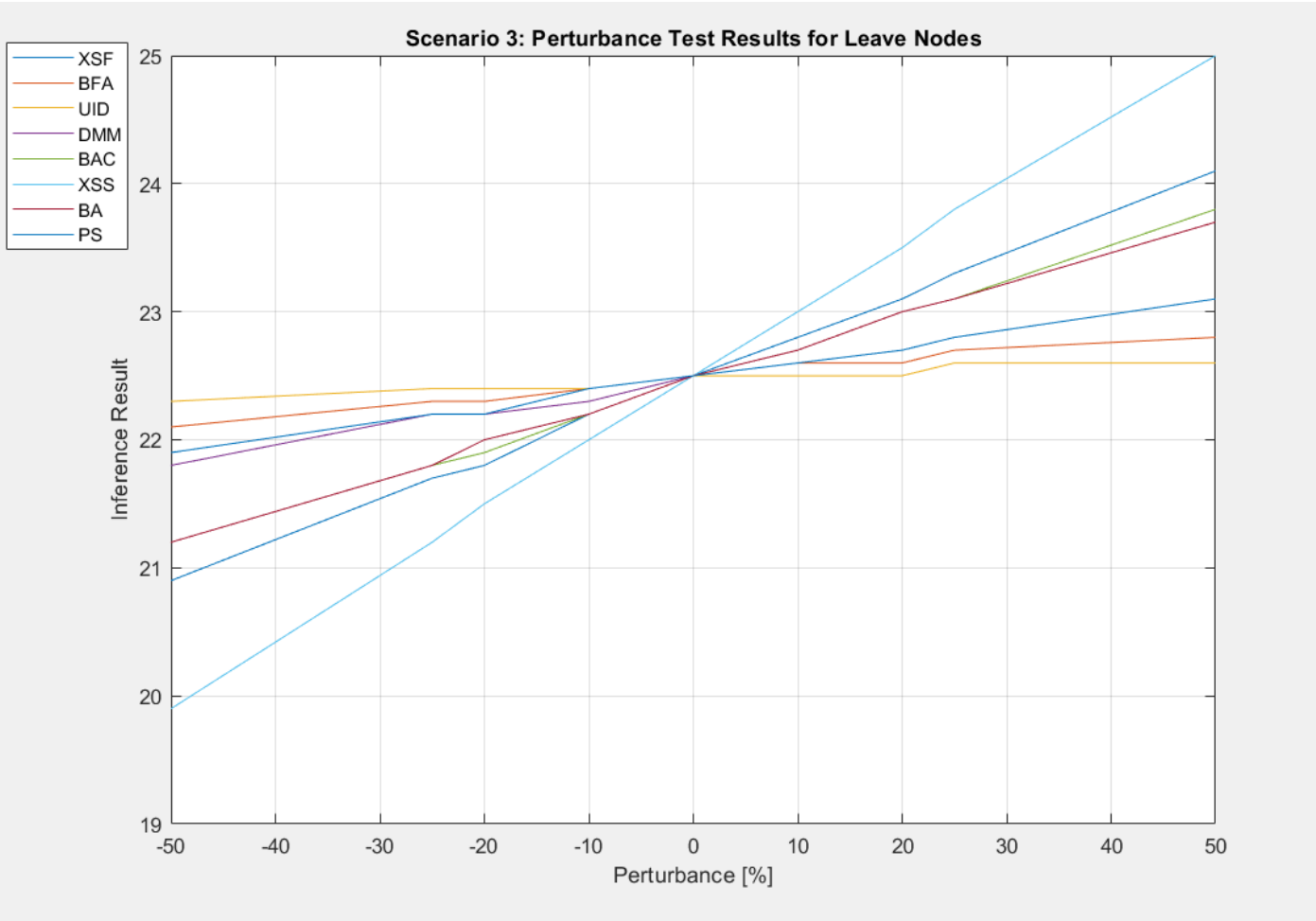

Fig 4.3.d – Relative variation perturbation test results for Scenario 3 plotted in MATLAB.

Node	SIA		ILI		SAS	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	15	21.6	22.5	17.8	12.5	21.9
-25%	22.5	22.1	33.75	20.1	18.75	22.2
-20%	24	22.1	36	20.6	20	22.2
-10%	27	22.3	40.5	21.5	22.5	22.4
0%	30	22.5	45	22.5	25	22.5
10%	33	22.6	49.5	23.4	27.5	22.6
20%	36	22.8	54	24.4	30	22.7
25%	37.5	22.9	56.25	24.8	31.25	22.8
50%	45	23.3	67.5	27.2	37.5	23.1

Node	IPS		ASID		SH	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	10	20.4	25	19.9	37.5	17.6
-25%	15	21.4	37.5	21.2	56.25	20
-20%	16	21.6	40	21.5	60	20.5
-10%	18	22.1	45	22	67.5	21.5
0%	20	22.5	50	22.5	75	22.5
10%	22	22.9	55	23	82.5	23.5
20%	24	23.3	60	23.5	90	24.4
25%	25	23.5	62.5	23.8	93.75	24.9
50%	30	24.6	75	25	100	25.7

Table 4.3.d – Relative variation perturbation test results for each non-deterministic middle node on the BN

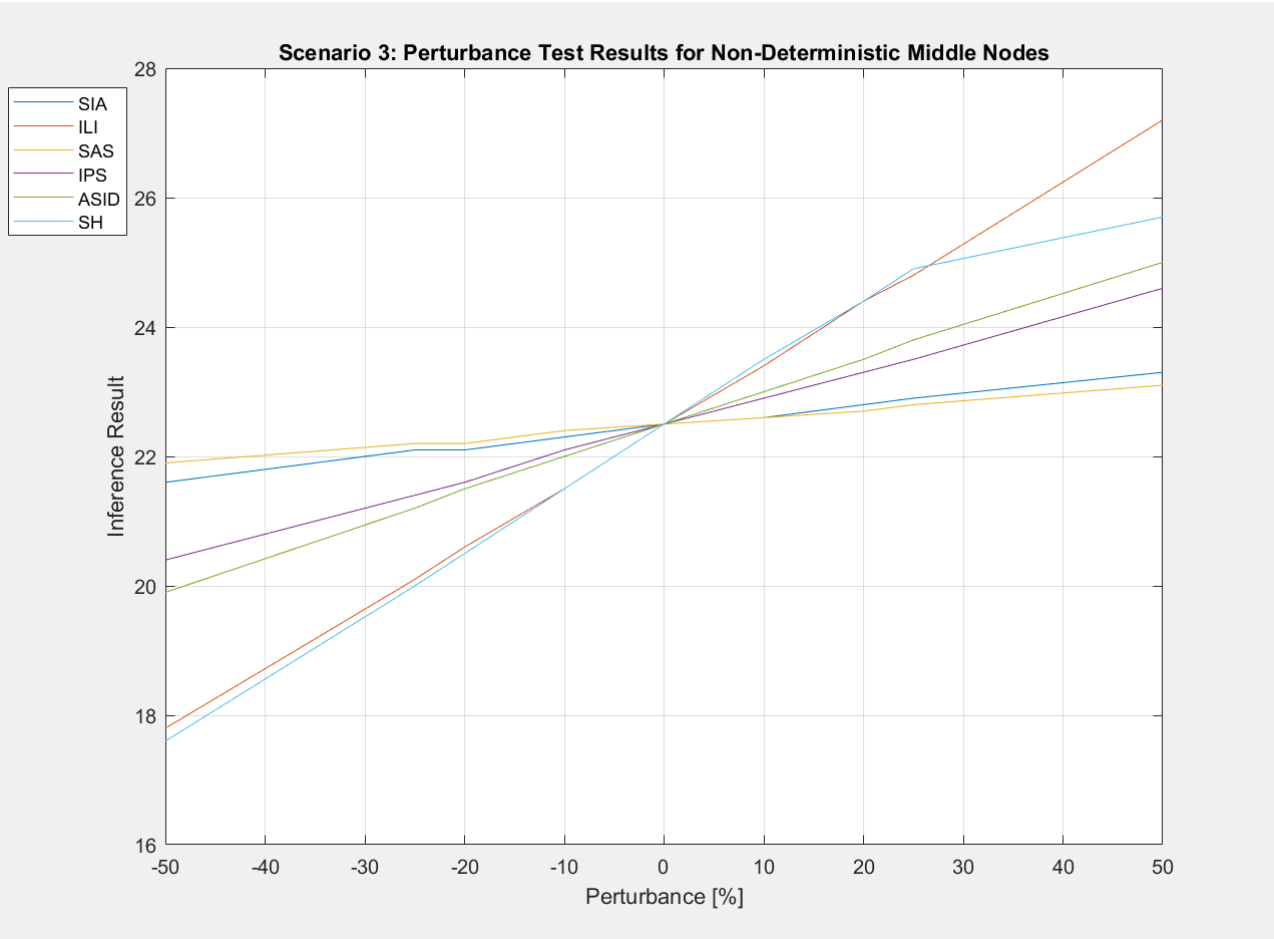

Fig 4.3.e – Relative variation perturbation test results for Scenario 3 plotted in MATLAB.

Node	XSF		BFA		UID		DMM	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results
-50%	0	19.2	0	21.7	0	22.2	0	21.2
-25%	0	19.2	0	21.7	0	22.2	0	21.2
-20%	0	19.2	0	21.7	0	22.2	0	21.2
-10%	2	19.8	0	21.7	0	22.2	8	21.8
0%	12	22.5	3	22.5	5	22.5	18	22.5
10%	22	25.2	13	24.9	15	23.1	28	23.2
20%	32	27.9	23	24.9	25	23.7	38	23.9
25%	37	28.7	28	24.9	30	24	43	24.3
50%	62	36	53	24.9	55	25.6	68	26.1

Node	BAC		XSS		BA		PS	
Perturbance Percentage	Value	Results	Value	Results	Value	Results	Value	Results
-50%	0	19.8	0	17.3	0	19.9	15	21.6
-25%	0	19.8	0	17.3	5	20.4	40	22
-20%	0	19.8	0	17.3	10	20.8	45	22.1
-10%	0	19.8	10	19.9	20	21.6	55	22.3
0%	10	22.5	20	22.5	30	22.5	65	22.5
10%	20	25.1	30	25	40	23.3	75	22.7
20%	30	27.8	40	27.6	50	24.2	85	22.8
25%	35	29.1	45	28.9	55	24.6	90	22.9
50%	60	35.7	70	35.3	80	26.7	100	23.1

Table 4.3.e – Absolut perturbation test results for each leaf node on the BN for Scenario 3.

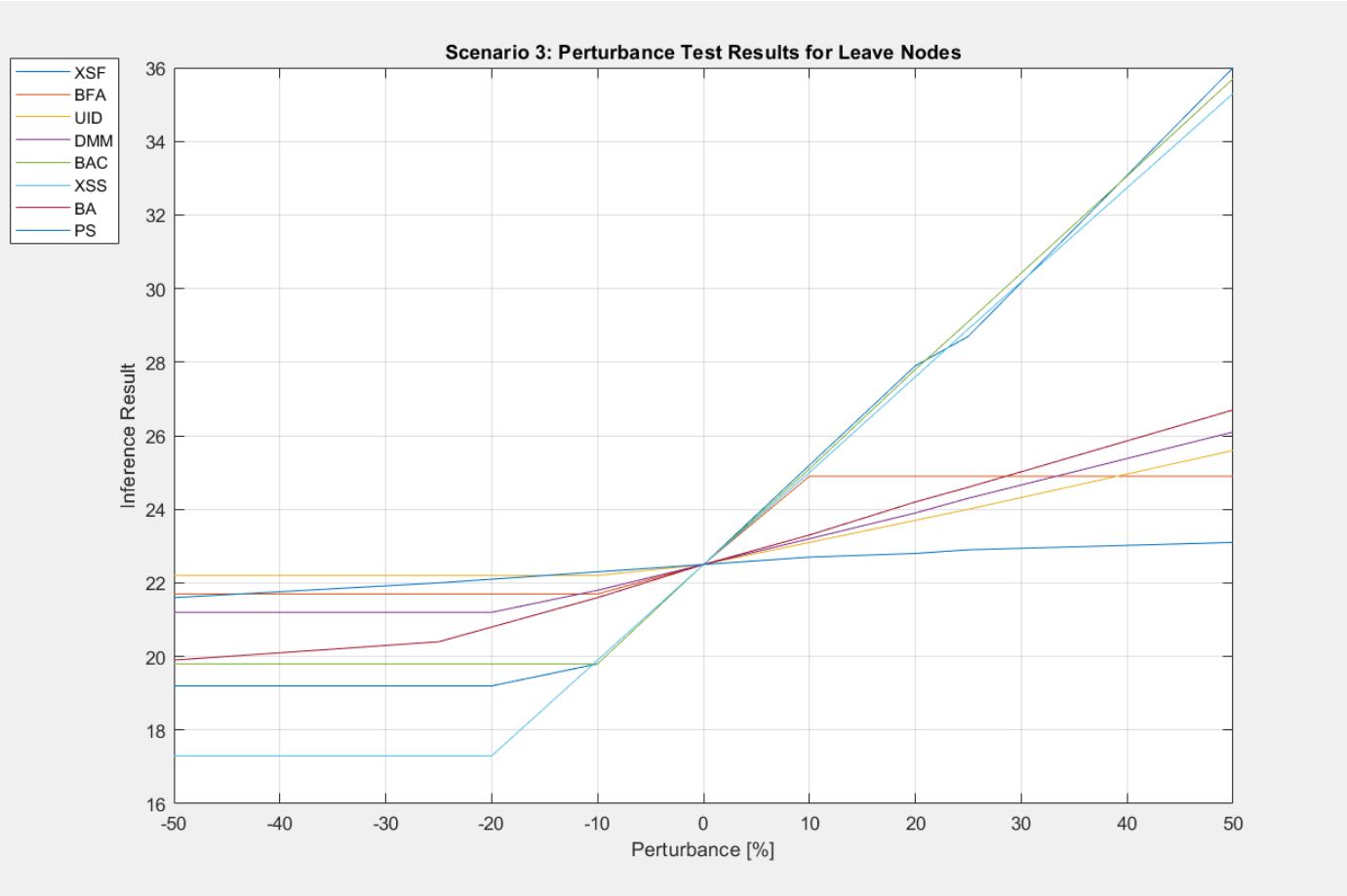

Fig 4.3.f - Absolute variation perturbation test results for Scenario 3 plotted in MATLAB.

Node	SIA		ILI		SAS	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	0	20.8	0	13.1	0	21.3
-25%	5	21.1	20	17.3	0	21.3
-20%	10	21.3	25	18.3	5	21.6
-10%	20	21.9	35	20.4	15	22
0%	30	22.5	45	22.5	25	22.5
10%	40	23	55	24.6	35	22.9
20%	50	23.6	65	26.7	45	23.4
25%	55	23.9	70	27.7	50	23.6
50%	80	25.3	95	32.9	75	24.8

Node	IPS		ASID		SH	
Perturbance Percentage	Value	Results	Value	Results	Value	Results
-50%	0	18.3	0	17.3	25	16
-25%	0	18.3	25	19.9	50	19.2
-20%	0	18.3	30	20.4	55	19.9
-10%	10	20.4	40	21.5	65	21.2
0%	20	22.5	50	22.5	75	22.5
10%	30	24.6	60	23.5	85	23.8
20%	40	26.6	70	24.5	95	25.1
25%	45	27.7	75	25	100	25.7
50%	70	32.9	100	27.6	100	25.7

Table 4.3.f - Absolute variation perturbation test results for each non-deterministic middle node on the BN

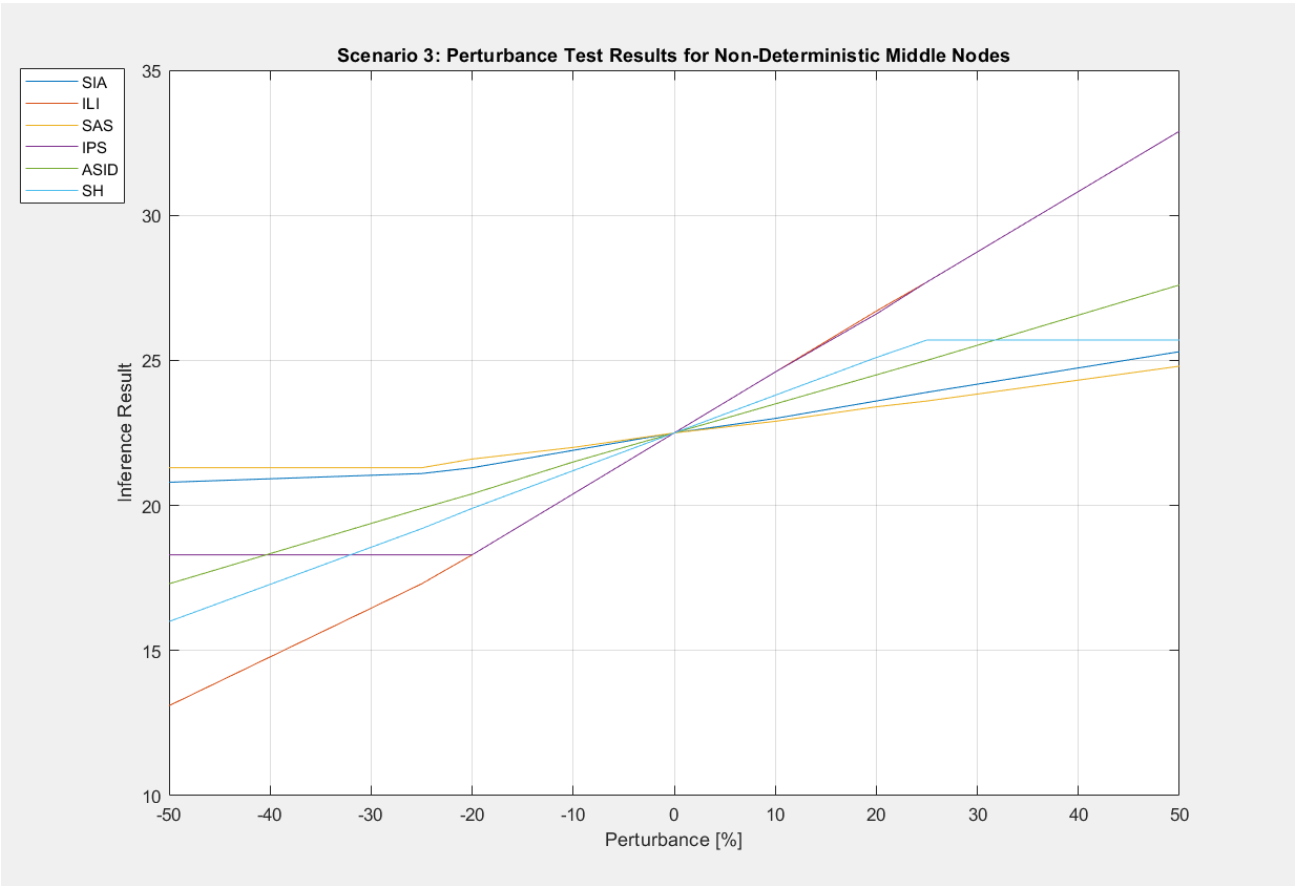

Fig 4.3.g - Absolute variation perturbation test results for Scenario 3 plotted in MATLAB.

5. Analysis

In order to make an assessment on the sensibility of the BN generated, two different sensitivity tests were performed. The first perturbance test applies relative variations. To do this I modified all leave nodes one at the time, by increasing/decreasing the estimated probabilities values by 5%, 10%, 15%, 20%, 25% and up to 50% relative to the originally estimated value (multiplication). The second perturbance test applies absolute variations. The approach is similar to the first test, but this time the estimated probability values are increased/decreased by 5%, 10%, 15%, 20%, 25% and up to 50% (simple addition). Results are reported in Chapters 4.2.1, 4.2.1 and 4.2.3 for scenarios 1, 2 and 3 respectively. Results were plotted in a graph where each node was assigned a label to ease identification. In each graph presented, the X axis indicates the perturbation percentage applied, while the Y axis represents the overall BN inference result on the isolated modification of that single probability. Each test result is represented in a different colour on the graph.

Results shown when setting the probability to 100% (or 'True') simulate what happens when the corresponding event is directly recognized by DETECT and hence updated in the BN. Regarding the criteria for generating the warnings or alarms to the SIEM, they can be based on pre-set thresholds on the root node probability or even percentage increase with respect to the initial value.

5.1 Scenario 1

For the first test on relative variations, the results indicate a perturbance in the range of -25% and +25% in each isolated leaf node, keeping the overall result of the inference within an acceptable margin-error threshold in range of positive-negative 10.56% from the outcome on the originally estimated probability of 19.99%. This margin shrinks significantly when the original estimated value reached the 100% value (i.e. 'True'). This indicates that higher estimated probabilities are more prone to perturbation errors, and this can be noticed on the pronounced slope of variables with high probability estimations. For instance, all middle nodes have estimated values of at least 60% and higher, and as a result they are more prone to sensibility perturbance. That means a greater attention must be paid in the estimation and fine-tuning of those values compared to leave nodes. Obviously, there is an upper bound limit on the perturbation range when an estimated value reaches 100%, and this explains the flat lines on the BD and AIC node perturbation results; conversely, negative perturbation values have a lower bound value of 0% (i.e. 'False'). For this first scenario, the average probability value is of 32%.

On the absolute variation test results, the impact of perturbation is higher. The second test results on absolute variation perturbation, indicate that in the range of -25% and +25% perturbances are in a range between -8.5% and +73.36% of the originally estimated value for all nodes. Nonetheless, in the range of -10% and +10%, the perturbance range is between -18.09% and +29.14% of the originally estimated value. Since perturbations are higher, upper and lower bounds are reached more often than in the first test. On the range -25% and +25% the average maximum obtain value is 27.11% (+36.23% more from the originally estimated value of 19.9%) and the average minimum obtained value is 15.95% (-19.84% less from the originally estimated value of 19.9%). Nonetheless, On the range -10% and +10%, the average

maximum obtain value is 23.01% (15.62% more from the originally estimated value of 19.9%) and the average minimum obtained value is 17.74% (-10.85% of the originally estimated value of 19.9%).

5.2 Scenario 2

The results for the second scenario relative variation perturbance test show a perturbance in the range of -25% and +25% in each isolated leaf node, keeping the overall result of the inference within an acceptable margin-error threshold in range of positive-negative 5.80% from the outcome on the originally estimated probability of 51.70%. This scenario is much less prone to perturbance than the first scenario presented. This is due to the fact that most estimation have extremely low values that are below a 20% threshold, and on average correspond to 15%, in contrast to an average of 32% from the first scenario, suggesting a direct correlation between the average percentage assigned and the level of perturbation sensibility on the BN. In contrast to the other scenarios analysed, this scenario counts with a node that has an estimation of 0% (labelled PTF), and as a result, extremely low arbitrary values were assigned for the perturbation testing. This scenario reaches an upper bound on EF and a lower bound on PTF as reflected on the graphics.

The second test results on absolute variation perturbation indicate different results. In the range of -25% and +25% perturbance, the results range from -23.4% and +29.20 from the originally estimated probability of 51.7%, with an average minimum value of 48.21% (6.75% less from the originally estimated value) and an average maximum value of 63.38% (11.68% more from the original value). This is result of the low values of the given estimations, with an average estimation of 15% in contrast to 32% from the first scenario. In the range of -10% and +10% perturbance, the results range from -11.6% and +11.79 from the originally estimated probability, with an average minimum value of 49.42% (4.41% less from the originally estimated value) and an average maximum value of 56.42% (9.13% more from the originally estimated value).

5.3 Scenario 3

The results from the last scenario confirms that relative variation perturbance is in the range of -25% and +25% in each isolated leaf node, keeping the overall result of the inference within an acceptable margin-error threshold in range of positive-negative 10.22% from the outcome on the originally estimated probability of 22.5%. This last scenario also presents less sensitivity to parameter perturbation, the average probability value is of 29%, and the results are similar to those on the first scenario presented, with similar values and perturbation sensibility. No upper or lower bounds are reached on this scenario.

The second test results on absolute variation perturbation indicate higher impact. In the range of -25% and +25% perturbance, the results range from -23.11% and +29.33% from the originally estimated probability of 22.5%, with an average minimum value of 20.06% (10.84% less from the originally estimated value) and an average maximum value of 25.78% (14.57% more from the original value). In the range of -10% and +10% perturbance, the results range from positive-negative 12% from the originally estimated probability, with an

average minimum value of 21.16% (5.95% less from the originally estimated value) and an average maximum value of 23.92% (6.31% more from the originally estimated value).

Contrary than the relative relation perturbance results, upper and lower bounds are easily reached on this test scenario.

5.4 Overall Analysis

In all three scenarios perturbation tests implementing relative variations demonstrate a low sensibility on the modelled BNs. With a relative variation of positive-negative 25% inside an acceptable threshold of ~10% of the originally estimated value. The impact of relative variations depends on the estimated probability of each node. The results in all perturbation tests indicate that higher estimated probabilities are more prone to perturbation errors. This can be noticed on the pronounced slope of variables with high probability estimations in all three scenarios, indicating that some parameters are more sensitive than others. As a result greater attention must be paid in the estimation and fine-tuning of nodes with higher estimated probabilities in contract to those of lower probabilities.

Absolute variations are of course, not as dependant on the estimated values as in relative variations but are a good parameter to measure the impact of possible misrepresentation of probabilistic values. In all three scenarios, perturbation tests implementing absolute variations demonstrated a higher sensibility in comparison to relative variations. Nonetheless, on a range of positive-negative 10% perturbance, the average minimum and maximum values obtained are in average inside an acceptable threshold of ~10% of the originally estimated values, revealing an acceptable perturbation sensibility results in all three scenarios presented.

6. Discussion

Companies, governments, experts and scholars around the world struggle to keep modern cyberthreats in line. A promising solution to the future of cybersecurity is the implementation of different AI techniques such as Bayesian Networks. Different studies on the implementation of BN to mitigate cyberattacks have had generally positive outcome. From the papers reviewed for this project, it is important to remark that most of the systems and frameworks studied have had positive results when applied in different fields of cybersecurity, and for different purposes. The logical and mathematical underpins of BN are perfect for inferring results when presented with partial observations and uncertainty. Nonetheless, after surveying the related literature, I could not find any information about any existent BN based security system that it is widely implemented on a large scale. Moreover, studies suggest that only 3.4% of organizations worldwide implement any AI based automated security implementation on their systems [15].

The results from the attack scenarios I have modelled, which included top vulnerabilities that are most exploited today, suggested a low sensitivity to parameter perturbation, with a lower error margin for absolute variations than for relative variations, confirming the feasibility of implementing BN techniques as part of a security framework.

The implementation of Attack Trees and Bayesian Networks on the proposed model of this thesis project, would allow us to make stochastic inferences in non-deterministic scenarios. The model presented provides with framework integration, a modular architecture scheme, and a formal language for threat modelling. This can be combined with a complete framework such as DETECT, that includes a database and a detection engine. Moreover, BN simplify probabilistic analysis by automating most computations including forward and backward inference. BN also allow modellers to manage the inherent uncertainty in threat scenarios by using aspects of fuzzy logic and stochastic inference. This features and characteristics are desired for a rapid adaptation on an everchanging and expeditious cybersecurity landscape.

7. Conclusion

It's important to understand the state of today cybersecurity, not only on its technical aspects, but on its tremendous impact in society, democracy, the business industry, and the world's economy overall. Online threats and vulnerabilities have exponentially grown along with the Internet, the largest human endeavour on history. As the usage of internet and computers continues to increase exponentially through the past decades, with over 4 billion users word wide, vulnerabilities in cyberspace have now become an everyday threat [18]. Knowledge on the latest technologies, and on the procedure that attackers implement, can significantly improve the defence against such attacks. Nonetheless, the intrinsic complexity and ever-changing nature of online threats can render this task extremely difficult. The future of cybersecurity strongly depends on the application of Artificial Intelligence methods to improve cyber situational awareness and to automate threat detections and countermeasures. Statistical and probability-based models, such as Bayesian Networks, can be very powerful tools. Based on conjectures derived from Bayes models, it is possible to automatize the best course of action for online defence [14]. By retrieving knowledge from previous studies, we can infer the course of action taken by attackers and systematize them into a framework.

Previous studies on the implementation of Bayesian Networks in cybersecurity recognized three parameters for the construction of stochastic models: modularization of all components in the framework, importance of the credibility of the data use for the population of the CPT tables, and low sensitivity to parameter perturbation. Following the architecture of the DETECT framework as a mean of inspiration, I have designed a blueprint for the application of Bayesian Networks for online threat detection that implements hybrid-detection analytics and complies with the recommendations from the existent literature.

For a static assessment, I have implemented estimated probabilities for all leave nodes in the BN, by consulting reputable sources and available statistics. The resulting BNs from the proposed scenarios were modelled using Netica, in order to proceed with the analysis of the resulting BNs. The results of the sensitivity analysis revealed an overall low perturbance sensitivity, with insignificant variations. When applying variations relative to the inferred probabilities, perturbations are inside a 10% threshold of acceptance. Therefore, a margin of error of 25% relative to the inferred value is acceptable and will not impact negatively on the overall probability outcome. Special attention must be paid when nodes have higher probabilities and miscalculations will have a bigger impact on the results. Moreover, when the variations applied are absolute, the margin of error lowers to 10% in order to get similar results.

The sensibility analysis of the presented model can help a security analyst to recognized the most influential factors on a global threat, in order to adjust the protection mechanism accordingly. This could save time and money, as well as help reallocate other resources in a more efficient manner.

The proposed model has been proved to be feasible, and it could be further developed into a fully functional implementation, that could be integrated into DETECT. The design presented can be used for off-line risk assessment and on-line risk evaluation/threat recognition. Even thought that for practical purposes I have implemented hypothetical probabilistic values, for

future implementations the data feed to the algorithm can be inferred from available datasets or studies. Data can also be harvested using collection techniques such as honey pots and attack simulations. In order to improve the performance of the proposed model, the probabilities could later be refined using Machine Learning techniques. The reliability of the proposed model relies on the power of Bayesian Networks for stochastic assessments in indeterministic scenarios, and it has been proven to be a potentially powerful tool, that will contribute to the improvement of cyber situational awareness.

7.1 Future research

Future research will aim at:

(i) Refining the methodology with Machine Learning approaches to take into account user's feedback for dynamic BN model update.

(ii) Developing a full set of generic cyberthreat scenarios to be used as a BN library and customized to specific organizations.

(iii) Completing the ongoing prototype development and full software integration of BN detection modules in the DETECT framework.

(iv) Real world experimentation after finalizing the module integration into the DETECT framework.

(v) Developing an enhanced software tool for guided construction of BN threat models based on the work presented on this Thesis.

Appendix 1: Basic Terminology

Antivirus: a software that is intended for detecting and destroying malware software [49].

Artificial Neural Networks (ANN): is a computational model that is inspired by the structure, processes and learning abilities of the human brain. An Artificial Neural Network counts with a series of nodes (neurons) that are divided in different layers or subsets and connected with each other with a series of weights (edges). The distribution of neurones, weights and layers correspond to a distributed representation of knowledge within the system that is able to produce an output from a given input. ANNs can be trained in different environments in order to improve the result from its inferences. ANN models are mostly applied for pattern classification, clustering, categorization, and prediction among other applications [50].

Attack Graphs: is any conceptual graphical representation intended to represent or identify an attack scenario in a system. In information technology, attack graphs are generated by security analysts in order to have a simplified overview of the systems behaviours and parameters, and to infer vulnerabilities and possible threats. Attack graphs are designed to map the system's front-lines, and to generate or optimize protective countermeasures [51].

Attack Trees (AT): is a conceptual representation of a known attack scenario of a system using tree shaped graphical diagrams. In the field of information technology and cybersecurity, attack trees are implemented to map all possible scenarios and ramifications that present a threat to a computer system. Different attack trees vary on formalities and schemes. Some attack trees models can present weights on their leaves, to indicate probability, or other unit of measurement. An attack tree might engulf all possible attack scenarios, or only a subset of scenarios involving a specific subsystem or other criteria. Attack trees are generated to capture existing adversities, and to generate or optimize countermeasures to protect a system [52].

Back Door: as the name indicates, a back door is an alternative mean of access to a computer system that bypasses any front security mechanism. An attacker will try to find and force any back door that might be installed in a computer system, or even try to install a back door without the user's consent or acknowledgement [53].

Bayesian Network (BN): also known as *Belief Networks, Knowledge Maps, Probabilistic Casual Networks* and more, are statistical graphical models representing a set of variables (nodes) and their conditional dependencies or possible state transitions (edges). Conditional dependencies are represented as a directed acyclic graph, where probability values give a weight to the edges of the graph. To specify the probability distributions in a Bayesian Network, it is necessary to know the probability of all the root nodes and middle nodes given all possible path combination to a desired node. Probabilities are updated and presented in Conditional Probability Tables (CPT). It is possible to infer the probabilities to different paths in the network by observing all circumventing nodes. BNs are used for probability analysis, probability based knowledge systems, predictions, and map learning among many other applications [26].

Brute Force Attack: also known as *brute force cracking, password guessing attack* or simply *brute force*, is an automated trial-and-error method used to obtain access to a secured system by trying a significant large number of predefined credentials. This method can be used to infer, for example, usernames, passwords, PIN numbers, passphrases or any other sort of access credential. A brute force attack can, for instance, use all the words in a dictionary to try to learn a password to a secure system (known as a *dictionary attack*), or use an algorithm specifically designed to try different combinations of commonly used patterns of words, numbers or other characters. Brute force attacks are resource-consuming, and their success usually rely on computational power rather than algorithmic complexity [54].

Common Vulnerability Scoring System (CVSS): is a framework developed by the U.S. government National Institute of Standards and Technology (NIST) and implemented in the National Vulnerability Database (NVD) for measuring common vulnerabilities. CVSS provides with a scoring system to measure the impact and severity of common system vulnerabilities when used on system exploitations. CVSS v3.0 measures severity based on a score that ranges from 0.0 to 10.0. Vulnerabilities are label according to their severity as follow: None (0.0), Low (0.1 to 3.), Medium (4.0 to 6.9), High (7.0 to 8.9) and Critical (9.0 to 10.0) [55].

Common Weakness Enumeration (CWE): is a web-based database of known vulnerabilities. This big community project is aimed to create automated tools to help identify, fix and prevent flaws during software development. It is maintained by the *National Cybersecurity FFRDC*, which is federally founded by the United States government, and owned by the *MITRE Corporation* [56].

Credential Stuffing: similar to a *brute force attack*, credential stuffing automatically implements breached username and password pairs or other sort of leaked credential datasets in order to get access to a secure system with a trial-and-error approach [57].

Data Mining (DM): is a process of inferring patterns and new data from the analysis of a large dataset (raw data) with the use of analysis tools and computational methods related to database systems, statistical analysis, Artificial Intelligence and Machine Learning techniques among other technologies [58].

Decision Triggering Event Composer and Tracker (DETECT): is a security framework that was developed in collaboration by *Ansaldo STS Italy* and the *University of Naples Federico II*. This security framework was originally developed as an automatic, early threat detection system for critical infrastructures. DETECT is a quick, fully automatic, emergency response system, and uses model analysis and sequence of events recognition in order to recognize known threats patterns and attack signatures [17].

Defence Advanced Research Projects Agency (DARPA): is a governmental, technology development and research agency that is part of the *United States Department of Defence*. DARPA was originally created for military-grade scientific research, and development of new technologies aimed to national security and economic development in the United States. DARPA is well known for developing the early prototype and founding technology for the modern Internet [59].

Encoder: is a program that obfuscates data using different encryption systems and other cryptographic techniques in order to hide or conceal information [60].

Exploit: an exploit refers to any means that can be used by an attacker for taking advantage of a target system's known vulnerability in order to gain access to it [61].

Firewall: a firewall is a security system designed to monitor network traffic according to a set of pre-established security rules. This way, a firewall acts like a filter between a trusted internal network and a wider outside network such as the Internet [62].

Honeypots: as the euphemism indicates, a honeypot is a server, system or token, that camouflaged as an attractive target (vulnerable system), is used as a bait to collect information on cybersecurity attacks [63].

Industrial Control Systems (ICS): this term encompasses computer systems that are implemented as control systems in different process and tools used in industry sectors, including: energy production, water, commercial factories, oil refineries, petrol tower stations, gas pipeline control and chemical processing plants among others [64].

Internet of Things (IoT): named assigned to the set of devices, vehicles, home appliances, and other physical items that have embedded software allowing network intercommunication, and that in its entirety encompass the global infrastructure for the *information society* [65].

Intrusion Detection Systems (IDS): an Intrusion Detection System monitors activity in order to detect any malicious attacks within a system that is set to protect. IDS rely on *SIEM* as knowledge base, and can be embed in hardware tools, programmed as part of a software system, or be completely independent software tools. Moreover, IDS can act in different levels and in different parts of a system. For instance, IDS can be used to monitor a network or act locally monitoring local processes and files [66].

Keylogging: also known as *keystroke logging* or *keyboard capturing*, is a monitoring software or hardware device that captures and records the key strucks on the keyboard of a target machine. The sequence of key strucks can be later retrieved by the attacker, with the purpose of revealing information such as login credentials, private data, credit card numbers and more [67].

Machine Learning*:* refers to the application of Artificial Intelligence technology in order to make computer systems capable of automated learning and improvement by experience and analysis of data. This is achieved with observation of data, implementing supervised machine training through examples, with the aim of reaching a pace of learning that does not require any human supervision [68].

Malware: an abbreviation for *malicious software*, it refers to any program designed to interrupt the normal functioning of a computer system. Malware is commonly used to exploit, gather information, display advertisement, or tempt the computer system in any way, without the user's consent or acknowledgement [69].

Man in the Middle (MITM): a *man in the middle* attack refers, as the metaphor suggests, to the possibility of intercepting a communication channel. Parties involved are deceived into thinking that they are communicating with each other in a secured channel. This includes not only the interception of the communication itself, but also the injection of misleading

messages and deceiving data or information [70].

National Security Agency (NSA): also referred as *Central Security Service (CSS)*, is a governmental entity and the intelligence wing of the *United States Department of Defence*. In order to safeguard the nation's best interests, the NSA is responsible for cryptology efforts that include *Signals Intelligence* (SIGINT) and global data monitoring and processing of information, referred as *Information Assurance* (IA). Moreover, the NSA is also responsible for the protection of communication networks and information systems within the United States [71].

Packet Sniffing: the act of intercepting packets of data in a network communication channel. This is done throughout a packet monitor software or hardware that can intercept and record traffic from a given network interface. *Packet sniffing* allows the attacker to observe the network traffic of a target system [72].

Payload: is any code that an attacker runs on a compromised target system. There are different payloads that have different purposes. A payload can be generic or platform specific, and it could be as simple as a few lines of code, or complex as a large software [73].

Penetration Testing: is a simulation of a real cyberattack that is performed, with the consent of the user, on a computer system by a security professional. During the simulation, the penetration tester intends to discover and exploit the target's vulnerabilities and weaknesses. After the exploitation, a tester will write a thorough report on all security flaws that were discovered, valued information that was compromised, and the severity of any potential damage resulted from the attack simulation [74].

Scenario: a single or a group of exploits coordinated together to compromise a system. A scenario is defined according to all pertinent information gathered about a target system towards which the attack is aimed [75].

Security Management Software (SMS): refers to software that is developed to aid on the management of security violations and threats. SMS systems aid security analysts and monitoring operators on the course of action after an immediate threat is detected in a system that is under protection. SMS software usually follows a deterministic flow of action, and aims to centralize the security environment by integrating heterogeneous computer hardware, software and other technological systems [76].

Security information and Event Management (SIEM): *SIEM* is the name given to the combination of *Security Information Management (SIM)* and *Security Event Management (SEM)* implementations in cybersecurity. The basic idea is to analyse data from multiple sources, identify abnormalities and act upon any possible threats, by triggering alerts and security controls. *SIEM* can be based on a determined set of rules or a statistical model for the analysis of data [77].

Session Hijacking Attack: an attack that exploits the session control mechanisms of a computer system. This is often achieved by stealing or predicting a valid session control token and using it to impersonate another user in order to gain unauthorized access to a secured system. The most common ways in which a session hijacking attack occurs are *MITM* attacks, the use of malicious scripts and *viruses*, *authenticated session sniffing* and the

use of predictable session tokens [78].

Social Engineering: defined within terms of information technology and cybersecurity, *social engineering* refers to the possibility to exploit human weaknesses in order to perform a cyberattack on a target. This include any exploitable human aspect, such as credibility, curiosity, naiveté, avarice or ignorance. *Social engineering* attacks represents a great treat to many organizations [79].

Spyware: as the name indicates, spyware refers to any kind of *malware* or *virus* that has the specific intention of gathering information from a target system. Most malicious spyware programs will gather information such as personal data, computer usage information, among others. Information gathered is later transmitted back to the attacker over the Internet [80].

Supervisory Control and Data Acquisition (SCADA)*:* is a module-based, system control architecture pattern, that is used for controlling and supervising remote systems. Most systems that implement SCADA belong to industrial and manufacturing companies and organizations. SCADA architecture uses both hardware and software to achieve a distributed control over a remote system, including computer systems, network communication, online data gathering, standardized automation protocols, and secondary hardware among other devices [81].

The Open Web Application Security Project (OWASP): is a non-profit online community that is focused on best practices and open security standards. OWASP periodically issues security reports and software tools, along with thorough documentation and cybersecurity related papers. It also aims to inform the community of computer users on security issues through its wiki and blog. It can be accessed at https://www.owasp.org [82].

Totalförsvarets Forskningsinstitut (FOI): is the *Swedish Defence Research Agency*, a governmental institution dedicated to defence and security research based in the Scandinavian country. Among its extended research program, there is a big emphasis on IT security and human system interaction. FOI works closely with the *Swedish Ministry of Defence* and the *Swedish Army*, and is one of the leading authorities in the field of security within the *European Union* [83].

Trojan: a *Trojan Horse* or *Trojan* is, as the name suggests, any apparently benign program, such as an office application or a game, which carries additional hidden malware that serves to exploit a target system [84].

Unified Modeling Language (UML) Diagram: UML is a standardized, general purpose, graphical representation scheme of a software system and the interaction between all of its parts [85].

Virus: a virus is an insidious program or script that deliberately waspish or deteriorates the normal functioning of a computer system with a particular goal, such as information gathering, data tampering or destruction, or deliberate generation of a system failure [86].

Vulnerability: is any software misconfiguration that can be taken advantage of or exploited. This include not only flaws and errors, but lack of features, lack of encapsulation and security

holes present in any part of a sub-system, service, or application of a system. Vulnerabilities can potentially be exploited by attackers in order to damage or compromise a system [87].

Vulnerability Assessment: an assessment that includes the implementation of different manual and automated tools in order to discover vulnerabilities in a system. These tools include scanners and scripts that can report findings and generate alerts, once a potentially exploitable vulnerability is found. *Penetration testing* is one of the most effective *vulnerability assessment* schemes today [88].

Worm: is a *virus* that can rapidly multiply itself and travel across different systems and networks. In the form of an invisible file, a worm is usually hard to detect, and normally does not modify any data in a system, but rather flows the memory and hard disk space causing said system to malfunction or crash [89].

Zero-day Vulnerability: is a vulnerability that has been recently discovered and can be easily exploited by attackers since there are no known security measures to counter fight a potential attack [90].

Appendix 2: Most common cyberattacks in 2017

OWASP's top 10 vulnerabilities in 2017

Several information security organizations and communities provide with extensive databases on known vulnerabilities. The *Common Weakness Enumeration* (CWE) is one of the biggest endeavours in vulnerability research and documentation. Moreover, OWASP periodically publishes a report on the most common vulnerabilities that afflict the cyberspace. Below, I have included the release of the most common vulnerabilities as for 2017, along with a CWE code for information retrieval on their online database [91].
These attacks are often combined together, or with other complementary attacks.

OWASP - Top 10 Vulnerabilities in 2017		
Rank #	Vulnerability	Code to CWE Entries
A1	Injection	CWE -77 -89 -564 -611
A2	Broken Authentication and Session Management	CWE -287 -384
A3	Sensitive Data Exposure	CWE -310 -312 -319 -326
A4	XML External Entities (XXE)	CWE -611
A5	Broken Access Control	CWE -285 -639 -22
A6	Security Missconfiguration	CWE -2
A7	Cross Site Scripting (XSS)	CWE -79
A8	Insecure Deserialization	CWE -502
A9	Using Components with known vulnerabilities	Not Available
A10	Insufficient Logging & Monitoring	CWE -223 -778

Table A2 – 2017 release of OWASP's top 10 vulnerabilities

A1 – Injection

Injection attacks are the most common threats that attain computer systems today. These attacks corrupt databases by modifying entries without authorization. They affect primarily websites and web applications. During an injection attack, information can be compromised, stolen, modified or deleted in a matter of seconds. The leakage of sensitive information can compromise both user privacy and the integrity of the company [92].

A2 - Broken authentication and session management

As the name indicates, this vulnerability is strictly related to the user's database table and/or session management of a system. Primary targets include websites, software applications and computer networks. When authentication is violated, an external attacker can access a computer system and impersonate another user. This can be achieved by hijacking a session, or by intercepting login information. This security breach is one of the most common vulnerabilities today, and it can be extremely severe depending on the sensitivity of the data that might be compromised [93].

A3 - Sensitive Data Exposure

Any vulnerability that includes the exposure of sensitive data, as a result of the use of weak ciphers, lack of encryption or other proper security measures. Data can be regarded as

sensitive because of many reasons, for instance its content could compromise user privacy, damage the integrity of the company or jeopardize an entire computer system [94].

A4 – XML External Entities (XXE)
This vulnerability relates to the exploitation of XML processors. Similar to an injection attack, attackers can upload hostile XML content in order to exploit vulnerable code, integrations or other dependencies. Once an attacker exploits these flaws, access to data, execution of remote requests, and the scanning of internal systems can take place [95].

A5- Broken Access Control
Temping the data of a website can occur when the attackers get unauthorized access to parts of the system. A very common security breach, broken access control, occurs as a product of negligence on file permissions, when files that should be restricted can be accessed directly from an URL. Sometimes, the name of the paths to unauthorised files or folders can be easily guessed. By accessing an index, the attacker could read data that is sensitive or confidential, and this could include databases or other components. Sometimes, this data might be central to the functionality of a system, and if compromised it would ensure a disaster. This vulnerability is very common and has an easy solution. Target resources must have permissions updated and must be encrypted when dealing with sensitive information [96].

A6 - Security Misconfiguration
Any vulnerability that directly involves a security flaw or misconfiguration can be exploited to obtain unauthorized access, or usually restricted knowledge on the functionality of the system. Some resources that are vulnerable to security misconfiguration include: default accounts, unused pages, unprotected files and directories among other targets [97].

A7 - Cross-site scripting (XSS)
Cross-site scripting vulnerabilities are found in websites in which an attacker is able to inject an external malicious script without authorization. These scripts are mostly client side execute, and they are usually used for identity theft, by retrieving sensitive data such as log in information or credit card numbers back to the attacker. A cross-site scripting attack is said to be persistent (stored) when the external script used during the attack is saved on the server, whereas on a non-persistent cross-site scripting attack (reflected), the server will reflect the malicious script to be executed by the victim's browser [98].

A8- Insecure Deserialization
A deserialization attack occurs when there is a misuse or introduction to malicious code during the conversion of an object into a different format or data structure. Examples of formats that are commonly serialize/deserialize online include XML, JSON, and YAML among others [99].

A9 - Using Components with known vulnerabilities
A very common vulnerability is the lack of information. When using any component with security flaws and potentially exploitable errors that are known to attackers, we are by proxy opening a hole in our system's security. This can be a product of unpatched or out of date software. This includes not only main software, but also components, libraries and external APIs. Not having enough information on versions and patches might leave the door open to attackers [100].

A10- Insufficient Logging and Monitoring
This vulnerability is based on lack, or insufficient, logging and monitoring of security-critical events that occur during the execution of a computer system. Consequently, the system does not count with enough information to decide the correct course of action, or even detect the existence of the incident [101].

Other common vulnerabilities exploited that are not on the top 10 OWASP yearly security report, but that are worth mentioning include:

Insufficient Attack Protection
Even though most applications and APIs might detect attacks and invalid input, a significant number of systems and subsystems do not count with sufficient attack protection. It is important to detect and block an attacker that might be in search of new vulnerabilities and exploitable errors on a system, in order to keep any existent security holes from being discovered [102].

Cross-Site Request Forgery
A malicious website can forge a *cookie* or certificate request that is not distinguishable from an original request, in order to compromise private information from an unaware user [103].

Under protected APIs
Modern systems implement all sorts of API and external libraries. Failing to understand the level of protection of any backend API that is in use could result on compromising the entire system. These subsystems can include micro-services, services, and other endpoints [104].

DoS and DDoS attacks
Denial of Services (DoS) attacks occur in networks that are vulnerable to receiving simultaneous access request from unwanted traffic, and as a result of this, prone to be overflown and taken down. These attacks can be potentiated when done through distributed systems. Distributed Denial of Services (DDoS) attacks are often aided with the propagation of worms, or other malware of pervasive nature that take advantage of a vulnerable system. When various systems are orchestrated, a DDoS attack can take a target down for hours, days or even weeks, and generally cause millions of dollars in damages. This sort of vulnerabilities are often exploited for financial gain, illegitimate competition or political reasons [105].

Appendix 3: Scenario 1 XML code

```
<?xml version="1.0" encoding="UTF-8"?>


<bayes_network>
  <!-- Scenario 1: Attacker gets Access to Internal System -->

  <!-- Define Nodes-->
  <node type="leaf" id="un">User Connects to Untrusted Network</node>
  <node type="leaf" id="mw">User Access Malicious Website</node>
  <node type="leaf" id="im">User Connects Infected Removable Media to the System</node>
  <node type="leaf" id="iw">User Access Website Infected with Malware</node>
  <node type="leaf" id="spe">User Opens Spear Pishing Email</node>
  <node type="leaf" id="zdv">Exploitation of Zero-Day Vulnerability</node>

  <node type="middle" id="kv">Using Components with Known Vulnerabilities</node>
  <node type="middle" id="euv">Exploitation of Unpatched Vulnerability</node>
  <node type="middle" id="bd">Attacker Installs Backdoor on Target System</node>
  <node type="middle" id="aic">Attacker Gets Access to Internal System</node>

  <node type="root" id="av">Authentication Violation</node>

  <!-- Assign Node Relations -->
  <relation parent="un" child="kv" configuration="OR"></relation>
  <relation parent="mw" child="kv" configuration="OR"></relation>
  <relation parent="im" child="kv" configuration="OR"></relation>
  <relation parent="iw" child="kv" configuration="OR"></relation>
  <relation parent="spe" child="kv" configuration="OR"></relation>
  <relation parent="kv" child="euv" configuration="OR"></relation>
  <relation parent="euv" child="bd" configuration="OR"></relation>
  <relation parent="zdv" child="bd" configuration="OR"></relation>
  <relation parent="bd" child="aic" configuration="OR"></relation>
  <relation parent="aic" child="av" configuration="OR"></relation>

  <!-- Populate Leave Nodes CPTs -->
  <probability node = "un">
    <state label='True'>0.24</state>
    <state label='False'>0.76</state>
  </probability>

  <probability node = "mw">
    <state label='True'>0.08</state>
    <state label='False'>0.92</state>
  </probability>

  <probability node = "im">
    <state label='True'>0.02</state>
    <state label='False'>0.98</state>
  </probability>

  <probability node = "iw">
    <state label='True'>0.09</state>
    <state label='False'>0.91</state>
  </probability>

  <probability node = "spe">
    <state label='True'>0.03</state>
    <state label='False'>0.97</state>
  </probability>
```

```
  <probability node="zdv">
    <state label='True'>0.03</state>
    <state label='False'>0.97</state>
  </probability>

  <!-- Populate Middle Nodes CPTs -->
  <probability node="euv">

    <conditional node= "kv" state="True">
      <state label='True'>0.6</state>
      <state label='False'>0.4</state>
    </conditional>

    <conditional node="kv" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional >

  </probability>

  <probability node="bd">

    <conditional node="euv" state="True">
      <state label='True'>0.85</state>
      <state label='False'>0.15</state>
    </conditional>

    <conditional node="euv" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

  </probability>

  <probability node="aic">

    <conditional node="bd" state="True">
      <state label='True'>0.9</state>
      <state label='False'>0.1</state>
    </conditional>

    <conditional node="bd" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

  </probability>

</bayes_network>
```

Appendix 4: Scenario 2 XML code

```
<?xml version="1.0" encoding="UTF-8"?>

<bayes_network>
  <!-- Scenario 2: Obtaining Username and Password  -->

  <!-- Define Nodes-->
  <node type="leaf" id="ptf">Plain Text File</node>
  <node type="leaf" id="ef">Encrypted File</node>
  <node type="leaf" id="cs">Credential Stuffing</node>
  <node type="leaf" id="ga">Guessing Attack</node>
  <node type="leaf" id="bfa">Brute Force Attack</node>
  <node type="leaf" id="si">Stolen Information</node>
  <node type="leaf" id="cf">Coersive Force</node>
  <node type="leaf" id="xsf">Cross-Site Request Forgery</node>
  <node type="leaf" id="kl">Key Logger</node>
  <node type="leaf" id="se">Social Engineering</node>

  <node type="middle" id="df">File is Decrypted</node>
  <node type="middle" id="fs">From System</node>
  <node type="middle" id="fu">From User</node>
  <node type="middle" id="oup">Obtaining Username and Password</node>

  <node type="root" id="av">Authentication Violation</node>

  <!-- Assign Node Relations -->
  <relation parent="ef" child="df" configuration="OR"></relation>
  <relation parent="ptf" child="fs" configuration="XOR"></relation>
  <relation parent="df" child="fs" configuration="XOR"></relation>

  <relation parent="si" child="fu" configuration="OR"></relation>
  <relation parent="cf" child="fu" configuration="OR"></relation>
  <relation parent="xsf" child="fu" configuration="OR"></relation>
  <relation parent="kl" child="fu" configuration="OR"></relation>
  <relation parent="se" child="fu" configuration="OR"></relation>

  <relation parent="fs" child="oup" configuration="XOR"></relation>
  <relation parent="fu" child="oup" configuration="XOR"></relation>
  <relation parent="cs" child="oup" configuration="XOR"></relation>
  <relation parent="ga" child="oup" configuration="XOR"></relation>
  <relation parent="bfa" child="oup" configuration="XOR"></relation>

  <relation parent="oup" child="av" configuration="OR"></relation>

  <!-- Populate Leave Nodes CPTs -->
  <probability node = "ptf">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </probability>

  <probability node = "ef">
    <state label='True'>1.0</state>
    <state label='False'>0.0</state>
  </probability>

  <probability node = "cs">
    <state label='True'>0.15</state>
    <state label='False'>0.85</state>
  </probability>
```

```
  <probability node = "ga">
    <state label='True'>0.05</state>
    <state label='False'>0.95</state>
  </probability>

  <probability node = "bfa">
    <state label='True'>0.03</state>
    <state label='False'>0.97</state>
  </probability>

  <probability node = "si">
    <state label='True'>0.02</state>
    <state label='False'>0.98</state>
  </probability>

  <probability node = "cf">
    <state label='True'>0.01</state>
    <state label='False'>0.99</state>
  </probability>

  <probability node = "xsf">
    <state label='True'>0.12</state>
    <state label='False'>0.88</state>
  </probability>

  <probability node = "kl">
    <state label='True'>0.02</state>
    <state label='False'>0.98</state>
  </probability>

  <probability node = "se">
    <state label='True'>0.20</state>
    <state label='False'>0.80</state>
  </probability>

  <!-- Populate Middle Nodes CPTs -->
  <probability node="df">

    <conditional node= "ef" state="True">
      <state label='True'>0.05</state>
      <state label='False'>0.95</state>
    </conditional>

    <conditional node= "ef" state="False">
      <state label='True'>1.0</state>
      <state label='False'>0.0</state>
    </conditional>

  </probability>

</bayes_network>
```

Appendix 5: Scenario 3 XML code

```
<?xml version="1.0" encoding="UTF-8"?>

<bayes_network>
  <!-- Scenario 3: Obtaining Username and Password -->

  <!-- Define Nodes-->
  <node type="leaf" id="bac">Broken Access Control</node>
  <node type="leaf" id="bfa">Brute Force Attack</node>
  <node type="leaf" id="xsf">Cross-Site Request Forgery</node>
  <node type="leaf" id="uid">Unsanitized Input Data</node>
  <node type="leaf" id="dmm">Defence Mechanism Misconfiguration</node>
  <node type="leaf" id="ba">Blind Attack</node>
  <node type="leaf" id="ps">Packet Sniffing</node>
  <node type="leaf" id="xss">Cross Site Scripting (XSS) Attack</node>

  <node type="middle" id="sia">SQL Injection Attack</node>
  <node type="middle" id="ili">Intercepting Login Information</node>
  <node type="middle" id="sas">Sniffing Authenticated Session</node>
  <node type="middle" id="ips">IP Spoofing</node>
  <node type="middle" id="asid">Access to Stored Session ID</node>
  <node type="middle" id="sh">Session Hijacking</node>

  <node type="root" id="av">Authentication Violation</node>

  <!-- Assign Node Relations -->
  <relation parent="sia" child="uid" configuration="OR"></relation>
  <relation parent="sia" child="dmm" configuration="OR"></relation>

  <relation parent="ili" child="sia" configuration="OR"></relation>
  <relation parent="ili" child="bac" configuration="OR"></relation>
  <relation parent="ili" child="bfa" configuration="OR"></relation>
  <relation parent="ili" child="xsf" configuration="OR"></relation>

  <relation parent="sas" child="ps" configuration="OR"></relation>
  <relation parent="ips" child="sas" configuration="OR"></relation>
  <relation parent="ips" child="ba" configuration="OR"></relation>

  <relation parent="asid" child="xss" configuration="OR"></relation>

  <relation parent="sh" child="ips" configuration="OR"></relation>
  <relation parent="sh" child="asid" configuration="OR"></relation>

  <relation parent="av" child="ili" configuration="OR"></relation>
  <relation parent="av" child="sh" configuration="OR"></relation>

  <!-- Populate Leave Nodes CPTs -->
  <probability node = "bac">
    <state label='True'>0.10</state>
    <state label='False'>0.90</state>
  </probability>

  <probability node = "bfa">
    <state label='True'>0.03</state>
    <state label='False'>0.97</state>
  </probability>

  <probability node = "xsf">
    <state label='True'>0.12</state>
    <state label='False'>0.88</state>
```

```
</probability>

<probability node = "uid">
  <state label='True'>0.05</state>
  <state label='False'>0.95</state>
</probability>

<probability node = "dmm">
  <state label='True'>0.18</state>
  <state label='False'>0.82</state>
</probability>

<probability node = "ba">
  <state label='True'>0.30</state>
  <state label='False'>0.70</state>
</probability>

<probability node = "ps">
  <state label='True'>0.65</state>
  <state label='False'>0.35</state>
</probability>

<probability node = "xss">
  <state label='True'>0.20</state>
  <state label='False'>0.80</state>
</probability>

<!-- Populate Middle Nodes CPTs -->
<probability node="sia">

  <conditional node= "uid" state="True">
    <state label='True'>0.30</state>
    <state label='False'>0.70</state>
  </conditional>

  <conditional node= "uid" state="False">
    <state label='True'>0.00</state>
    <state label='False'>1.00</state>
  </conditional>

  <conditional node= "dmm" state="True">
    <state label='True'>0.30</state>
    <state label='False'>0.70</state>
  </conditional>

  <conditional node= "dmm" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>

</probability>

<probability node="ili">

  <conditional node= "bac" state="True">
    <state label='True'>0.45</state>
    <state label='False'>0.65</state>
  </conditional>

  <conditional node= "bac" state="False">
    <state label='True'>0.0</state>
```

```
    <state label='False'>1.0</state>
  </conditional>

  <conditional node= "bfa" state="True">
    <state label='True'>0.45</state>
    <state label='False'>0.65</state>
  </conditional>

  <conditional node= "bfa" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>

  <conditional node= "sia" state="True">
    <state label='True'>0.45</state>
    <state label='False'>0.65</state>
  </conditional>

  <conditional node= "sia" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>

  <conditional node= "xsf" state="True">
    <state label='True'>0.45</state>
    <state label='False'>0.65</state>
  </conditional>

  <conditional node= "xsf" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>

</probability>

<probability node="sas">

  <conditional node= "ps" state="True">
    <state label='True'>0.25</state>
    <state label='False'>0.75</state>
  </conditional>

  <conditional node= "ps" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>

</probability>

<probability node="ips">

  <conditional node= "sas" state="True">
    <state label='True'>0.20</state>
    <state label='False'>0.80</state>
  </conditional>

  <conditional node= "sas" state="False">
    <state label='True'>0.0</state>
    <state label='False'>1.0</state>
  </conditional>
```

```
    <conditional node= "ba" state="True">
      <state label='True'>0.20</state>
      <state label='False'>0.80</state>
    </conditional>

    <conditional node= "ba" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

  </probability>

  <probability node="asid">

    <conditional node= "xss" state="True">
      <state label='True'>0.50</state>
      <state label='False'>0.50</state>
    </conditional>

    <conditional node= "xss" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

  </probability>

  <probability node="sh">

    <conditional node= "ips" state="True">
      <state label='True'>0.75</state>
      <state label='False'>0.25</state>
    </conditional>

    <conditional node= "ips" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

    <conditional node= "asid" state="True">
      <state label='True'>0.75</state>
      <state label='False'>0.25</state>
    </conditional>

    <conditional node= "asid" state="False">
      <state label='True'>0.0</state>
      <state label='False'>1.0</state>
    </conditional>

  </probability>

</bayes_network>
```

Appendix 6: **List of acronyms used in alphabetical order**

AI - *Artificial Intelligence*
ANN - *Artificial Neural Networks*
AT - *Attack Tree*
BN - *Bayesian Network*
CALM - *Correlated Attack Modeling Language*
CIP - *Critical Infrastructure Protection*
CPT - *Conditional Probability Table*
CVSS - *Common Vulnerability Scoring System*
CWE - *Common Weakness Enumeration*
DA - *Data Analytics*
DAG – *Directed Acyclic Graph*
DBN - *Dynamic Bayesian Network*
DETECT - *Decision Triggering Event Composer and Tracker*
DM - *Data Mining*
DoS – *Denial of Service* (attack)
DDoS – *Distributed Denial of Service* (attack)
EDL - *Event Description Language*
FFNN - *Fast Forward Neural Networks*
FOI - *Totalförsvarets Forskningsinstitut* (Swedish Defence Research Agency)
FT - *Fault Tree*
ICS - *Industrial Control System*
IDS - *Intrusion Detection System*
KDD - *Knowledge Discovery in Database*
M2M - *Model to Model*
MITM - *Man in the Middle* **(**attack**)**
ML - *Machine Learning*
MM - *Markov Model*
NIST - *National Institute of Standards and Technology*
NN - *Neural Networks*
NSA - *National Security Agency*
NVD - *National Vulnerability Database*
PSIM - *Physical Security Information Management*
SCADA - *Supervisory Control and Data Acquisition*
SIEM - *Security Information and Event Management*
SMS - *Security Management Software*
SOC - *Security Operations Centers*
UML – *Unified Modeling Language*
XML - *Extensible Markup Language*

References

[1] Romanosky S. (2016). Examining the costs and causes of cyber incidents, Journal of Cybersecurity, Volume 2, Issue 2, Pages 121–135. Available at https://doi.org/10.1093/cybsec/tyw001 [Accessed 03 March 2018]

[2] Juniper Research (2015). Cybercrime will cost businesses over $2 trillion by 2019. [Online]. Available at: https://www.juniperresearch.com/press/press-releases/cybercrime-cost-businesses-over-2trillion [Accessed 03 March 2018].

[3] Wee, S. and Oreskovic, A. (2011). Google reveals Gmail hacking, says likely from China. Available at: http://www.reuters.com/article/us-google-hacking-idUSTRE7506U320110602 [Accessed 03 March 2018]

[4] Kushner, D. (2013). The Real Story of Stuxnet. IEEE Spectrum, 53(3), pp. 48-50.

[5] Holloway, M. (2015). Stuxnet Worm Attack on Iranian Nuclear Facilities. [Online] Large.stanford.edu. Available at: http://large.stanford.edu/courses/2015/ph241/holloway1/ [Accessed 03 March 2018].

[6] W. Broad, J. Markoff, and D. Sanger, The New York Times, "Israeli Test on Worm Called Crucial in Iran Nuclear Delay," [Online]. Available at http://www.bbc.com/news/technology-10757263 [Accessed 15 November 2018]

[7] J. Fildes, British Broadcasting Corporation (BBC) (2010). What is Wikileaks? [Online]. Available at http://www.bbc.com/news/technology-10757263 [Accessed 12 March 2018]

[8] N. Khomami, The Guardian (2018). Judge refuses to withdraw Julian Assange arrest warrant. [Online]. Available at https://www.theguardian.com/media/2018/feb/13/judge-refuses-to-withdraw-julian-assange-arrest-warrant [Accessed 12 March 2018]

[9] British Broadcasting Corporation (BBC) (2014). Edward Snowden: Leaks that exposed US spy programme. Available at http://www.bbc.com/news/world-us-canada-23123964 [Accessed 03 March 2018]

[10] France24 (2015). The global fallout of the Ashley Madison hack. [Online]. Available at http://www.france24.com/en/20150820-global-fall-out-ashley-madison-hack [Accessed 12 March 2018]

[11] S. Thomsen, Business Insider Australia (2015). Extramarital affair website Ashley Madison has been hacked and attackers are threatening to leak data online. [Online]. Available at http://www.businessinsider.com/cheating-affair-website-ashley-madison-hacked-user-data-leaked-2015-7?r=US&IR=T&IR=T [Accessed 12 March 2018]

[12] R. Bilton, British Broadcasting Corporation (BBC) (2016). Panama Papers: Mossack Fonseca leak reveals elite's tax havens. BBC news. [Online]. Available at:

http://www.bbc.com/news/world-35918844 [Accessed 03 March 2018]

[13] EMC Digital Universe and IDC (2014). The Digital Universe of Opportunities: Rich Data and the Increasing Value of the Internet of Things. [Online]. Available at https://www.emc.com/leadership/digital-universe/2014iview/executive-summary.htm [Accessed 12 March 2018]

[14] IEEE, Syntegrity (2017), "Artificial Intelligence and Machine Learning Applied to Cybersecurity" presented in Washington DC, USA, 6th-8th October 2017, [Online]. Available at: https://www.ieee.org/content/dam/ieee-org/ieee/web/org/about/industry/ieee_confluence_report.pdf?utm_source=lp-link-text&utm_medium=industry&utm_campaign=confluence-paper [Accessed 20 March 2018].

[15] D. Shackleford (2016), SANS 2016 Security Analytics Survey, SANS Institute. [Online]. Available at: https://www.sans.org/reading-room/whitepapers/analyst/2016-security-analytics-survey-37467 [Accessed 03 March 2018]

[16] A. Buczal, E. Guven (2016), "A Survey of Data Mining and Machine Learning Methods for Cybersecurity Intrusion Detection", in IEEE Communications Surveys & Tutorials, Vol. 18. No. 2

[17] F. Flammini, A. Gaglione, N. Mazzocca, and C. Pragliola (2018). DETECT: a novel framework for the detection of attacks to critical infrastructures. In: Safety, Reliability and Risk Analysis: Theory, Methods and Applications - Proceedings of ESREL'08, pp. 105-112 (CRC Press, Taylor & Francis Group), London.

[18] Internet World Stats (2018). Internet Usage Statistics, The Internet Big Picture [Online]. Available at https://www.internetworldstats.com/stats.htm [Accessed 12 September 2018]

[19] M.R. Endsley, Design and evaluation for situation awareness enhancement. In: Proceedings of the Human Factors Society 32nd Annual Meeting. Santa Monica CA, USA; 1988. pp. 97-101.

[20] U. Frank, J. Brynielsson (2014). Cyber Situational Awareness – A systematic review of literature. Computer Security 46, 18–31 (2014), Stockholm, Sweden.

[21] J. Cannady, "Artificial Neural Networks for Misuse detection", in Proc. 1998 Nat. Inf. Syst. Secur. Conf., Arlington, VA, USA, 1998, pp. 443-356

[22] R. P. Lippmann and R. K. Cunningham, "Improving intrusion detection performance using keyword selection and neural networks", Computer Networks, vol. 34, pp. 597–603, 2000

[23] A. Bivens, C. Palagiri, R. Smith, B. Szymanski, and M. Embrechts, "Network-based intrusion detection using neural networks," Intelligent Engineering Systems Artificial

Neural Networks, vol. 12, no. 1, pp. 579–584, 2002

[24] E. K. Viegas, A. O. Santin, and L. S. Oliveira, "Toward a reliable anomaly-based intrusion detection in real-world environments", Computer Networks, vol. 127, pp. 200–216, 2017

[25] H. Hindy, D. Brosset, E. Bayne, A. Seeam, C. Tachtatzis, R. Atkinson, and X. Bellekens. (2018). A Taxonomy and Survey of Intrusion Detection System Design Techniques, Network Threats and Datasets. [Online]. Available at https://doi.org/10.1145/nnnnnnn.nnnnnnn [Accessed on 10 August 2018]

[26] Eugene Charniak (1991), Bayesian networks without tears: making Bayesian networks more accessible to the probabilistically unsophisticated, AI Magazine, v.12 n.4, pp.50-63

[27] C. Livadas, R. Walsh, D. Lapsley, and W. Strayer, "Using machine learning techniques to identify botnet traffic," in Proc 31st IEEE Conf. Local Computer Networks, 2006, pp. 967–974.]

[28] F. Jemili, M. Zaghdoud, and A. Ben, "A framework for an adaptive intrusion detection system using Bayesian network," in Proc. IEEE Intell. Secur. Informat., 2007, pp. 66–70

[29] P. Xie, J. H. Li, X. Ou, P. Lui, R. Levy (2010). "Using Bayesian Networks for Cybersecurity Analysis" in IEEE/IFIP International Conference on Dependable Systems & Networks (DNS) presented in Chicago, IL, USA, June 28th – July 1st 2010.

[30] S. Chockalingam, W. Peters, A. Teixeira, P. van Gelder (2017). Bayesian Network Models in Cybersecurity: A Systematic Review, Delf University of Technology, 2017.

[31] G. Modelo-Howard, S. Bagchi, G. Lebanon. Determining Placement of Intrusion Detectors for a Distributed Application thought Bayesian Network Modeling, School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN, USA, 2006.

[32] M. Frigault, L Wang, A. Singhal, S. Jajodia (2008). Measuring Network Security using Dynamic Bayesian Network, ACM, Alexandria, VA, USA, 2008.

[33] S. Abraham, S. Nair (2015). "A predictive framework for cybersecurity analytics using attack graphs" in International Journal of Computer Networks & Communications (IJCNC) Vol.7, No.1, Dallas, TX, USA, January 2015.

[34] D. M. Nicol, W. H. Sanders, K.S. Trivedi, "Model-Based Evaluation: From Dependability to Security" in IEEE Transactions on Dependable and Secure Computing, Vol. 1, No. 1, January-March 2004.

[35] J. Shin, H. Son, R. Khalil, G. Heo. "Development of a cybersecurity risk model using Bayesian Networks" in Reliability Engineering and System Safety 134, 2015, pp. 208-217.

[36] S. Cheung, U. Lindqvist and M. W. Fong. "Modeling Multistep Cyberattacks for Scenario Recognition" in Proceedings of the Third DARPA Information Survivability Conference and Exposition (DISCEX III), Washington D.C., USA, April 22–24, 2003, Volume I, pages 284–292

[37] F. Flammini, A. Gaglione, F. Otello, A. Pappalardo, C. Pragliola, A. Tedesco (2010). Towards Wireless Sensor Networks for Railway Infrastructure Monitoring, Ansaldo STS Italy, Università di Napoli Federico II, 2010.

[38] A. Gaglione (2009). Threat Analysis and Detection in Critical Infrastructure Security, Università di Napoli Federico II, Comunità Europea Fondo Sociale Europeo, November 2009.

[39] F. Flammini, A. Gaglione, N. Mazzocca, V. Moscato, C. Pragliola (2009). "Online Integration and reasoning for multi-sensor data to enhance infrastructure surveillance" in Journal of Information Assurance and Security 4, pp. 183-191, 2009.

[40] F. Flammini, A. Gaglione, N. Mazzocca, V. Moscato, C. Pragliola (2009). "Wireless Sensor Data Fusion for Critical Infrastructuer Security" in CISIS, Springer, Berlin Germany, 2009, pp. 92-99.

[41] B. Schneier (1999). "Attack Trees" in Dr. Dobb's Journal, pp. 21-22, 24, 26, 28-29, December 1999. [Online]. Available at: https://www.schneier.com/academic/archives/1999/12/attack_trees.html [Accessed 20 March 2018].

[42] A. Bobbio, L. Portinale, M. Minichino, E. Ciancamerla (2001). "Improving the analysis of dependable systems by mapping fault trees into Bayesian Networks" in Reliability Engineering and System Safety, 71, Rome, Italy, 2001, pp. 249-260.

[43] M. Gribaldo, M. Iacono, S. Marrone (2015). "Exploiting Bayesian Networks for the Analysis of Combined Attack Trees" in Electronic Notes in Theoretical Computer Science, 310, Elsevier B.V., 2015, pp 91-11

[44] Stefn Einarsson and Marvin Rausand. An approach to vulnerability analysis of complex industrial systems. Risk Analysis, 18(5):535–546, 1998.

[45] Kenneth S. Edge, George C. Dalton, II, Richard A. Raines, and Robert F. Mills. Using attack and protection trees to analyze threats and defenses to homeland security. In Proceedings of the 2006 IEEE Conference on Military Communications, MILCOM'06, pages 953–959, Piscataway, NJ, USA, 2006. IEEE Press.

[46] G. Park, C. K. Lee, J. G. Choi, D. H. Kim, Y. J. Lee, K. Kwon. "Cybersecurity Analysis by Attack Trees for a Reactor Protection System" in Transactions of the Korean Nuclear Society Autumn Meeting, PyeongChang, Korea, October 30th-31st, 2008

[47] S. Mauw and M. Oostdijk. Foundations of attack trees. In International Conference on Information Security and Cryptology ICISC 2005. LNCS 3935, pages 186–198. Springer, 2005.

[48] Symantec Corporation (2017), The Internet Security Threat Report (ISTR) 2017. [Online]. Available at https://www.symantec.com/content/dam/symantec/docs/reports/istr-22-2017-en.pdf [Accessed 13 March 2018]

[49] Pentest Standard (2014). Exploitation. [Online]. Available at http://www.pentest-standard.org/index.php/Exploitation#Anti-Virus [Accessed 03 March 2018]

[50] A. Krenker, J. Bester, A. Kos (2011), "Introduction to Artificial Neural Networks", in Artificial Neural Networks – Methodological Advances and Biomedical Applications, Prof. Kenji Suzuki (Ed.) InTech.

[51] Vivek Shandilya, Chris B. Simmons, and Sajjan Shiva (2014), "Use of Attack Graphs in Security Systems," Journal of Computer Networks and Communications, vol. 2014

[52] Chris Salter, O. Sami Saydjari, Bruce Schneier, Jim Wallner (1998), "Toward a Secure System Engineering Methodology", in New Security Paradigms Workshop (NSPW), pp. 2-10

[53] TechTarget (2017). Backdoor. [Online]. Available at http://searchsecurity.techtarget.com/definition/back-door [Accessed 03 March 2018]

[54] C. Paar, J. Pelzl, B. Preneel (2010). "Introduction to Cryptography and Data Security" in Understanding Cryptography: A Textbook for Students and Practitioners, Springer 2010, pp. 1-23

[55] National Institute of Standards and Technology (NIST) (2018). Vulnerability Metrics. [Online]. Available at https://nvd.nist.gov/vuln-metrics/cvss [Accessed 10 March 2018]

[56] CWE (2017). About CWE. [Online]. Available at https://cwe.mitre.org/about/index.html [Accessed 03 March 2018]

[57] OWASP (2015). Credential Stuffing. [Online]. Available at https://www.owasp.org/index.php/Credential_stuffing [Accessed 12 March 2018]

[58] C. C. Aggarwal (2015) "An Introduction to Data Mining" in Data Mining: The Textbook, Springer 2015, pp 1-6

[59] Defense Advanced Research Projects Agency (DARPA) (2018). About DARPA. [Online]. Available at https://www.darpa.mil/about-us/about-darpa [Accessed 12 March 2018]

[60] Pentest Standard (2014). Encoding. [Online]. Available at http://www.pentest-standard.org/index.php/Exploitation#Encoding [Accessed 03 March 2018]

[61] TechTarget (2017). Computer Exploit. [Online]. Available at http://searchsecurity.techtarget.com/definition/exploit [Accessed 03 March 2018]

[62] CISCO Systems (2002). Evolution of the Firewall Industry. [Online]. Available at https://docstore.mik.ua/univercd/cc/td/doc/product/iaabu/centri4/user/scf4ch3.htm [Accessed 03 March 2018]

[63] E. Cole, S. Northcutt. Honeypots: A Security Manager's Guide to Honeypots. [Online]. Available at https://www.sans.edu/cyber-research/security-laboratory/article/honeypots-guide [Accessed 18 March 2018]

[64] TrendMicro (2018). Industrial Control System. [Online]. Available at https://www.trendmicro.com/vinfo/us/security/definition/industrial-control-system [Accessed 12 March 2018]

[65] G. Fortino, A. Molisch, L. Shafer, et al (2018) "Introduction to the Internet of Things" in Internet of Things A to Z Technologies and Applications, IEEE Press, Wiley 2018, pp. 3-10

[66] TechTarget (2018). Intrusion Detection System (IDS). [Online]. Available at http://searchsecurity.techtarget.com/definition/intrusion-detection-system [Accessed 10 March 2018]

[67] N. Grebennikov (2007). Keyloggers: How they work and how to detect them (Part 1). [Online]. Available at https://securelist.com/keyloggers-how-they-work-and-how-to-detect-them-part-1/36138/ [Accessed 12 March 2018]

[68] Expert Systems (2018), What is Machine Learning? A definition. [Online]. Available at http://www.expertsystem.com/machine-learning-definition [Accessed 03 March 2018]

[69] TechTarget (2016). Malware (malicious software). [Online]. Available at http://searchsecurity.techtarget.com/definition/malware [Accessed 03 March 2018]

[70] OWASP (2015). Man in the Middle Attack. [Online]. Available at https://www.owasp.org/index.php/Man-in-the-middle_attack [Accessed 03 March 2018]

[71] NSA, CSS (2018). About Us. [Online]. Available at https://www.nsa.gov/about/ [Accessed 12 March 2018]

[72] OWASP (2009). Sniffing Application Traffic Attack. [Online]. Available at https://www.owasp.org/index.php/Sniffing_application_traffic_attack [Accessed 12 March 2018]

[73] TechTarget (2005). Payload. [Online]. Available at http://searchsecurity.techtarget.com/definition/payload [Accessed 03 March 2018]

[74] D. Kennedy, J. O'Gorman, D. Kearns, M. Aharoni, "1 – The absolute basics of Penetration Testing", in Metasploit, The Penetration Tester's Guide, No Starch Press, San Francisco CA, USA, 2011, pp. 2-4

[75] H. Mouratidis, P. Giorgini, G. Manson (2004) 'Using Security Attack Scenarios to Analyse Security During Information Systems Design', Proceedings International Conference on Enterprise Information Systems, Porto-Portugal, pp. 10-17.

[76] D. Swift (2006) A Practical Application of SIM/SEM/SIEM Automating Threat Identification, SANS Institute (2006), pp. 2-6. [Online]. Available at https://www.sans.org/reading-room/whitepapers/logging/practical-application-sim-sem-siem-automating-threat-identification-1781 [Accessed 06 March 2018]

[77] TechTarget (2018). Security Information and Event Management (SIEM). [Online]. Available at http://searchsecurity.techtarget.com/definition/security-information-and-event-management-SIEM [Accessed 03 March 2018]

[78] OWASP (2014). Session Hijacking Attack. [Online]. Available at https://www.owasp.org/index.php/Session_hijacking_attack [Accessed 03 March 2018]

[79] C. Hadnagy (2011), "Chapter 1: A look into the World of Social Engineer" in Social Engineering: The Art of Human Hacking, Wiley (2011), Indianapolis, IN, USA, pp. 16-31

[80] OWASP (2009). Spyware. [Online]. Available at https://www.owasp.org/index.php/Spyware [Accessed 03 March 2018]

[81] TechTarget (2005). SCADA (supervisory control and data acquisition). [Online]. Available at https://whatis.techtarget.com/definition/SCADA-supervisory-control-and-data-acquisition [Accessed 05 March 2018]

[82] OWASP (2018). About the Open Web Application Security Project. [Online]. https://www.owasp.org/index.php/About_The_Open_Web_Application_Security_Project [Accessed 03 March 2018]

[83] Totalförsvarets Forskningsinstitut (FOI) (2018). About FOI. [Online]. Available at https://www.foi.se/en/about-foi.html [Accessed 12 March 2018]

[84] OWASP (2009). Trojan Horse. [Online]. Available at https://www.owasp.org/index.php/Trojan_Horse [Accessed 03 March 2018]

[85] Grady B., James R., Ivar J. (2005), Unified Modeling Language User Guide, The, 2nd Edition, Addison-Wesley, Boston MA, USA, 2005, pp 496

[86] OWASP (2010). Computer Viruses. [Online]. Available at https://www.owasp.org/index.php/Computer_Viruses [Accessed 03 March 2018]

[87] CWE (2009). CWE Glossary. [Online]. Available at https://cwe.mitre.org/documents/glossary/index.html#Vulnerability [Accessed 03 March 2018]

[88] Pentest Standard (2014). Vulnerability Analysis. [Online]. Available at http://www.pentest-standard.org/index.php/Vulnerability_Analysis [Accessed 03 March 2018]

[89] TechTarget (2017). Computer Worm. [Online]. Available at http://searchsecurity.techtarget.com/definition/worm [Accessed 03 March 2018]

[90] Kaspersky (2018). What is a Zero Day Vulnerability? [Online]. Available at https://www.kaspersky.com/resource-center/definitions/zero-day-exploit [Accessed 03 March 2018]

[91] OWASP (2017). Top 10 – 2017. [Online]. Available at https://www.owasp.org/index.php/Top_10_2017-Top_10 [Accessed 13 March 2018]

[92] OWASP (2017). Top 10 – 2017 A1- Injection. [Online]. Available at https://www.owasp.org/index.php/Top_10_2017-A1-Injection [Accessed 13 March 2018]

[93] OWASP (2017). Top 10 – 2017 A2- Broken Authentication and Session Management. [Online]. Available at https://www.owasp.org/index.php/Top_10_2017-A2-Broken_Authentication_and_Session_Management [Accessed 13 March 2018]

[94] OWASP (2017). Top 10 – 2017 A3-Sensitive Data Exposure. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A3-Sensitive_Data_Exposure [Accessed 13 March 2018]

[95] OWASP (2017). Top 10 – 2017 A4-XML External Entities (XXE). [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A4-XML_External_Entities_(XXE) [Accessed 13 March 2018]

[96] OWASP (2017). Top 10 – 2017 A5-Broken Access Control. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A5-Broken_Access_Control [Accessed 13 March 2018]

[97] OWASP (2017). Top 10 – 2017 A6-Security Misconfiguration. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A6-Security_Misconfiguration [Accessed 13 March 2018]

[98] OWASP (2017). Top 10 – 2017 A7-Cross-Site Scripting (XSS). [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A7-Cross-Site_Scripting_(XSS)

[Accessed 13 March 2018]

[99] OWASP (2017). Top 10 – 2017 A8-Insecure Deserialization. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A8-Insecure_Deserialization [Accessed 13 March 2018]

[100] OWASP (2017). Top 10 – 2017 A9-Using Components with Known Vulnerabilities. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A9-Using_Components_with_Known_Vulnerabilities [Accessed 13 March 2018]

[101] OWASP (2017). Top 10 – 2017 A10-Insufficient Logging & Monitoring. [Online]. Available at https://www.owasp.org/index.php/Top_10-2017_A10-Insufficient_Logging%26Monitoring [Accessed 13 March 2018]

[102] OWASP (2017). Top 10 – 2017 Insufficient Attack Protection. [Online]. Available at https://www.owasp.org/index.php/Top_10_2017-A7-Insufficient_Attack_Protection [Accessed 13 March 2018]

[103] OWASP (2017). Cross-Site Request Forgery (CSRF). [Online]. Available at https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF) [Accessed 13 March 2018]

[104] OWASP (2017). Top 10 – 2017 Underprotected APIs. [Online]. Available at https://www.owasp.org/index.php/Top_10_2017-A10-Underprotected_APIs [Accessed 13 March 2018]

[105] S. Rao (2011). Denial of Service attacks and mitigation techniques: Real time implementation with detailed analysis, SANS Institute (2011). [Online]. Available at: https://www.sans.org/reading-room/whitepapers/detection/denial-service-attacks-mitigation-techniques-real-time-implementation-detailed-analysi-33764 [Accessed 13 March 2018]